

SDMTSA_project

Chiara Genuardi

Introduction

The project analyses a univariate time series about an hourly count of people passing in front of a sensor installed in a public street in Australia with some missing values which have to be forecast.

The forecast is going to be performed with three main methods: - ARIMA; - unobserved components model (UCM); - machine learning method.

```
library(tidyverse)
library(lubridate)
library(forecast)
library(tsibble)
library(tseries)
library(moments)
library(purrr)
library(Kendall)
library(gridExtra)
library(KFAS)
library(xgboost)
```

Data Loading and Cleaning

The dataset contains 42767 rows for two columns. Those tell us: - time: the hour for the specific day (variable character); - value: the count of people passing the sensor (variable integer). Of the 42767, 1439 miss the information for the “value” column, to be forecasted.

```
data <- read.csv("student_dataset.csv")
head(data)
```

```
##           time value
## 1 2015-04-15 00:00:01    13
## 2 2015-04-15 01:00:01     2
## 3 2015-04-15 02:00:01     5
## 4 2015-04-15 03:00:01     3
## 5 2015-04-15 04:00:01     8
## 6 2015-04-15 05:00:01    27
```

```
glimpse(data)
```

```
## Rows: 42,767
## Columns: 2
## $ time <chr> "2015-04-15 00:00:01", "2015-04-15 01:00:01", "2015-04-15 02:00:~
## $ value <int> 13, 2, 5, 3, 8, 27, 30, 111, 260, 303, 299, 280, 320, 335, 273, ~
```

Time variable

As a first step the column “time” should be formatted with the actual information it is encoding, changing it to time variable. As a time zone is used “UCT”, so no daylight saving correction has to be apported later.

```
df <- data %>% mutate(  
  time = as.POSIXct(data$time, format="%Y-%m-%d %H:%M:%S", tz="UTC"),  
  year = year(time),  
  month = month(time),  
  day = day(time),  
  hour = hour(time)  
)
```

Columns specifying year, month, day and hour are added; minutes and seconds are not considered as the count is made hourly.

Completeness

Data collected goes from 15 April 2015 to 29 February 2020, covering almost 5 complete years.

```
summary(data)
```

```
##           time           value  
## Length :42767 Min. : 0.0  
## N.unique :42767 1st Qu.: 14.0  
## N.blank : 0 Median : 69.0  
## Min.nchar: 19 Mean :101.5  
## Max.nchar: 19 3rd Qu.:159.0  
##           Max. :906.0  
##           NAs :1439
```

```
nas_time <- sum(is.na(df$time))  
nas_value <- sum(is.na(df$value))
```

```
cat("Missing value for time variable ", nas_time, ".")
```

```
## Missing value for time variable 0 .
```

```
cat("\nMissing value for value variable ", nas_value, ".")
```

```
##
```

```
## Missing value for value variable 1439 .
```

```
tot <- nrow(df)  
perc <- (nas_value/tot)*100  
cat("Missing values are ", perc, "% of the total.")
```

```
## Missing values are 3.364744 % of the total.
```

As we have hourly data is checked whether there’s a row for every hour of every day of every month of every year.

```
full_seq <- seq(min(df$time), max(df$time), by = "hour")
```

```
expected <- tibble(time = full_seq) %>%  
  mutate(year = year(time), month = month(time), day = day(time), hour = hour(time))
```

```
observed_times <- df$time
```

```

expected <- expected %>%
  mutate(present = time %in% observed_times)

cat("Total expected timestamps:", nrow(expected), "\n")

## Total expected timestamps: 42767

cat("Present timestamps:", sum(expected$present), "\n")

## Present timestamps: 42767

cat("Absent timestamps:", sum(!expected$present), "\n")

## Absent timestamps: 0

missing_rows <- expected %>% filter(!present)
print(missing_rows)

## # A tibble: 0 x 6
## #   i 6 variables: time <dtm>, year <dbl>, month <dbl>, day <int>, hour <int>,
## #   present <lgl>

completeness_by_day <- df %>%
  group_by(year, month, day) %>%
  summarise(n_hours_present = n_distinct(hour),
            n_missing_value = sum(is.na(value)),
            .groups = "drop") %>%
  mutate(complete_day = n_hours_present == 24)

cat("Days with less than 24 rows:",
    sum(!completeness_by_day$complete_day), "\n")

## Days with less than 24 rows: 1

print(completeness_by_day %>% filter(!complete_day))

## # A tibble: 1 x 6
##   year month   day n_hours_present n_missing_value complete_day
##   <dbl> <dbl> <int>          <int>          <int> <lgl>
## 1  2020     2    29              23              23 FALSE

completeness_by_month <- df %>%
  group_by(year, month) %>%
  summarise(
    n_obs = n(),
    n_expected = days_in_month(make_date(first(year), first(month), 1)) * 24,
    n_missing_value = sum(is.na(value)),
    .groups = "drop"
  ) %>%
  mutate(n_rows_missing = n_expected - n_obs)

print(completeness_by_month, n = Inf)

## # A tibble: 59 x 6
##   year month n_obs n_expected n_missing_value n_rows_missing
##   <dbl> <dbl> <int>          <dbl>          <int>          <dbl>
## 1  2015     4   384          720              0            336
## 2  2015     5   744          744              0              0

```

##	3	2015	6	720	720	0	0
##	4	2015	7	744	744	0	0
##	5	2015	8	744	744	0	0
##	6	2015	9	720	720	0	0
##	7	2015	10	744	744	0	0
##	8	2015	11	720	720	0	0
##	9	2015	12	744	744	0	0
##	10	2016	1	744	744	0	0
##	11	2016	2	696	696	0	0
##	12	2016	3	744	744	0	0
##	13	2016	4	720	720	0	0
##	14	2016	5	744	744	0	0
##	15	2016	6	720	720	0	0
##	16	2016	7	744	744	0	0
##	17	2016	8	744	744	0	0
##	18	2016	9	720	720	0	0
##	19	2016	10	744	744	0	0
##	20	2016	11	720	720	0	0
##	21	2016	12	744	744	0	0
##	22	2017	1	744	744	0	0
##	23	2017	2	672	672	0	0
##	24	2017	3	744	744	0	0
##	25	2017	4	720	720	0	0
##	26	2017	5	744	744	0	0
##	27	2017	6	720	720	0	0
##	28	2017	7	744	744	0	0
##	29	2017	8	744	744	0	0
##	30	2017	9	720	720	0	0
##	31	2017	10	744	744	0	0
##	32	2017	11	720	720	0	0
##	33	2017	12	744	744	0	0
##	34	2018	1	744	744	0	0
##	35	2018	2	672	672	0	0
##	36	2018	3	744	744	0	0
##	37	2018	4	720	720	0	0
##	38	2018	5	744	744	0	0
##	39	2018	6	720	720	0	0
##	40	2018	7	744	744	0	0
##	41	2018	8	744	744	0	0
##	42	2018	9	720	720	0	0
##	43	2018	10	744	744	0	0
##	44	2018	11	720	720	0	0
##	45	2018	12	744	744	0	0
##	46	2019	1	744	744	0	0
##	47	2019	2	672	672	0	0
##	48	2019	3	744	744	0	0
##	49	2019	4	720	720	0	0
##	50	2019	5	744	744	0	0
##	51	2019	6	720	720	0	0
##	52	2019	7	744	744	0	0
##	53	2019	8	744	744	0	0
##	54	2019	9	720	720	0	0
##	55	2019	10	744	744	0	0
##	56	2019	11	720	720	0	0

```
## 57 2019    12  744      744          0          0
## 58 2020     1  744      744      744          0
## 59 2020     2  695      696      695          1
```

```
cat("\n--- RECAP COMPLETENESS ---\n")
```

```
##
```

```
## --- RECAP COMPLETENESS ---
```

```
cat("Total rows:", nrow(df), "\n")
```

```
## Total rows: 42767
```

```
cat("Expected rows:", length(full_seq), "\n")
```

```
## Expected rows: 42767
```

```
cat("Missing rows:", sum(!expected$present), "\n")
```

```
## Missing rows: 0
```

```
cat("Rows with missing 'value':", sum(is.na(df$value)), "\n")
```

```
## Rows with missing 'value': 1439
```

Considering the number of days for each month, the starting and ending data in the dataset, we have all expected rows. The dataset is complete.

Duplicates

```
dup_times <- df %>% filter(duplicated(time) | duplicated(time, fromLast = TRUE))
cat("Duplicated timestamps:", nrow(dup_times), "\n")
```

```
## Duplicated timestamps: 0
```

```
if (nrow(dup_times) > 0) {
  print(dup_times %>% arrange(time))
}
```

We can see that there's no duplicate rows; the usage of a timezone prevented the need to do some daylight saving corrections.

Continuity

The hourly sequence is fully regular across the entire observation period: the number of observed timestamps exactly matches the number of expected timestamps (42767).

```
full_seq <- seq(min(df$time), max(df$time), by = "hour")
cat("Expected timestamps for continuity:", length(full_seq), "\n")
```

```
## Expected timestamps for continuity: 42767
```

```
cat("Observed timestamps:", nrow(df), "\n")
```

```
## Observed timestamps: 42767
```

```
missing_timestamps <- setdiff(full_seq, df$time)
cat("Absent timestamps file:", length(missing_timestamps), "\n")
```

```
## Absent timestamps file: 0
```

Missing values

Let's check where missing values lie by year and month.

```
missing_by_year_month <- df %>%  
  group_by(year, month) %>%  
  summarise(n_obs = n(), missing = sum(is.na(value)), .groups = "drop") %>%  
  arrange(year, month)  
print(missing_by_year_month, n = Inf)
```

```
## # A tibble: 59 x 4  
##   year month n_obs missing  
##   <dbl> <dbl> <int>    <int>  
## 1  2015     4   384        0  
## 2  2015     5   744        0  
## 3  2015     6   720        0  
## 4  2015     7   744        0  
## 5  2015     8   744        0  
## 6  2015     9   720        0  
## 7  2015    10   744        0  
## 8  2015    11   720        0  
## 9  2015    12   744        0  
## 10 2016     1   744        0  
## 11 2016     2   696        0  
## 12 2016     3   744        0  
## 13 2016     4   720        0  
## 14 2016     5   744        0  
## 15 2016     6   720        0  
## 16 2016     7   744        0  
## 17 2016     8   744        0  
## 18 2016     9   720        0  
## 19 2016    10   744        0  
## 20 2016    11   720        0  
## 21 2016    12   744        0  
## 22 2017     1   744        0  
## 23 2017     2   672        0  
## 24 2017     3   744        0  
## 25 2017     4   720        0  
## 26 2017     5   744        0  
## 27 2017     6   720        0  
## 28 2017     7   744        0  
## 29 2017     8   744        0  
## 30 2017     9   720        0  
## 31 2017    10   744        0  
## 32 2017    11   720        0  
## 33 2017    12   744        0  
## 34 2018     1   744        0  
## 35 2018     2   672        0  
## 36 2018     3   744        0  
## 37 2018     4   720        0  
## 38 2018     5   744        0  
## 39 2018     6   720        0  
## 40 2018     7   744        0  
## 41 2018     8   744        0  
## 42 2018     9   720        0
```

```
## 43 2018    10   744      0
## 44 2018    11   720      0
## 45 2018    12   744      0
## 46 2019     1   744      0
## 47 2019     2   672      0
## 48 2019     3   744      0
## 49 2019     4   720      0
## 50 2019     5   744      0
## 51 2019     6   720      0
## 52 2019     7   744      0
## 53 2019     8   744      0
## 54 2019     9   720      0
## 55 2019    10   744      0
## 56 2019    11   720      0
## 57 2019    12   744      0
## 58 2020     1   744    744
## 59 2020     2   695    695
```

Missing values are concentrated in a single uninterrupted block, concerning the last two months of collected data; we can try to use data from previous years to forecast the last two months.

This has an important methodological implication: this is not a missing data imputation problem, but an out-of-sample forecasting problem. The 1439 values must be produced by forecasting forward from the pre-2020 training data, using the seasonal patterns identified in this EDA.

```
rle_na <- rle(is.na(df$value))
gap_lengths <- rle_na$lengths[rle_na$values == TRUE]
cat("Number of missing consecutive blocks:", length(gap_lengths), "\n")
```

```
## Number of missing consecutive blocks: 1
```

```
cat("Length of blocks (consecutive missing hours):\n")
```

```
## Length of blocks (consecutive missing hours):
```

```
print(table(gap_lengths))
```

```
## gap_lengths
```

```
## 1439
```

```
##      1
```

```
cat("Longest block of consecutive missings:", max(gap_lengths), "hours\n")
```

```
## Longest block of consecutive missings: 1439 hours
```

Data Exploration

At first the dataframe is converted in a tibble.

```
df_complete <- df %>%
  filter(!is.na(value))
```

```
df_ts <- df %>%
  select(time, value) %>%
  as_tsibble(index=time)
```

and its regularity is checked

```
has_gaps(df_ts)
```

```
## # A tibble: 1 x 1
##   .gaps
##   <lgl>
## 1 FALSE
```

```
count_gaps(df_ts)
```

```
## # A tibble: 0 x 3
## # i 3 variables: .from <dtm>, .to <dtm>, .n <int>
```

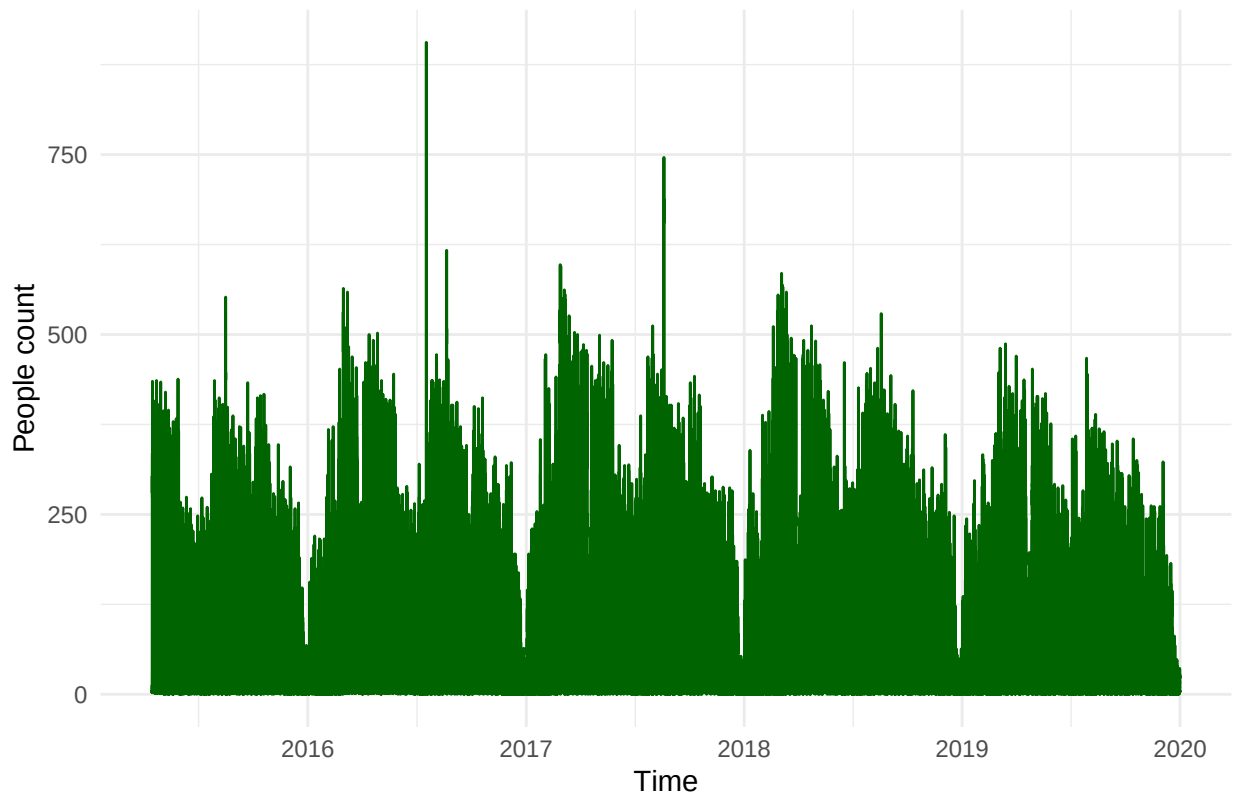
Whole time series

At hourly resolution, the series is dominated by a strong daily seasonality (near-zero counts at night, sharp daytime peaks), which appears as a dense filled band rather than a readable line at this time scale. An annual seasonal pattern is superimposed, with recurring troughs around late December/early January and elevated activity in spring and August. Two isolated extreme spikes (~900 and ~750 people) are visible around 2016 and 2017, well above the typical peak range (~400-500), and warrant further investigation as potential outliers before modelling as they could reflect a one-off public event or a sensor malfunction. The overall amplitude of daily peaks also appears to decline slightly in the later years of the series, hinting at a mild long-term downward trend, to be confirmed through formal decomposition.

```
wholets <- df_complete %>%
  ggplot(aes(x = time, y = value)) +
  geom_line(color = "darkgreen", linewidth = 0.5) +
  geom_area(fill = "darkgreen", alpha = 0.1) +
  labs(title = "Time series counts",
       x = "Time", y = "People count") +
  theme_minimal()

wholets
```

Time series counts

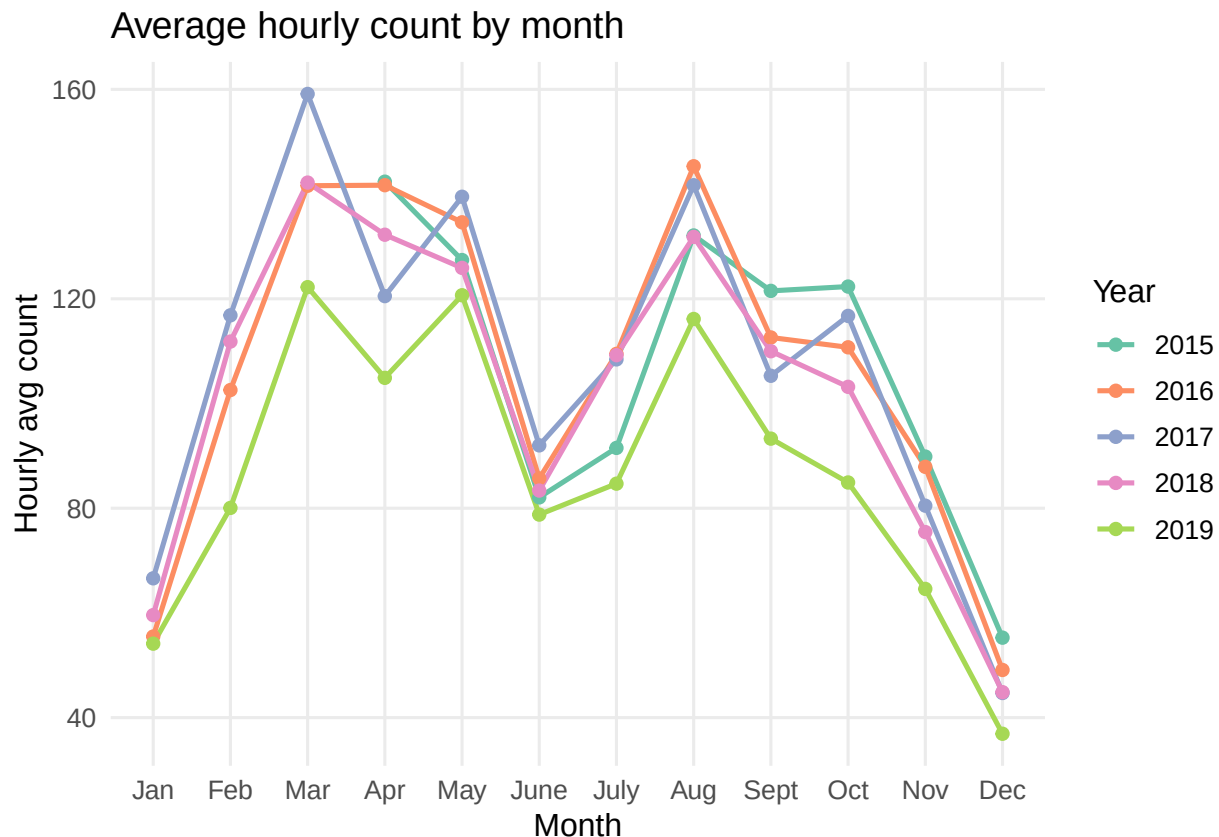


```
ggsave("slides_wholets.png", wholets, width = 10, height = 5, dpi = 300)
```

Aggregating by month reveals a consistent bimodal seasonal shape repeated across all five years: two peaks (March and August) and two troughs (June and December). This year-over-year consistency indicates a strong and stable annual seasonality, which is a favourable property for forecasting. The dip in June is a distinctive feature that interrupts what would otherwise be a smooth spring-to-summer rise, and may reflect a local calendar effect. While the seasonal shape is stable, the overall level is not: 2019 sits systematically below the other years, particularly at the March and August peaks, suggesting a downward trend in the series' baseline level over time.

```
hcount <- df %>%
  filter(!is.na(value)) %>%
  group_by(year, month) %>%
  summarise(mean_value = mean(value), .groups = "drop") %>%
  mutate(year = factor(year)) %>%
  ggplot(aes(x = month, y = mean_value, color = year, group = year)) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 1.8) +
  scale_x_continuous(breaks = 1:12,
                     labels = c("Jan", "Feb", "Mar", "Apr", "May", "June",
                                "July", "Aug", "Sept", "Oct", "Nov", "Dec")) +
  scale_color_brewer(palette = "Set2") +
  labs(title = "Average hourly count by month",
       x = "Month", y = "Hourly avg count", color = "Year") +
  theme_minimal(base_size = 12) +
  theme(panel.grid.minor = element_blank(),
        legend.position = "right")
```

```
hcount
```



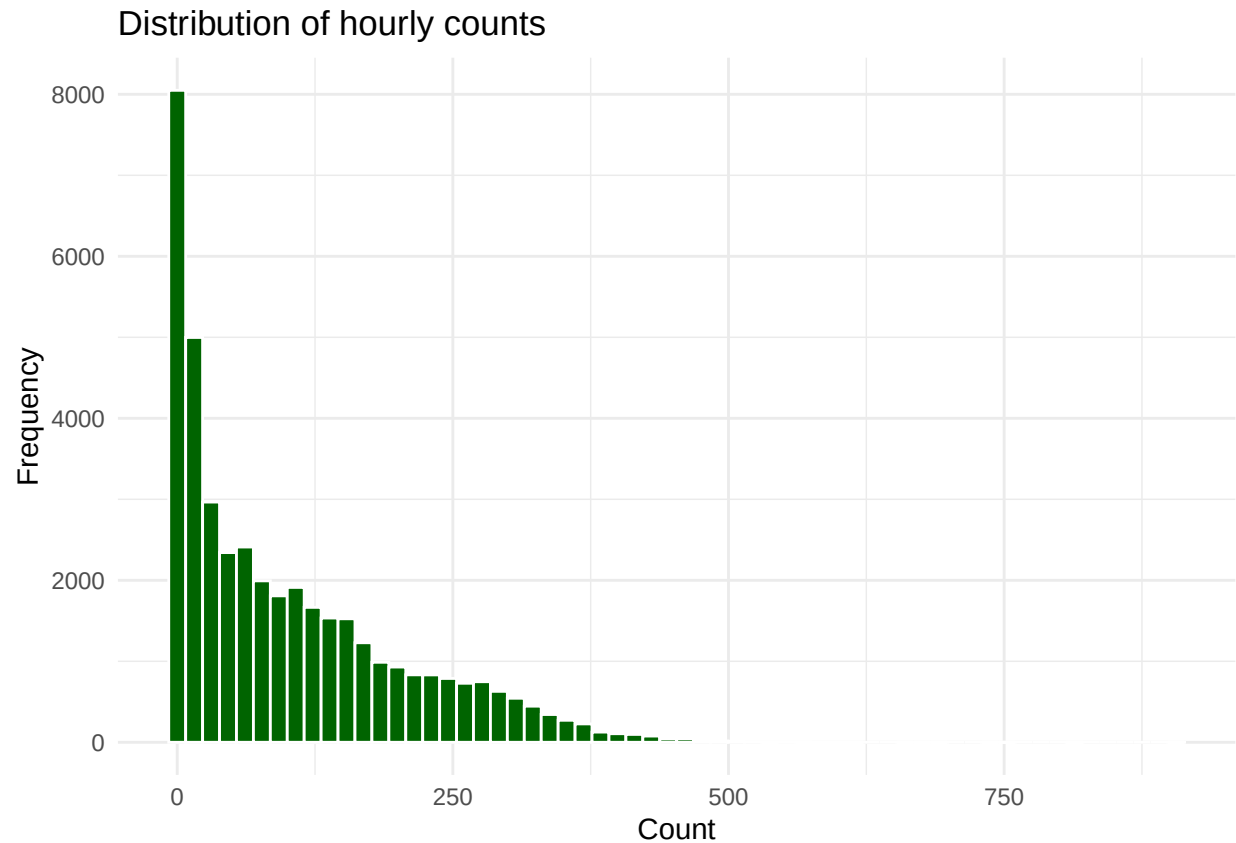
```
ggsave("slides_hcount.png", hcount, width = 10, height = 5, dpi = 300)
```

Distribution

The distribution of hourly counts is strongly right-skewed (skewness = 1.14), with a large mass of low values (2.66% exact zeros, concentrated at night) and a long tail of high daytime counts extending up to 906. Kurtosis (3.85) is close to the normal value of 3, indicating the tail is heavy but not extreme once the skew is accounted for. The Q-Q plot confirms clear departure from normality in the upper tail, consistent with the outliers identified earlier. Splitting the distribution into day (6-20h) and night (21-5h) periods reveals two distinct sub-populations: night counts are tightly concentrated near zero, while day counts form a much wider, right-skewed distribution of their own; this bimodal-like structure when pooling all hours together explains the shape of the overall histogram. The log-transformed distribution is visibly more symmetric (aside from the discrete spikes from exact zero/low counts), supporting the case for a variance-stabilising transformation before ARIMA fitting, formalised further below via Box-Cox.

```
distr <- df_complete %>%  
  ggplot(aes(x = value)) +  
  geom_histogram(bins = 60, fill = "darkgreen", color = "white") +  
  labs(title = "Distribution of hourly counts",  
        x = "Count", y = "Frequency") +  
  theme_minimal()
```

```
distr
```



```
ggsave("slides_distr.png", distr, width = 10, height = 5, dpi = 300)

cat("Skewness:", skewness(df_complete$value), "\n")

## Skewness: 1.143816

cat("Kurtosis:", kurtosis(df_complete$value), "\n")

## Kurtosis: 3.853965

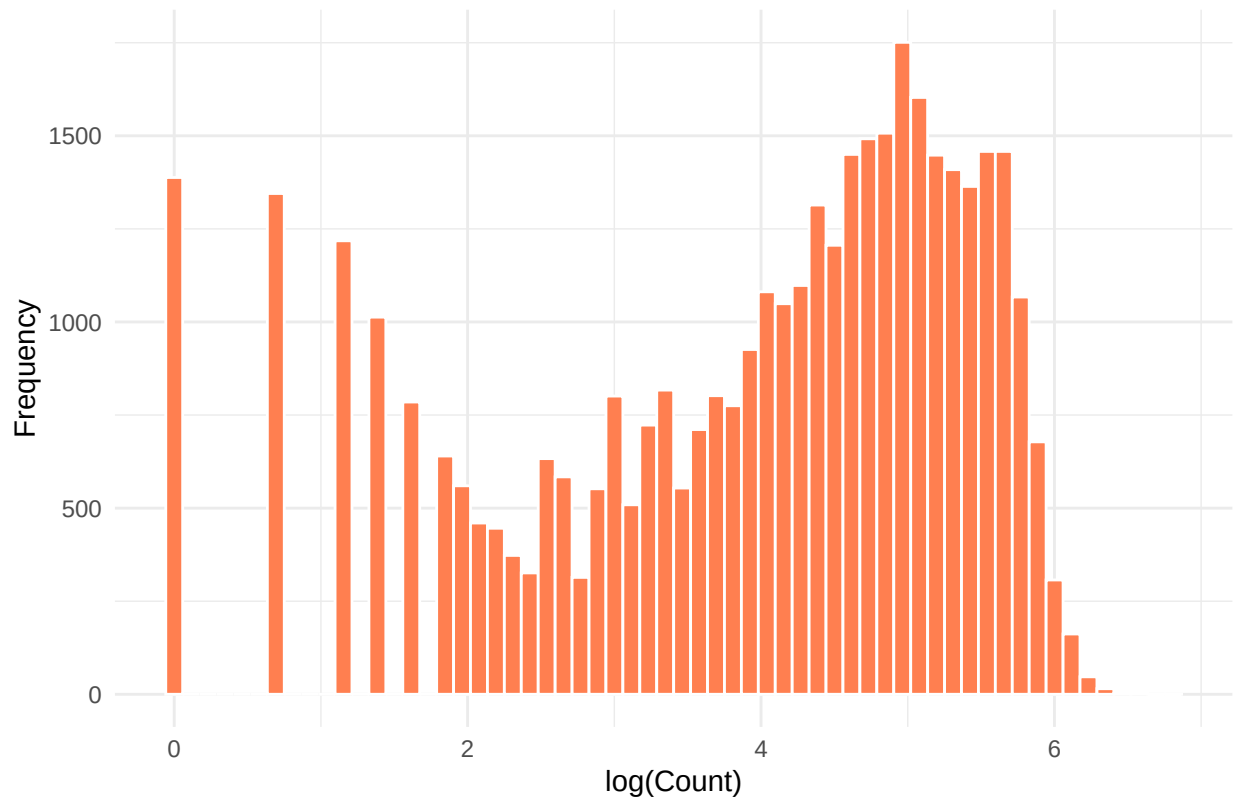
n_zeros <- sum(df_complete$value == 0)
cat("Zeros:", n_zeros, "(", round(n_zeros/nrow(df_complete)*100, 2), "% of total)\n")

## Zeros: 1100 ( 2.66 % of total)

logdistr <- df_complete %>%
  filter(value > 0) %>%
  ggplot(aes(x = log(value))) +
  geom_histogram(bins = 60, fill = "coral", color = "white") +
  labs(title = "Distribution of log(hourly counts)",
        x = "log(Count)", y = "Frequency") +
  theme_minimal()

logdistr
```

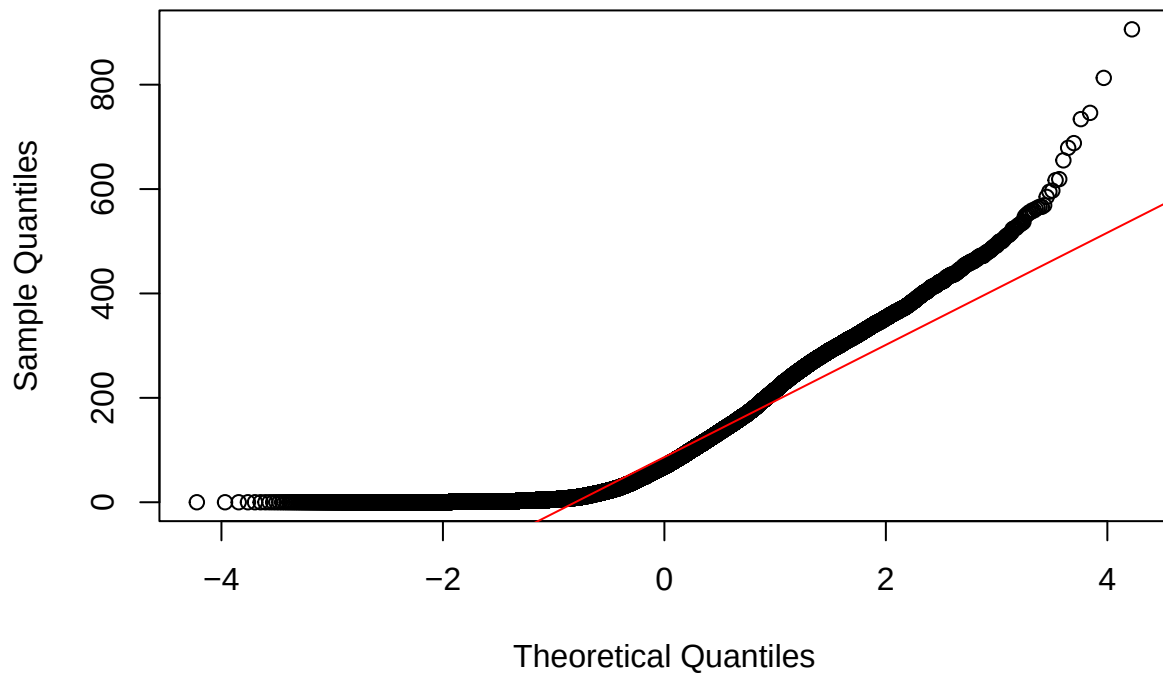
Distribution of log(hourly counts)



```
ggsave("slides_logdistr.png", logdistr, width = 10, height = 5, dpi = 300)
```

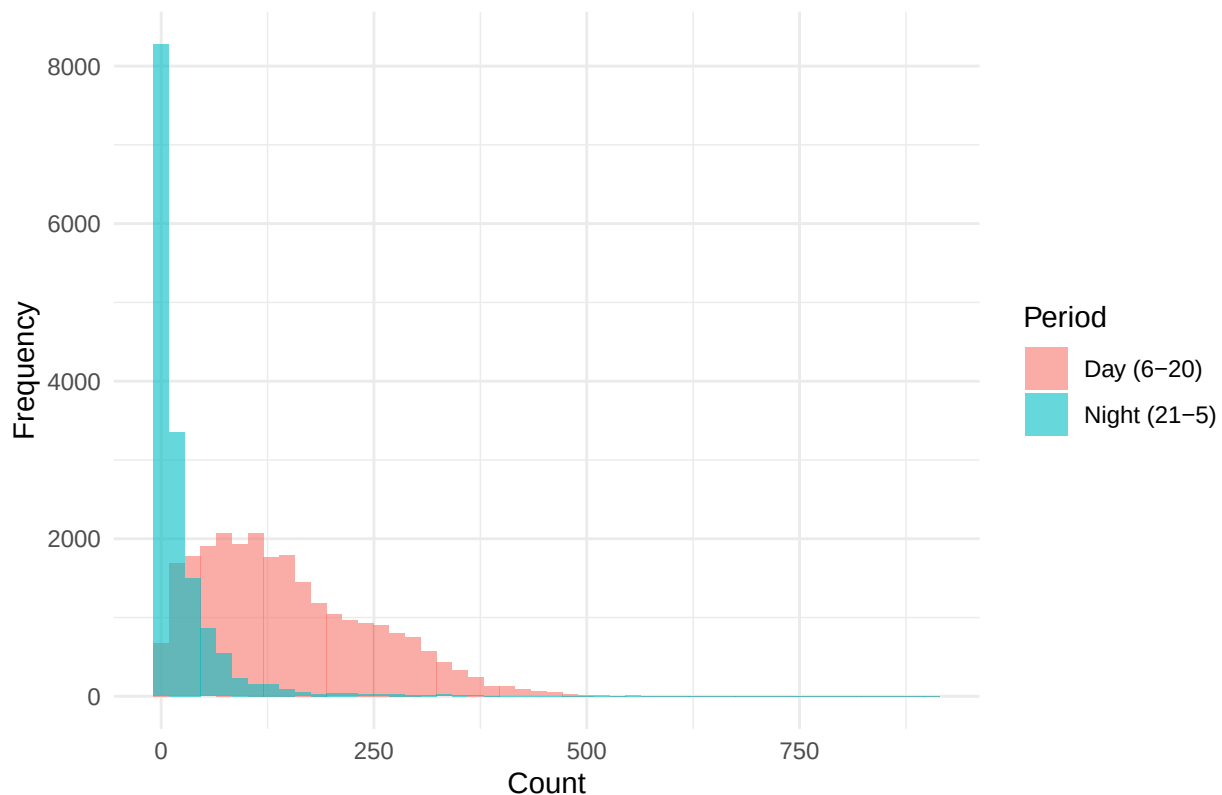
```
qqnorm(df_complete$value, main = "Q-Q Plot: Count")  
qqline(df_complete$value, col = "red")
```

Q-Q Plot: Count



```
dayvsnight <- df_complete %>%  
  mutate(period = ifelse(hour >= 6 & hour <= 20, "Day (6-20)", "Night (21-5)")) %>%  
  ggplot(aes(x = value, fill = period)) +  
  geom_histogram(bins = 50, alpha = 0.6, position = "identity") +  
  labs(title = "Distribution of counts: day vs night",  
       x = "Count", y = "Frequency", fill = "Period") +  
  theme_minimal()  
  
dayvsnight
```

Distribution of counts: day vs night



```
ggsave("slides_dayvsnight.png", dayvsnight, width = 10, height = 5, dpi = 300)
```

Time series decomposition

```
summary(df$value)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs
##      0.0   14.0    69.0   101.5   159.0   906.0   1439
```

```
cat("Standard deviation:", sd(df$value, na.rm = TRUE), "\n")
```

```
## Standard deviation: 103.1693
```

```
cat("Variance:", var(df$value, na.rm = TRUE), "\n")
```

```
## Variance: 10643.9
```

The hourly coefficient of variation ($CV = SD/mean$) is highest at night (up to 3.45 at 2 AM) and lowest around midday (around 0.5 at noon), showing that while night counts are small in absolute terms, they are relatively more volatile than daytime counts. The mean $\pm$ 1 SD band plot makes the heteroscedasticity visually explicit: the band is narrow overnight and widens substantially during the day, particularly in the afternoon peak hours where both the level and its variability are highest. The variance-vs-mean scatter plot shows a clear positive, roughly linear relationship between hourly mean and hourly variance across the 24 hours.

```
sd_by_hour <- df_complete %>%
  group_by(hour) %>%
  summarise(mean_value = mean(value), sd_value = sd(value),
```

```

    cv = sd_value / mean_value, .groups = "drop")

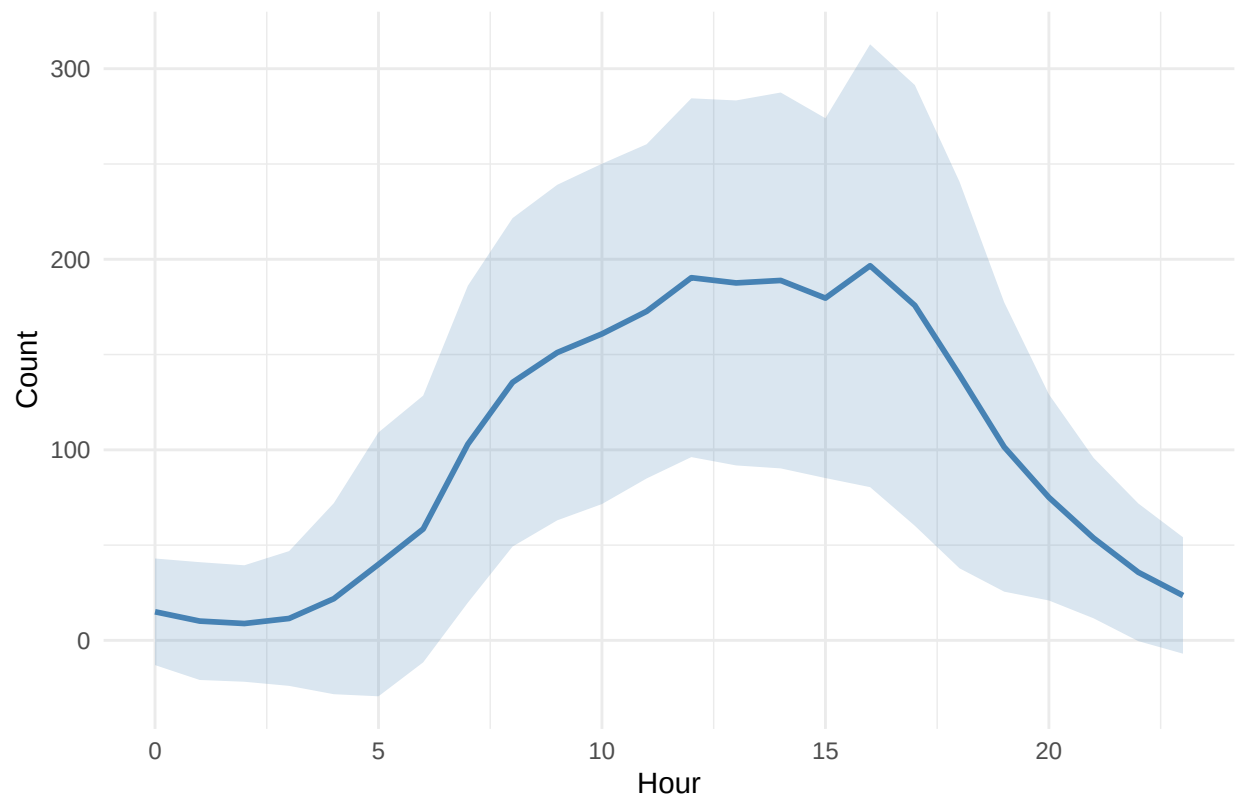
print(sd_by_hour, n = Inf)

## # A tibble: 24 x 4
##   hour mean_value sd_value   cv
##   <int>      <dbl>    <dbl> <dbl>
## 1     0       15.0      28.0 1.86
## 2     1       10.1      30.9 3.05
## 3     2        8.85     30.6 3.45
## 4     3       11.5      35.4 3.08
## 5     4       21.9      50.1 2.29
## 6     5       39.9      69.3 1.74
## 7     6       58.5      70.0 1.20
## 8     7      103.      83.2 0.808
## 9     8      135.      86.2 0.636
## 10    9      151.      88.0 0.583
## 11   10      161.      89.3 0.555
## 12   11      173.      87.7 0.508
## 13   12      190.      94.1 0.495
## 14   13      188.      95.8 0.510
## 15   14      189.      98.6 0.522
## 16   15      180.      94.4 0.526
## 17   16      197.     116. 0.591
## 18   17      176.     116. 0.658
## 19   18      139.     101. 0.728
## 20   19      101.      75.9 0.748
## 21   20       75.0      54.1 0.721
## 22   21       53.7      42.1 0.784
## 23   22       35.7      36.1 1.01
## 24   23       23.6      30.5 1.30

ggplot(sd_by_hour, aes(x = hour)) +
  geom_line(aes(y = mean_value), color = "steelblue", linewidth = 1) +
  geom_ribbon(aes(ymin = mean_value - sd_value, ymax = mean_value + sd_value),
    fill = "steelblue", alpha = 0.2) +
  labs(title = "Mean count by hour with +/- 1 SD band",
    x = "Hour", y = "Count") +
  theme_minimal()

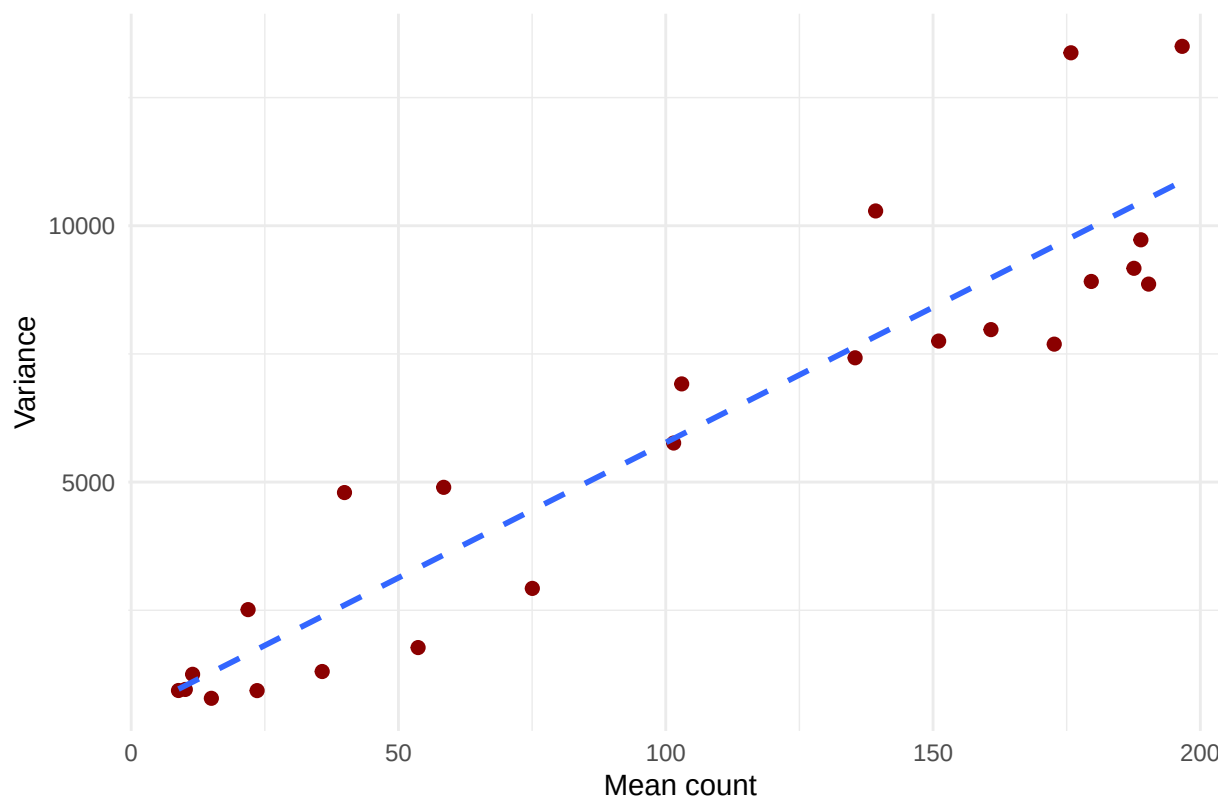
```

Mean count by hour with ± 1 SD band



```
ggplot(sd_by_hour, aes(x = mean_value, y = sd_value^2)) +  
  geom_point(size = 2, color = "darkred") +  
  geom_smooth(method = "lm", se = FALSE, linetype = "dashed") +  
  labs(title = "Variance vs Mean by hour",  
        x = "Mean count", y = "Variance") +  
  theme_minimal()
```

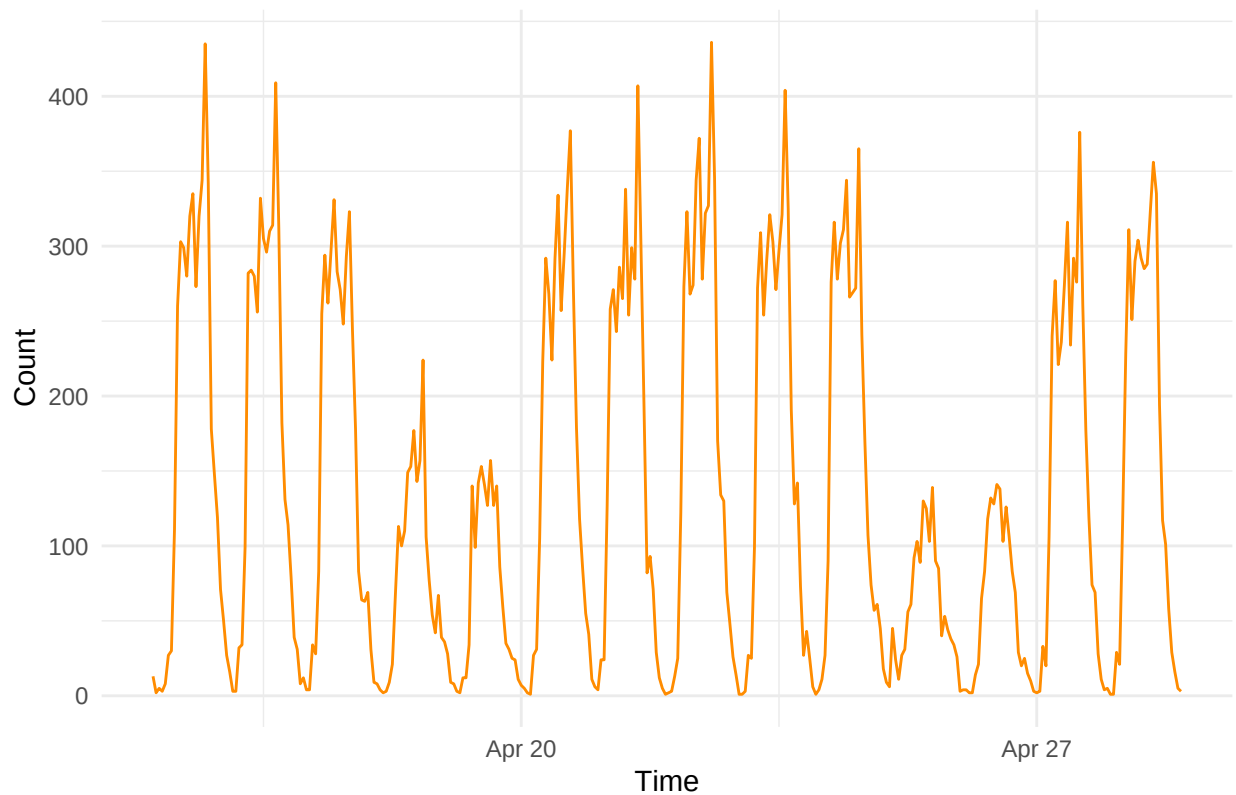
Variance vs Mean by hour



Zooming into a two-week window makes the daily cycle clearly visible as a sequence of sharp, roughly triangular peaks rather than the dense filled and seen at full scale. Each day shows a similar shape: near-zero counts overnight, a rise starting around 5-6 AM, a broad daytime plateau, and a decline into the evening, which is consistent with the average hourly profile shown earlier. Day-to-day variation in peak height is also visible, with somewhat lower peaks around the second weekend in the window, consistent with the weekday/weekend contrast already identified (lower average counts on Friday/Saturday/Sunday). This confirms at a finer time resolution what the aggregated plots suggested: a stable, repeating daily pattern modulated by a secondary weekly effect.

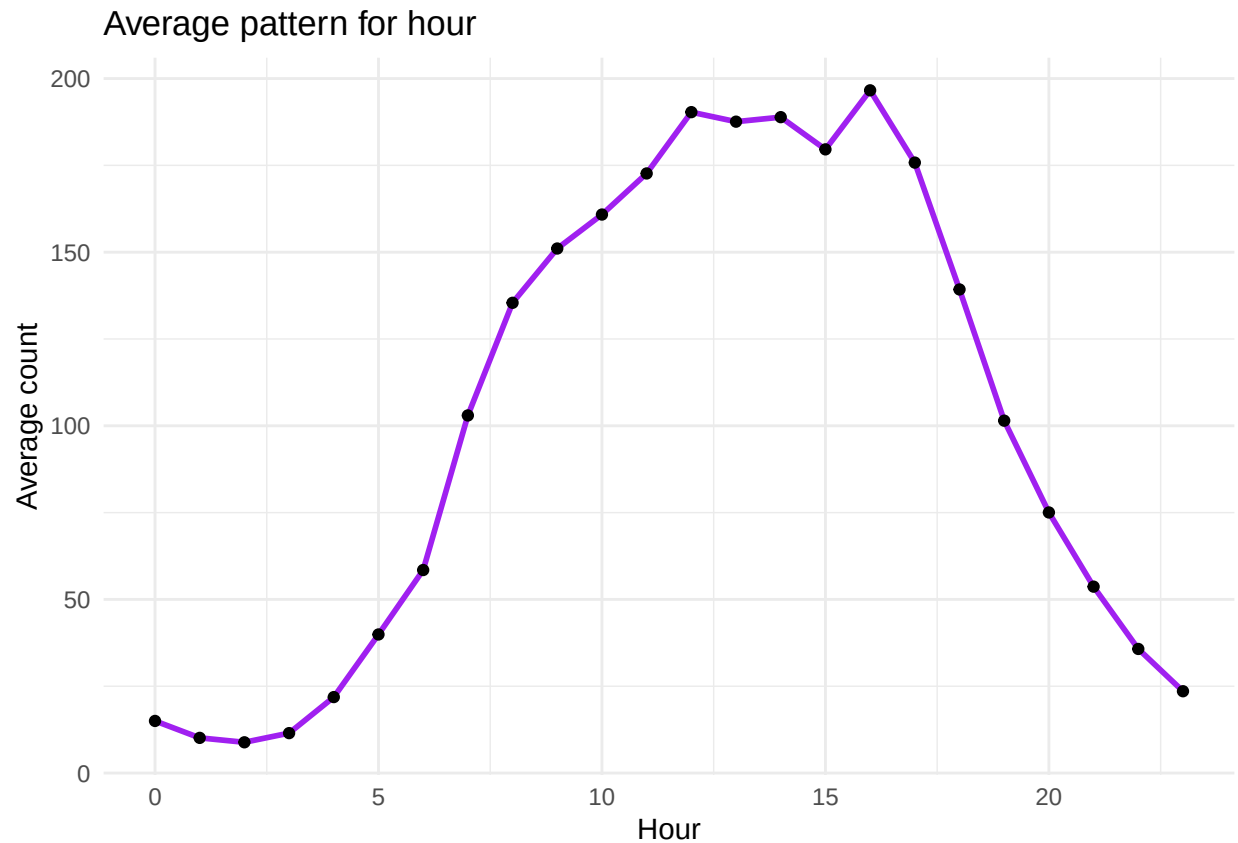
```
df %>%
  filter(time >= as.POSIXct("2015-04-15", tz="UTC"),
         time < as.POSIXct("2015-04-29", tz="UTC")) %>%
  ggplot(aes(x = time, y = value)) +
  geom_line(color = "darkorange") +
  labs(title = "Zoom: two weeks", x = "Time", y = "Count") +
  theme_minimal()
```

Zoom: two weeks



The average hourly profile shows the expected diurnal pattern of pedestrian activity: counts are near zero between midnight and 4 AM, rise sharply from 5 AM, and remain high and relatively flat between roughly 11 AM and 5 PM (peaking around 4 PM), before declining steadily into the evening. This single, smooth daily cycle (rather than the twin morning/evening commute peaks typical of transit data) suggests the sensor is more likely capturing general foot traffic than a commuter-driven pattern.

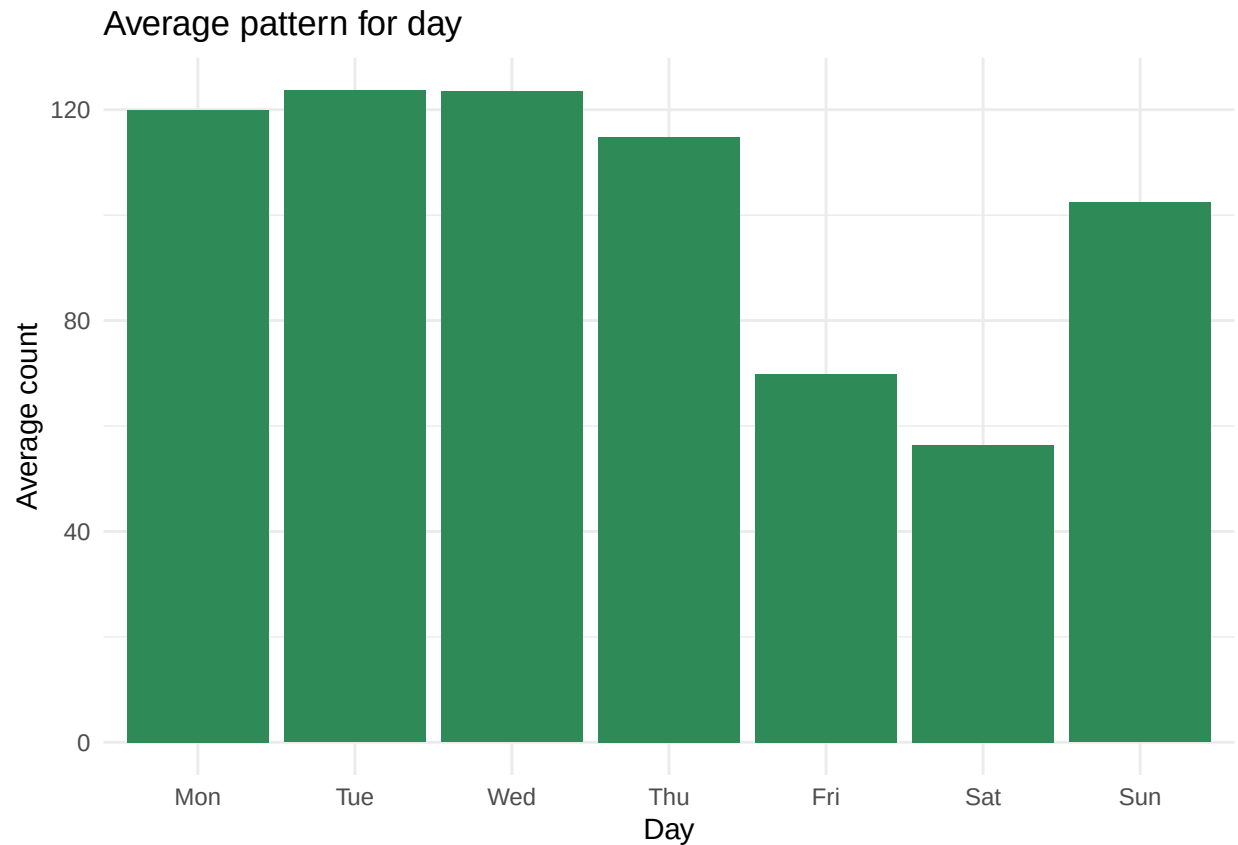
```
df %>%  
  group_by(hour) %>%  
  summarise(mean_value = mean(value, na.rm = TRUE)) %>%  
  ggplot(aes(x = hour, y = mean_value)) +  
  geom_line(color = "purple", linewidth = 1) +  
  geom_point() +  
  labs(title = "Average pattern for hour",  
        x = "Hour", y = "Average count") +  
  theme_minimal()
```



Average counts are clearly higher on weekdays (Monday-Thursday, ~115-125) than on the weekend, with a sharp drop on Friday, and the lowest levels on Saturday (~55) and a partial recovery on Sunday (~100). This weekly pattern is a strong argument for explicitly modelling weekly seasonality (period $168 = 24 \times 7$) in addition to the daily cycle, rather than relying on daily seasonality alone.

```
pattern <- df %>%
  mutate(wday = wday(time, label = TRUE, week_start = 1)) %>%
  group_by(wday) %>%
  summarise(mean_value = mean(value, na.rm = TRUE)) %>%
  ggplot(aes(x = wday, y = mean_value)) +
  geom_col(fill = "seagreen") +
  labs(title = "Average pattern for day",
       x = "Day", y = "Average count") +
  theme_minimal()

pattern
```

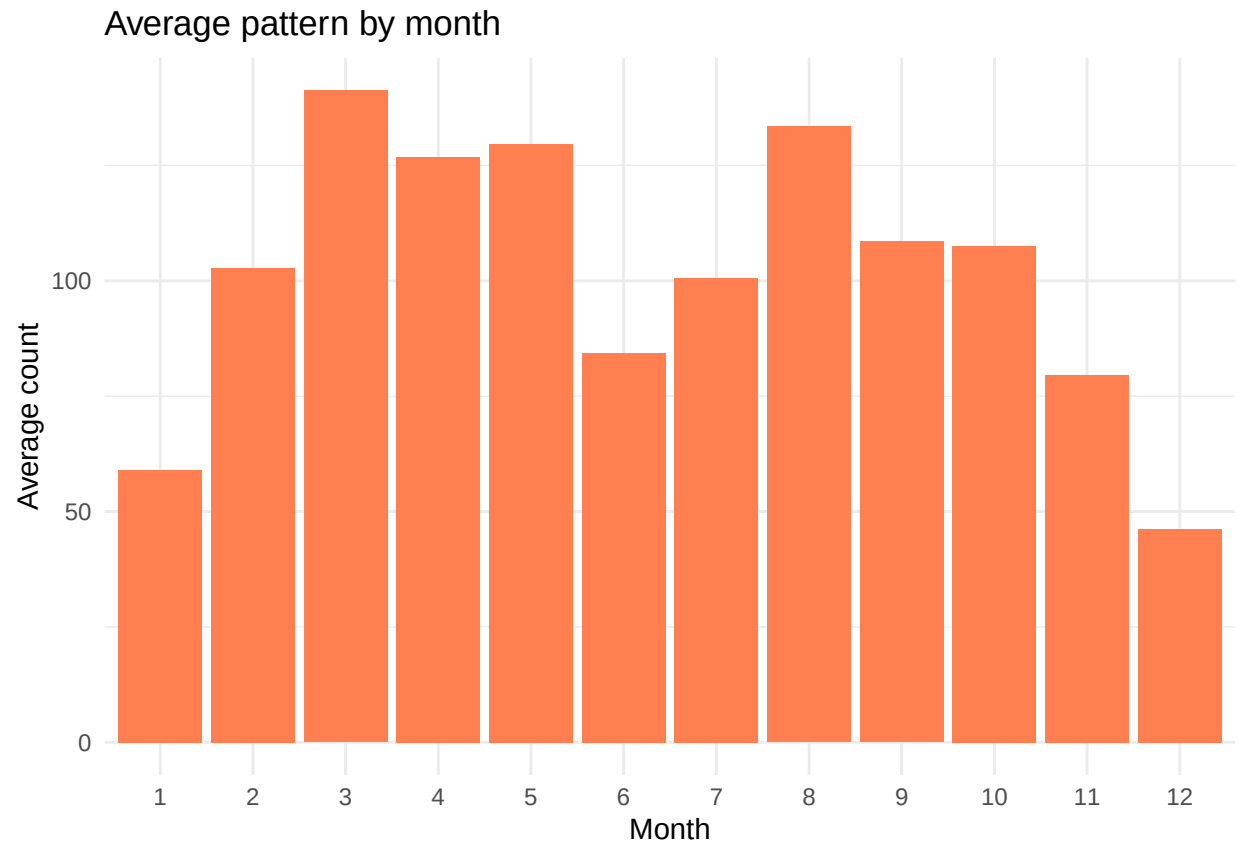


```
ggsave("slides_pattern.png", pattern, width = 10, height = 5, dpi = 300)
```

This view confirms, in a more compact form, the bimodal seasonal pattern already observed in the year-by-year monthly plot: highest average counts in March (~140) and August (~135), lowest in December (~45) and January (~55), with June forming a secondary local minimum (~85) between the two peaks.

```
patternm <- df %>%
  group_by(month) %>%
  summarise(mean_value = mean(value, na.rm = TRUE)) %>%
  ggplot(aes(x = factor(month), y = mean_value)) +
  geom_col(fill = "coral") +
  labs(title = "Average pattern by month", x = "Month", y = "Average count") +
  theme_minimal()
```

```
patternm
```



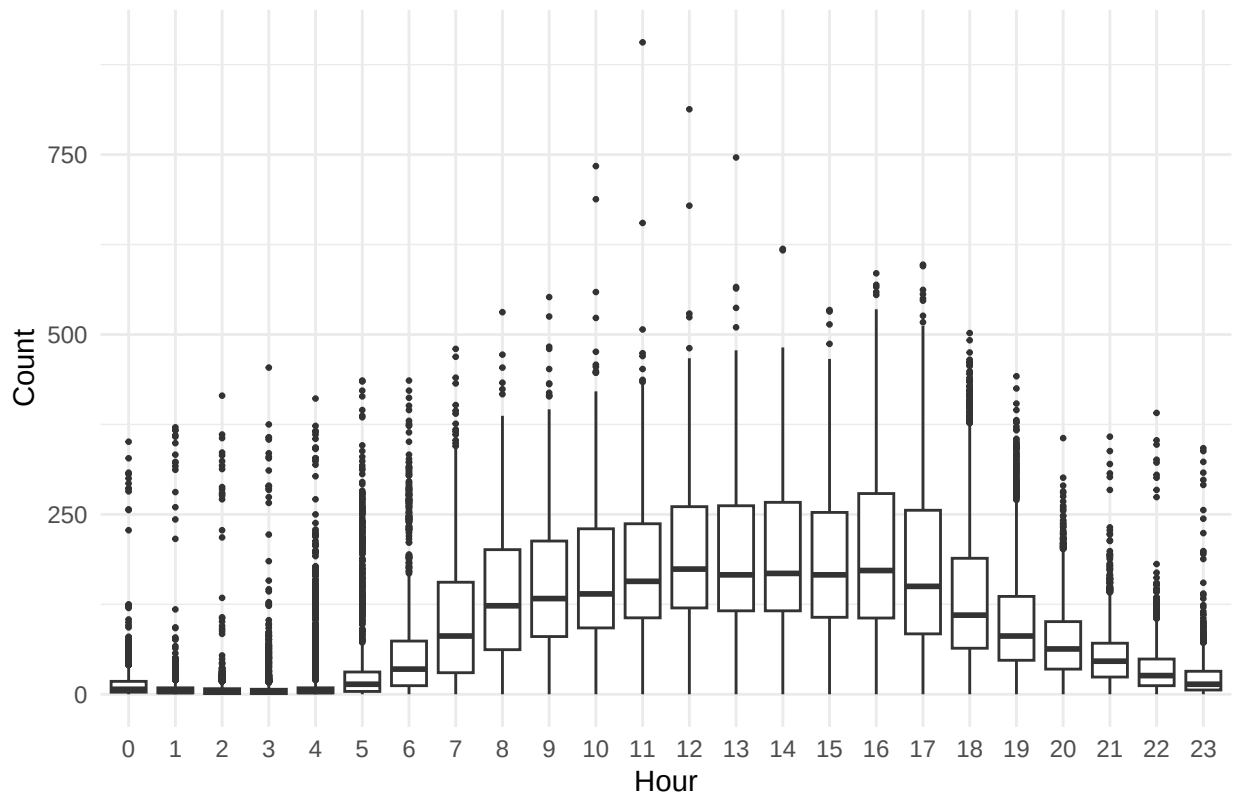
```
ggsave("slides_patternm.png", patternm, width = 10, height = 5, dpi = 300)
```

The hourly boxplots confirm the diurnal pattern shown in the line plot, while also revealing important features about the data's distribution: - Variance is clearly not constant across hours (heteroscedasticity): night hours (0-4) show low counts with a tight distribution, while daytime hours (10-18) show both higher medians and much wider spread. - Right-skewness is present at every hour, with numerous high outliers, most visible during daytime hours - consistent with the extreme spikes already seen in the full time series plot. This heteroscedasticity is worth noting for modelling: it may justify a variance-stabilising transformation (such as log or Box-Cox) before fitting ARIMA-family models, which generally assume homoscedastic residuals.

```
counthour <- df %>%
  ggplot(aes(x = factor(hour), y = value)) +
  geom_boxplot(outlier.size = 0.5) +
  labs(title = "Distribuion of count by hour of the day",
        x = "Hour", y = "Count") +
  theme_minimal()
```

```
counthour
```

Distribuion of count by hour of the day

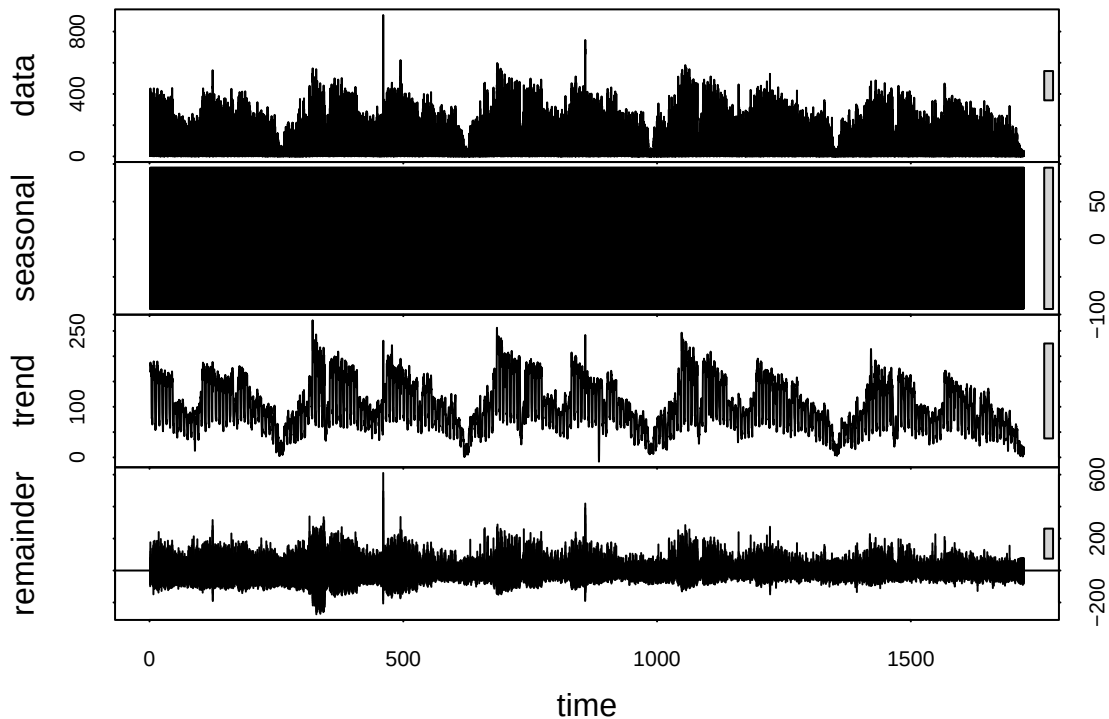


```
ggsave("slides_counthour.png", counthour, width = 10, height = 5, dpi = 300)
```

Seasonality

The STL decomposition (with daily periodicity, frequency = 24) confirms the presence of a strong, regular seasonal component and a smoother underlying trend that itself oscillates with a longer, roughly annual cycle, consistent with the annual seasonality already observed in the monthly aggregates. The remainder component still shows visible spikes aligned with the extreme outliers identified earlier, indicating these are not explained by the daily seasonal or trend components and reinforcing the need to address them explicitly (e.g. via winsorising or a dedicated outlier treatment step) before model fitting. Since this decomposition only accounts for daily seasonality, a second decomposition step (or a multi-seasonal approach via `msts()`/TBATS) would be needed to properly separate the weekly and annual seasonal components as well.

```
ts_data <- ts(df_complete$value, frequency = 24)
stl_decomp <- stl(ts_data, s.window = "periodic")
plot(stl_decomp)
```



```
hourpattheat <- df_complete %>%
  mutate(wday = wday(time, label = TRUE, week_start = 1)) %>%
  group_by(wday, hour) %>%
  summarise(mean_value = mean(value), .groups = "drop") %>%
  ggplot(aes(x = hour, y = wday, fill = mean_value)) +
  geom_tile() +
  scale_fill_viridis_c(option = "magma", direction = -1) +
  labs(title = "Average count: hour x weekday interaction",
       x = "Hour", y = "Weekday", fill = "Avg count") +
  theme_minimal()

hourpatt <- df_complete %>%
  mutate(is_weekend = wday(time, week_start = 1) %in% c(6, 7)) %>%
  group_by(hour, is_weekend) %>%
  summarise(mean_value = mean(value), .groups = "drop") %>%
  ggplot(aes(x = hour, y = mean_value, color = is_weekend)) +
  geom_line(linewidth = 1) +
  scale_color_manual(values = c("steelblue", "darkorange"),
                    labels = c("Weekday", "Weekend")) +
  labs(title = "Hourly pattern: weekday vs weekend",
       x = "Hour", y = "Average count", color = "") +
  theme_minimal()

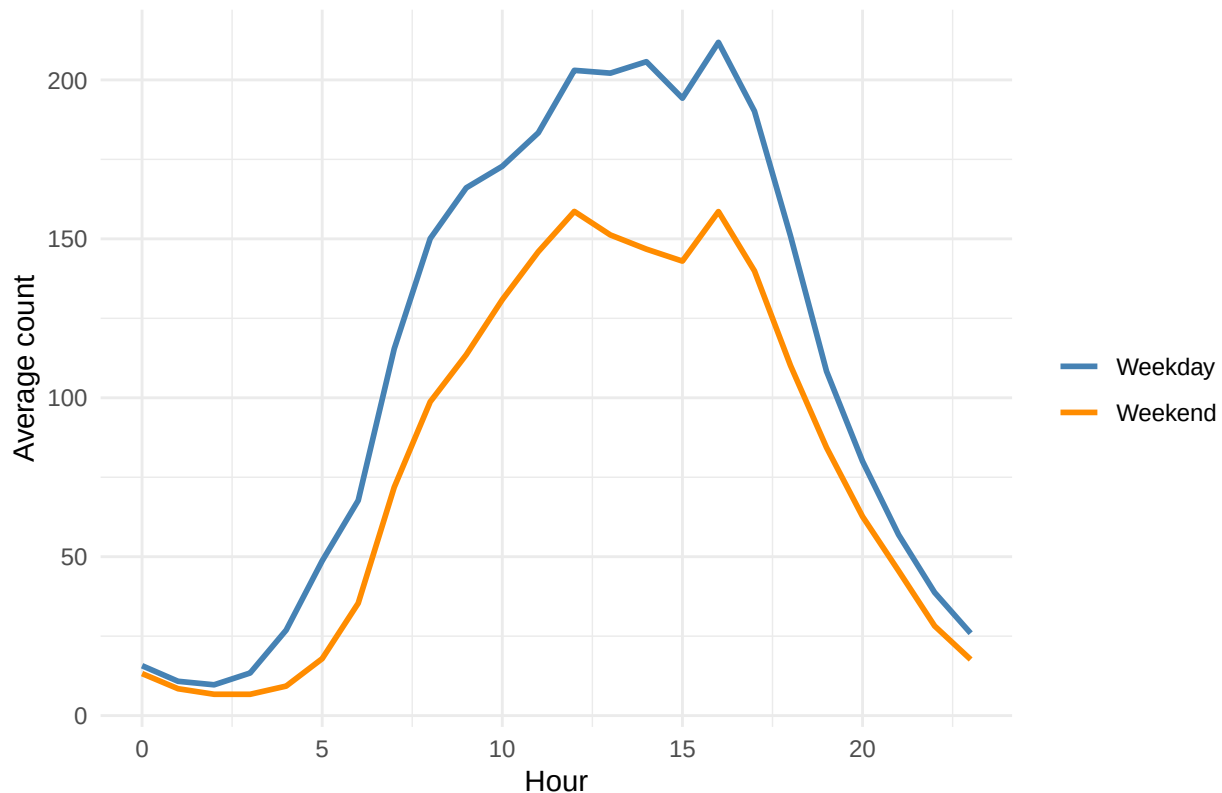
hourpattheat
```



```
ggsave("slides_hourpattheat.png", hourpattheat, width = 10, height = 5, dpi = 300)
```

```
hourpatt
```

Hourly pattern: weekday vs weekend



```
ggsave("slides_hourpatt.png", hourpatt, width = 10, height = 5, dpi = 300)
```

Stationarity

The ADF test rejects the null hypothesis of a unit root (Dickey-Fuller = -18.539, $p = 0.01$), suggesting the series is stationary. However, the KPSS test also rejects its null hypothesis of stationarity (KPSS level = 5.409, $p = 0.01$), pointing to non-stationarity. This apparent contradiction is common when strong, unaddressed seasonality is present in the raw series: ADF is testing for a stochastic trend, which is absent here, while KPSS is more sensitive to the deterministic seasonal fluctuations that dominate the series, which it interprets as a form of non-stationarity around a moving level. Both tests were run on the raw value series rather than on deseasonalised residuals; re-running them on the STL remainder component would give a cleaner read on whether genuine stochastic non-stationarity remains once seasonality is accounted for.

```
adf.test(df_complete$value)
```

```
##
## Augmented Dickey-Fuller Test
##
## data: df_complete$value
## Dickey-Fuller = -18.539, Lag order = 34, p-value = 0.01
## alternative hypothesis: stationary
```

```
kpss.test(df_complete$value)
```

```
##
## KPSS Test for Level Stationarity
##
## data: df_complete$value
```

```

## KPSS Level = 5.409, Truncation lag parameter = 18, p-value = 0.01
As the two test gives opposite results, they are repeated on residuals.
resid_stl <- stl_decomp$time.series[, "remainder"]

adf.test(resid_stl)

##
## Augmented Dickey-Fuller Test
##
## data: resid_stl
## Dickey-Fuller = -59.261, Lag order = 34, p-value = 0.01
## alternative hypothesis: stationary
kpss.test(resid_stl)

##
## KPSS Test for Level Stationarity
##
## data: resid_stl
## KPSS Level = 0.00034384, Truncation lag parameter = 18, p-value = 0.1
cat("=== RAW SERIES ===\n")

## === RAW SERIES ===
print(adf.test(df_complete$value))

##
## Augmented Dickey-Fuller Test
##
## data: df_complete$value
## Dickey-Fuller = -18.539, Lag order = 34, p-value = 0.01
## alternative hypothesis: stationary
print(kpss.test(df_complete$value))

##
## KPSS Test for Level Stationarity
##
## data: df_complete$value
## KPSS Level = 5.409, Truncation lag parameter = 18, p-value = 0.01
cat("\n=== STL RESIDUALS ===\n")

##
## === STL RESIDUALS ===
print(adf.test(resid_stl))

##
## Augmented Dickey-Fuller Test
##
## data: resid_stl
## Dickey-Fuller = -59.261, Lag order = 34, p-value = 0.01
## alternative hypothesis: stationary
print(kpss.test(resid_stl))

##

```

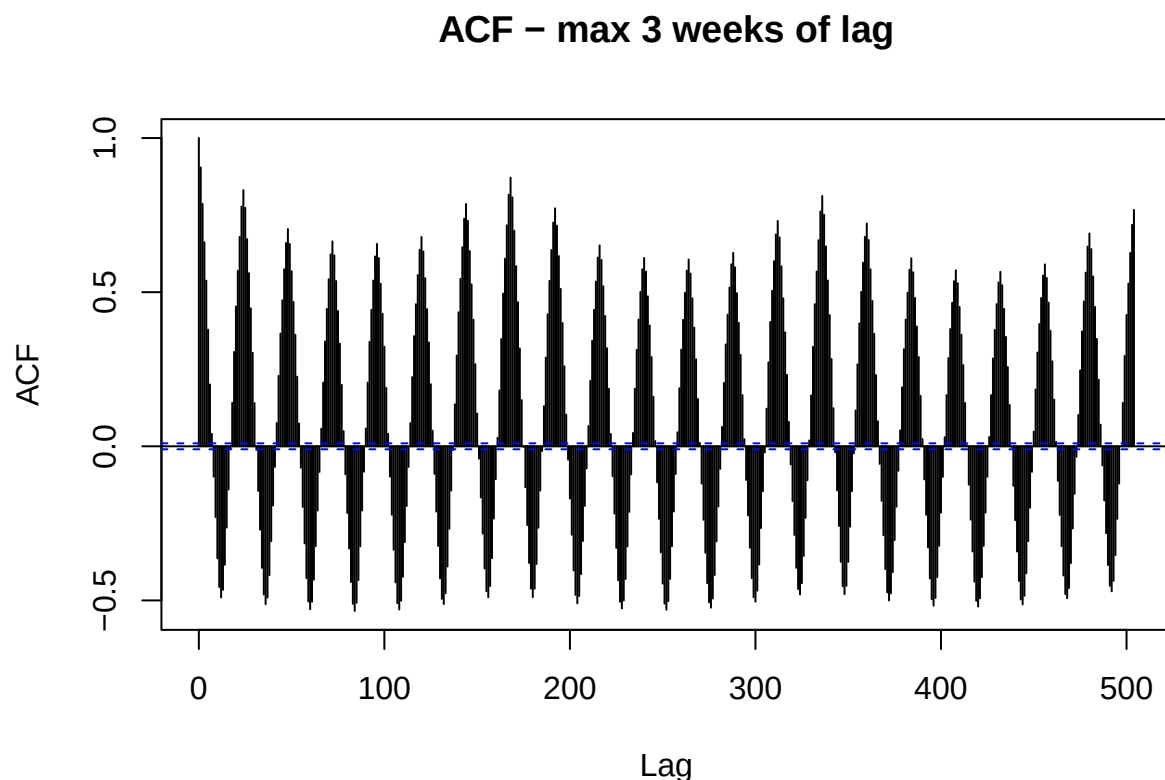
```
## KPSS Test for Level Stationarity
##
## data: resid_stl
## KPSS Level = 0.00034384, Truncation lag parameter = 18, p-value = 0.1
```

Once the daily seasonal component is removed via STL decomposition, both ADF and KPSS tests agree: the residual series is stationary. This confirms that the earlier disagreement between the two tests on the raw series was driven entirely by the unmodelled seasonal structure, not by a genuine stochastic trend. In other words, the series has no unit root - its apparent “non-stationarity” is fully explained by deterministic daily (and likely weekly/annual) seasonal patterns, which will need to be modelled explicitly rather than removed via differencing.

Autocorrelation function (ACF)

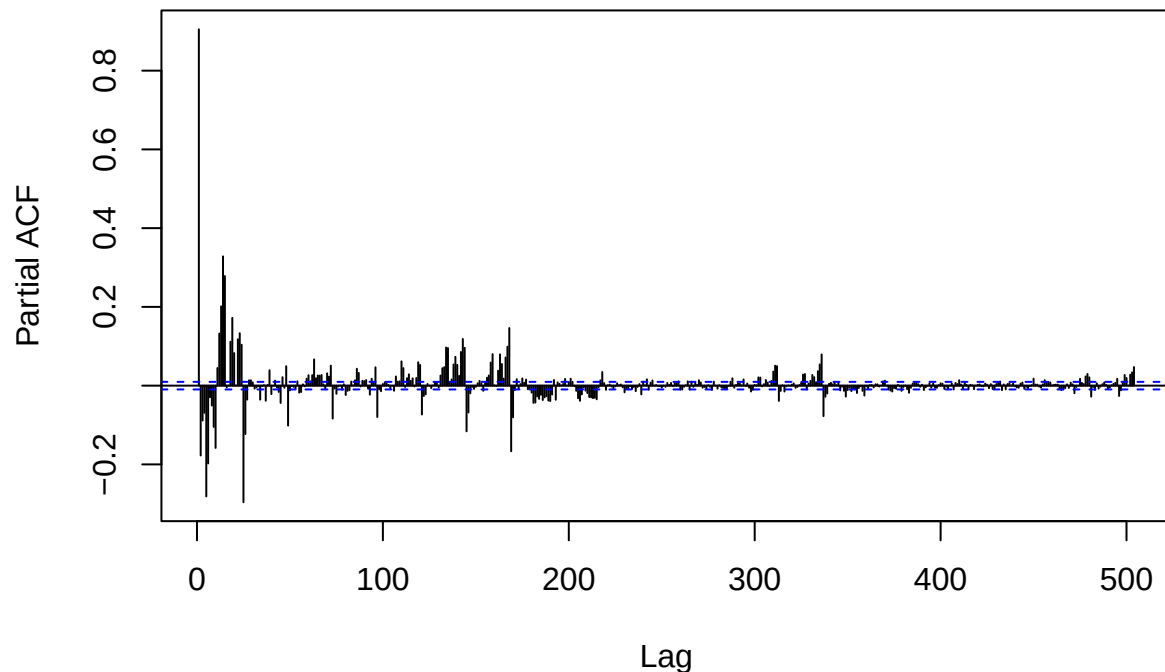
The ACF shows a clear, slowly-decaying oscillating pattern with peaks recurring every 24 lags, confirming strong daily seasonality: correlation is highest at lag 24 (~0.8) and remains elevated at lags 48, 72, 96... up to 3 weeks, where it is nearly as strong as at lag 24 (~0.85). This indicates the weekly cycle reinforces same-hour, same-weekday correlation, consistent with the weekday/weekend contrast already seen in the day-of-week barplot. The PACF shows a large spike at lag 1 (~0.9, strong short-term autocorrelation between consecutive hours) followed by smaller but non-negligible spikes clustering around lag 24 and its neighbours, and again around lag 504. This pattern is consistent with a seasonal ARIMA structure combining a short-term autoregressive component with seasonal AR/MA terms at the daily (and possibly weekly) period, and will guide the order selection for the ARIMA model in the next section.

```
acf(df_complete$value, lag.max = 24*21, main = "ACF - max 3 weeks of lag")
```



```
pacf(df_complete$value, lag.max = 24*21, main = "PACF - max 3 weeks of lag")
```

PACF – max 3 weeks of lag



Modelling

Dataset split

Since the missing values are concentrated at the end of our time series, the dataset split follows data's natural structure. All observations with a known value are further split into training (up to 31 October 2019) and validation (last two complete months), and the test set is the forecast set (January-February 2020). This design keeps chronological order intact throughout, avoiding any leakage from future to past.

```
test_df <- df %>%
  filter(is.na(value))

cat("Test set range:", as.character(min(test_df$time)), "->",
    as.character(max(test_df$time)), "\n")

## Test set range: 2020-01-01 00:00:01 -> 2020-02-29 22:00:01
cat("Test set rows:", nrow(test_df), "\n")

## Test set rows: 1439

val_start <- as.POSIXct("2019-11-01 00:00:00", tz = "UTC")

val_df <- df_complete %>%
  filter(time >= val_start)

cat("Validation set range:", as.character(min(val_df$time)), "->",
```

```

    as.character(max(val_df$time)), "\n")

## Validation set range: 2019-11-01 00:00:01 -> 2019-12-31 23:00:01
cat("Validation set rows:", nrow(val_df), "\n")

## Validation set rows: 1464
train_df <- df_complete %>%
  filter(time < val_start)

cat("Train set range:", as.character(min(train_df$time)), "->",
    as.character(max(train_df$time)), "\n")

## Train set range: 2015-04-15 00:00:01 -> 2019-10-31 23:00:01
cat("Train set rows:", nrow(train_df), "\n")

## Train set rows: 39864
cat("\n--- Concordance check ---\n")

##
## --- Concordance check ---
cat("Train + Val + Test rows:", nrow(train_df) + nrow(val_df) + nrow(test_df),
    " (should equal", nrow(df), ")\n")

## Train + Val + Test rows: 42767 (should equal 42767 )
cat("Any overlap train/val:", any(train_df$time %in% val_df$time), "\n")

## Any overlap train/val: FALSE
cat("Any overlap val/test:", any(val_df$time %in% test_df$time), "\n")

## Any overlap val/test: FALSE

```

Outlier analysis

```

Q1 <- quantile(df_complete$value, 0.25)
Q3 <- quantile(df_complete$value, 0.75)
IQR_val <- Q3 - Q1
upper_bound <- Q3 + 3 * IQR_val

cat("Q1:", Q1, " Q3:", Q3, " IQR:", IQR_val, "\n")

## Q1: 14 Q3: 159 IQR: 145
cat("Upper bound (3x IQR):", upper_bound, "\n")

## Upper bound (3x IQR): 594
outliers <- df_complete %>%
  filter(value > upper_bound) %>%
  arrange(desc(value))

print(outliers)

##
time value year month day hour

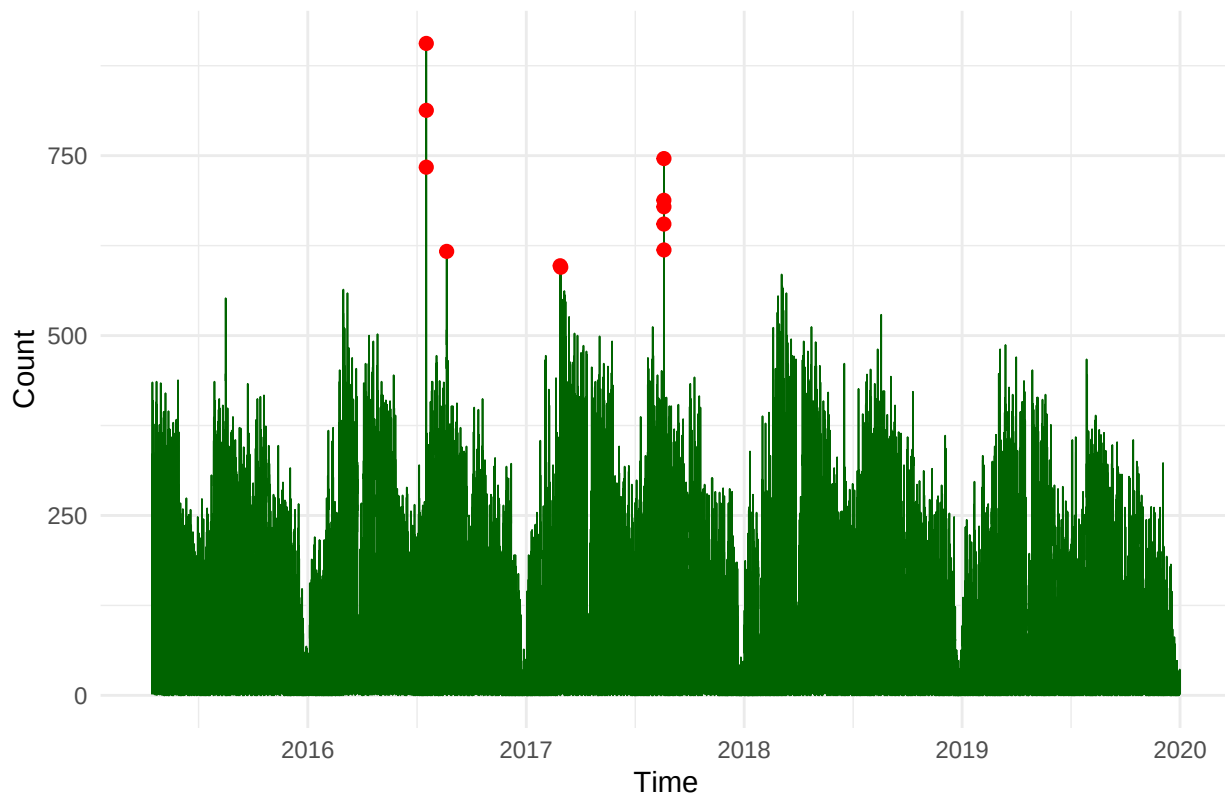
```

```
## 1 2016-07-17 11:00:01 906 2016 7 17 11
## 2 2016-07-17 12:00:01 813 2016 7 17 12
## 3 2017-08-19 13:00:01 746 2017 8 19 13
## 4 2016-07-17 10:00:01 734 2016 7 17 10
## 5 2017-08-19 10:00:01 688 2017 8 19 10
## 6 2017-08-19 12:00:01 679 2017 8 19 12
## 7 2017-08-19 11:00:01 655 2017 8 19 11
## 8 2017-08-19 14:00:01 619 2017 8 19 14
## 9 2016-08-20 14:00:01 617 2016 8 20 14
## 10 2017-02-26 17:00:01 597 2017 2 26 17
## 11 2017-02-27 17:00:01 595 2017 2 27 17
```

```
outlier <- df_complete %>%
  ggplot(aes(x = time, y = value)) +
  geom_line(color = "darkgreen", linewidth = 0.3) +
  geom_point(data = outliers, aes(x = time, y = value), color = "red", size = 2) +
  labs(title = "Time series with outliers highlighted (> Q3 + 3xIQR)",
       x = "Time", y = "Count") +
  theme_minimal()
```

outlier

Time series with outliers highlighted (> Q3 + 3xIQR)



```
ggsave("slides_outlier.png", outlier, width = 10, height = 5, dpi = 300)

outlier_days <- unique(as_date(outliers$time))
cat("\nUnique days involved:", length(outlier_days), "\n")
```

```
##
## Unique days involved: 5
print(outlier_days)

## [1] "2016-07-17" "2017-08-19" "2016-08-20" "2017-02-26" "2017-02-27"
for (day_out in outlier_days) {

  day_out <- as_date(day_out)

  cat("\n===== \n")
  cat("DAY:", as.character(day_out), "\n")
  cat("===== \n")

  day_profile <- df_complete %>%
    filter(as_date(time) == day_out) %>%
    arrange(time) %>%
    select(time, hour, value) %>%
    mutate(
      diff_prev_hour = value - lag(value),
      pct_change = round((value - lag(value)) / lag(value) * 100, 1)
    )

  cat("\n--- Full hourly profile + hour-to-hour change --- \n")
  print(day_profile)

  wd <- wday(day_out)
  typical_profile <- df_complete %>%
    filter(wday(time) == wd, as_date(time) != day_out) %>%
    group_by(hour) %>%
    summarise(mean_value = mean(value), .groups = "drop")

  comparison <- day_profile %>%
    left_join(typical_profile, by = "hour") %>%
    mutate(deviation_from_typical = value - mean_value) %>%
    select(time, hour, value, mean_value, deviation_from_typical)

  cat("\n--- Comparison vs typical same-weekday hourly average --- \n")
  print(comparison)

  surrounding_days <- df_complete %>%
    filter(as_date(time) >= day_out - days(3),
           as_date(time) <= day_out + days(3))

  outlier_times_this_day <- outliers %>%
    filter(as_date(time) == day_out) %>%
    pull(time)

  p <- ggplot(surrounding_days, aes(x = time, y = value)) +
    geom_line(color = "darkgreen") +
    geom_vline(xintercept = as.numeric(outlier_times_this_day),
               color = "red", linetype = "dashed") +
    labs(title = paste("Days around outlier:", as.character(day_out)),
          subtitle = "Red dashed lines = outlier timestamps",
```

```

        x = "Time", y = "Count") +
    theme_minimal()

print(p)
}

```

```

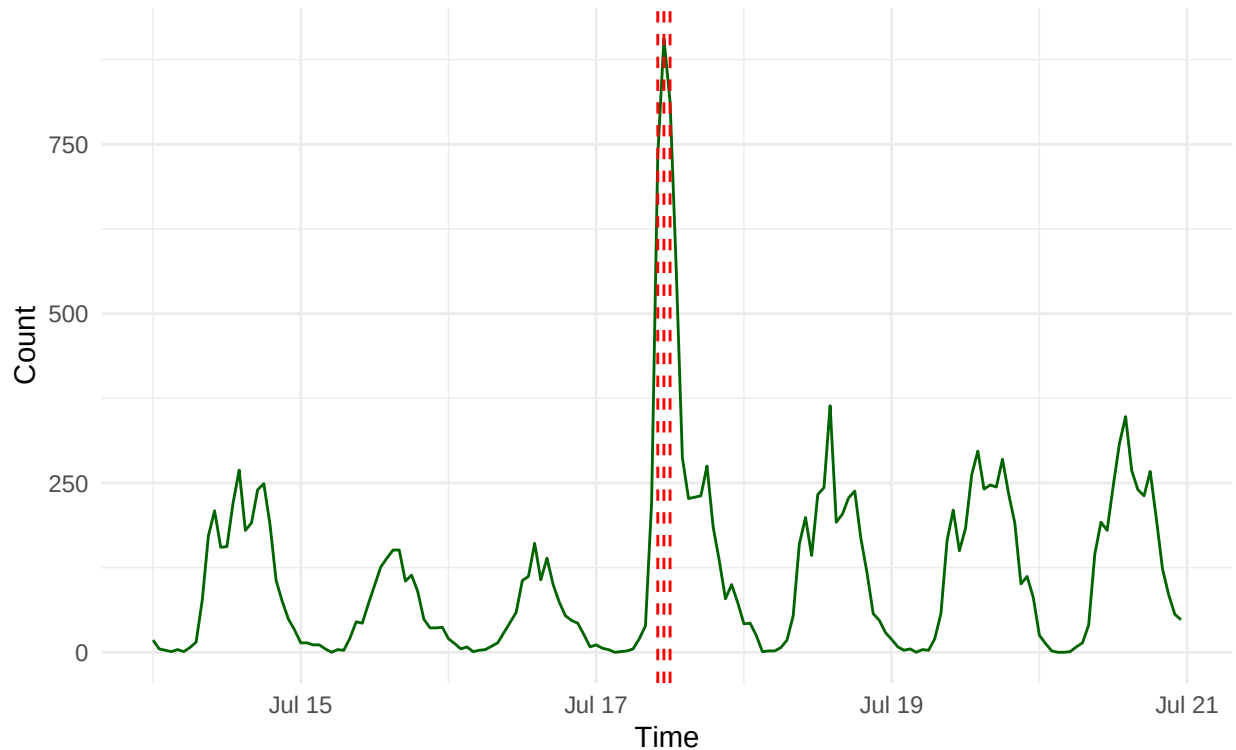
##
## =====
## DAY: 2016-07-17
## =====
##
## --- Full hourly profile + hour-to-hour change ---
##
##      time hour value diff_prev_hour pct_change
## 1  2016-07-17 00:00:01      0      11          NA          NA
## 2  2016-07-17 01:00:01      1       6          -5         -45.5
## 3  2016-07-17 02:00:01      2       4          -2         -33.3
## 4  2016-07-17 03:00:01      3       0          -4        -100.0
## 5  2016-07-17 04:00:01      4       1           1           Inf
## 6  2016-07-17 05:00:01      5       2           1        100.0
## 7  2016-07-17 06:00:01      6       5           3        150.0
## 8  2016-07-17 07:00:01      7      20          15        300.0
## 9  2016-07-17 08:00:01      8      39          19         95.0
## 10 2016-07-17 09:00:01     9     221         182        466.7
## 11 2016-07-17 10:00:01    10     734         513        232.1
## 12 2016-07-17 11:00:01    11     906         172         23.4
## 13 2016-07-17 12:00:01    12     813          -93        -10.3
## 14 2016-07-17 13:00:01    13     566        -247        -30.4
## 15 2016-07-17 14:00:01    14     287        -279        -49.3
## 16 2016-07-17 15:00:01    15     227         -60        -20.9
## 17 2016-07-17 16:00:01    16     229           2          0.9
## 18 2016-07-17 17:00:01    17     231           2          0.9
## 19 2016-07-17 18:00:01    18     275          44         19.0
## 20 2016-07-17 19:00:01    19     185         -90        -32.7
## 21 2016-07-17 20:00:01    20     135         -50        -27.0
## 22 2016-07-17 21:00:01    21      79         -56        -41.5
## 23 2016-07-17 22:00:01    22     100          21         26.6
## 24 2016-07-17 23:00:01    23      73         -27        -27.0
##
## --- Comparison vs typical same-weekday hourly average ---
##
##      time hour value mean_value deviation_from_typical
## 1  2016-07-17 00:00:01      0      11  10.412245      0.5877551
## 2  2016-07-17 01:00:01      1       6   5.648980      0.3510204
## 3  2016-07-17 02:00:01      2       4   4.730612     -0.7306122
## 4  2016-07-17 03:00:01      3       0   5.130612     -5.1306122
## 5  2016-07-17 04:00:01      4       1   8.881633     -7.8816327
## 6  2016-07-17 05:00:01      5       2  20.559184    -18.5591837
## 7  2016-07-17 06:00:01      6       5  46.914286    -41.9142857
## 8  2016-07-17 07:00:01      7      20 107.473469    -87.4734694
## 9  2016-07-17 08:00:01      8      39 142.171429   -103.1714286
## 10 2016-07-17 09:00:01     9     221 153.159184     67.8408163
## 11 2016-07-17 10:00:01    10     734 166.461224    567.5387755
## 12 2016-07-17 11:00:01    11     906 181.253061    724.7469388
## 13 2016-07-17 12:00:01    12     813 202.224490    610.7755102
## 14 2016-07-17 13:00:01    13     566 184.273469    381.7265306

```

##	15	2016-07-17 14:00:01	14	287	179.563265	107.4367347
##	16	2016-07-17 15:00:01	15	227	181.669388	45.3306122
##	17	2016-07-17 16:00:01	16	229	218.040816	10.9591837
##	18	2016-07-17 17:00:01	17	231	191.848980	39.1510204
##	19	2016-07-17 18:00:01	18	275	148.195918	126.8040816
##	20	2016-07-17 19:00:01	19	185	106.461224	78.5387755
##	21	2016-07-17 20:00:01	20	135	77.828571	57.1714286
##	22	2016-07-17 21:00:01	21	79	53.404082	25.5959184
##	23	2016-07-17 22:00:01	22	100	31.595918	68.4040816
##	24	2016-07-17 23:00:01	23	73	17.942857	55.0571429

Days around outlier: 2016-07-17

Red dashed lines = outlier timestamps



```
##
## =====
## DAY: 2017-08-19
## =====
##
## --- Full hourly profile + hour-to-hour change ---
##      time hour value diff_prev_hour pct_change
## 1  2017-08-19 00:00:01    0    19          NA          NA
## 2  2017-08-19 01:00:01    1    20           1           5.3
## 3  2017-08-19 02:00:01    2    16          -4          -20.0
## 4  2017-08-19 03:00:01    3     5         -11          -68.8
## 5  2017-08-19 04:00:01    4     6           1           20.0
## 6  2017-08-19 05:00:01    5     7           1           16.7
## 7  2017-08-19 06:00:01    6     6          -1          -14.3
## 8  2017-08-19 07:00:01    7    36          30          500.0
## 9  2017-08-19 08:00:01    8    82          46          127.8
```

## 10	2017-08-19 09:00:01	9	356	274	334.1
## 11	2017-08-19 10:00:01	10	688	332	93.3
## 12	2017-08-19 11:00:01	11	655	-33	-4.8
## 13	2017-08-19 12:00:01	12	679	24	3.7
## 14	2017-08-19 13:00:01	13	746	67	9.9
## 15	2017-08-19 14:00:01	14	619	-127	-17.0
## 16	2017-08-19 15:00:01	15	514	-105	-17.0
## 17	2017-08-19 16:00:01	16	294	-220	-42.8
## 18	2017-08-19 17:00:01	17	222	-72	-24.5
## 19	2017-08-19 18:00:01	18	123	-99	-44.6
## 20	2017-08-19 19:00:01	19	86	-37	-30.1
## 21	2017-08-19 20:00:01	20	65	-21	-24.4
## 22	2017-08-19 21:00:01	21	42	-23	-35.4
## 23	2017-08-19 22:00:01	22	54	12	28.6
## 24	2017-08-19 23:00:01	23	24	-30	-55.6

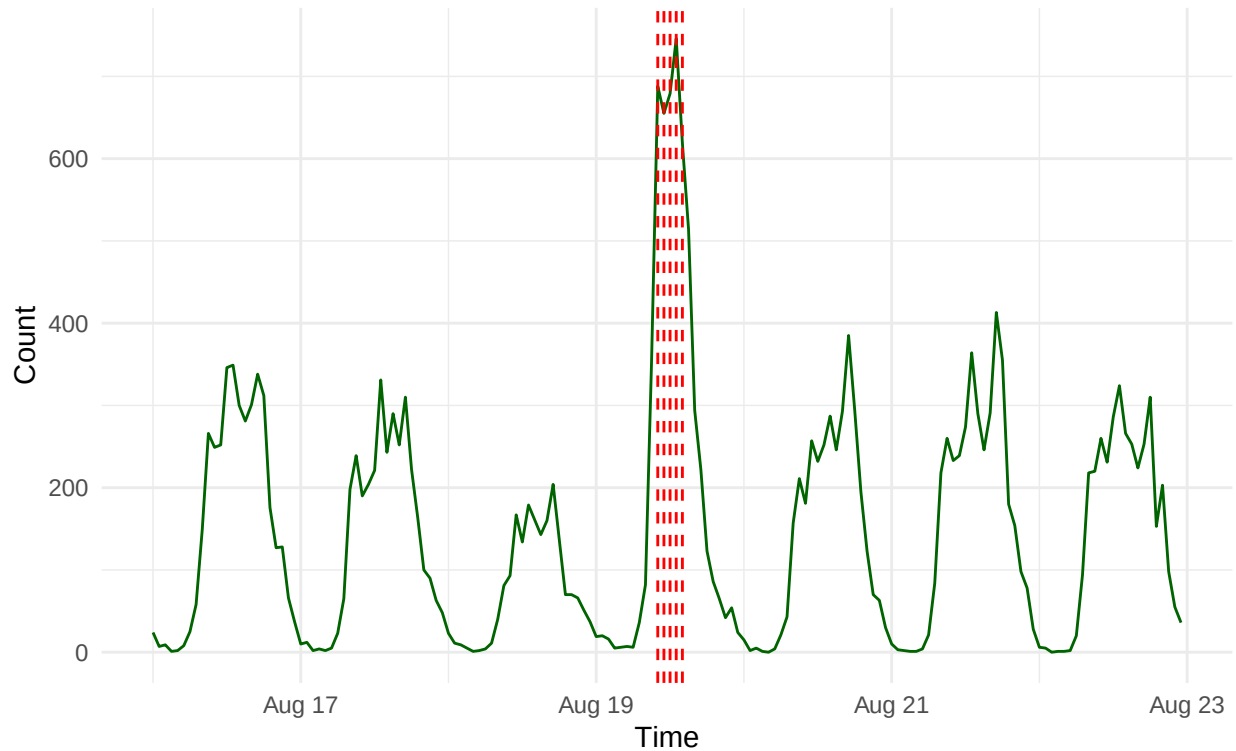
##

--- Comparison vs typical same-weekday hourly average ---

##		time	hour	value	mean_value	deviation_from_typical
## 1	2017-08-19 00:00:01	0	19	16.044898		2.9551020
## 2	2017-08-19 01:00:01	1	20	11.228571		8.7714286
## 3	2017-08-19 02:00:01	2	16	8.689796		7.3102041
## 4	2017-08-19 03:00:01	3	5	8.322449		-3.3224490
## 5	2017-08-19 04:00:01	4	6	9.751020		-3.7510204
## 6	2017-08-19 05:00:01	5	7	15.395918		-8.3959184
## 7	2017-08-19 06:00:01	6	6	24.053061		-18.0530612
## 8	2017-08-19 07:00:01	7	36	36.620408		-0.6204082
## 9	2017-08-19 08:00:01	8	82	55.640816		26.3591837
## 10	2017-08-19 09:00:01	9	356	72.514286		283.4857143
## 11	2017-08-19 10:00:01	10	688	90.534694		597.4653061
## 12	2017-08-19 11:00:01	11	655	105.473469		549.5265306
## 13	2017-08-19 12:00:01	12	679	110.220408		568.7795918
## 14	2017-08-19 13:00:01	13	746	114.048980		631.9510204
## 15	2017-08-19 14:00:01	14	619	111.457143		507.5428571
## 16	2017-08-19 15:00:01	15	514	102.465306		411.5346939
## 17	2017-08-19 16:00:01	16	294	98.261224		195.7387755
## 18	2017-08-19 17:00:01	17	222	87.240816		134.7591837
## 19	2017-08-19 18:00:01	18	123	71.359184		51.6408163
## 20	2017-08-19 19:00:01	19	86	61.795918		24.2040816
## 21	2017-08-19 20:00:01	20	65	47.122449		17.8775510
## 22	2017-08-19 21:00:01	21	42	37.587755		4.4122449
## 23	2017-08-19 22:00:01	22	54	24.420408		29.5795918
## 24	2017-08-19 23:00:01	23	24	17.134694		6.8653061

Days around outlier: 2017-08-19

Red dashed lines = outlier timestamps



```
##
## =====
## DAY: 2016-08-20
## =====
##
## --- Full hourly profile + hour-to-hour change ---
##      time hour value diff_prev_hour pct_change
## 1  2016-08-20 00:00:01    0    39          NA         NA
## 2  2016-08-20 01:00:01    1    32          -7        -17.9
## 3  2016-08-20 02:00:01    2    27          -5        -15.6
## 4  2016-08-20 03:00:01    3     7         -20       -74.1
## 5  2016-08-20 04:00:01    4    10           3        42.9
## 6  2016-08-20 05:00:01    5     9          -1       -10.0
## 7  2016-08-20 06:00:01    6     2          -7       -77.8
## 8  2016-08-20 07:00:01    7     4           2       100.0
## 9  2016-08-20 08:00:01    8    55          51      1275.0
## 10 2016-08-20 09:00:01   9    66          11       20.0
## 11 2016-08-20 10:00:01  10   251         185      280.3
## 12 2016-08-20 11:00:01  11   470         219       87.3
## 13 2016-08-20 12:00:01  12   454          -16        -3.4
## 14 2016-08-20 13:00:01  13   537           83       18.3
## 15 2016-08-20 14:00:01  14   617           80       14.9
## 16 2016-08-20 15:00:01  15   487        -130      -21.1
## 17 2016-08-20 16:00:01  16   508           21        4.3
## 18 2016-08-20 17:00:01  17   360        -148      -29.1
## 19 2016-08-20 18:00:01  18   148        -212      -58.9
```

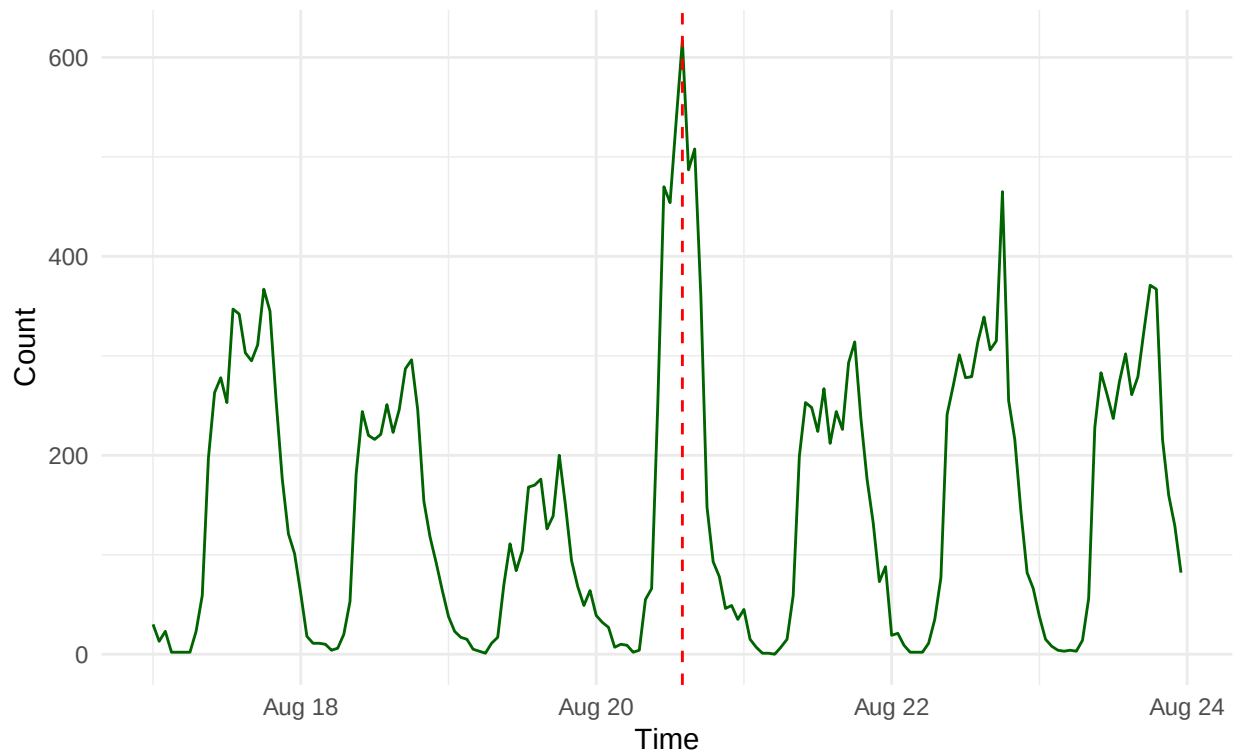
```

## 20 2016-08-20 19:00:01 19 93 -55 -37.2
## 21 2016-08-20 20:00:01 20 78 -15 -16.1
## 22 2016-08-20 21:00:01 21 46 -32 -41.0
## 23 2016-08-20 22:00:01 22 49 3 6.5
## 24 2016-08-20 23:00:01 23 35 -14 -28.6
##
## --- Comparison vs typical same-weekday hourly average ---
##      time hour value mean_value deviation_from_typical
## 1 2016-08-20 00:00:01 0 39 15.963265 23.0367347
## 2 2016-08-20 01:00:01 1 32 11.179592 20.8204082
## 3 2016-08-20 02:00:01 2 27 8.644898 18.3551020
## 4 2016-08-20 03:00:01 3 7 8.314286 -1.3142857
## 5 2016-08-20 04:00:01 4 10 9.734694 0.2653061
## 6 2016-08-20 05:00:01 5 9 15.387755 -6.3877551
## 7 2016-08-20 06:00:01 6 2 24.069388 -22.0693878
## 8 2016-08-20 07:00:01 7 4 36.751020 -32.7510204
## 9 2016-08-20 08:00:01 8 55 55.751020 -0.7510204
## 10 2016-08-20 09:00:01 9 66 73.697959 -7.6979592
## 11 2016-08-20 10:00:01 10 251 92.318367 158.6816327
## 12 2016-08-20 11:00:01 11 470 106.228571 363.7714286
## 13 2016-08-20 12:00:01 12 454 111.138776 342.8612245
## 14 2016-08-20 13:00:01 13 537 114.902041 422.0979592
## 15 2016-08-20 14:00:01 14 617 111.465306 505.5346939
## 16 2016-08-20 15:00:01 15 487 102.575510 384.4244898
## 17 2016-08-20 16:00:01 16 508 97.387755 410.6122449
## 18 2016-08-20 17:00:01 17 360 86.677551 273.3224490
## 19 2016-08-20 18:00:01 18 148 71.257143 76.7428571
## 20 2016-08-20 19:00:01 19 93 61.767347 31.2326531
## 21 2016-08-20 20:00:01 20 78 47.069388 30.9306122
## 22 2016-08-20 21:00:01 21 46 37.571429 8.4285714
## 23 2016-08-20 22:00:01 22 49 24.440816 24.5591837
## 24 2016-08-20 23:00:01 23 35 17.089796 17.9102041

```

Days around outlier: 2016-08-20

Red dashed lines = outlier timestamps



```
##
## =====
## DAY: 2017-02-26
## =====
##
## --- Full hourly profile + hour-to-hour change ---
##      time hour value diff_prev_hour pct_change
## 1  2017-02-26 00:00:01    0    18           NA         NA
## 2  2017-02-26 01:00:01    1     7          -11       -61.1
## 3  2017-02-26 02:00:01    2     2           -5       -71.4
## 4  2017-02-26 03:00:01    3     3            1        50.0
## 5  2017-02-26 04:00:01    4     1           -2       -66.7
## 6  2017-02-26 05:00:01    5     4            3       300.0
## 7  2017-02-26 06:00:01    6    28           24       600.0
## 8  2017-02-26 07:00:01    7   110           82      292.9
## 9  2017-02-26 08:00:01    8  417          307      279.1
## 10 2017-02-26 09:00:01    9  366          -51       -12.2
## 11 2017-02-26 10:00:01   10  458           92        25.1
## 12 2017-02-26 11:00:01   11  431          -27        -5.9
## 13 2017-02-26 12:00:01   12  437            6         1.4
## 14 2017-02-26 13:00:01   13  402          -35        -8.0
## 15 2017-02-26 14:00:01   14  461           59        14.7
## 16 2017-02-26 15:00:01   15  534           73        15.8
## 17 2017-02-26 16:00:01   16  488          -46        -8.6
## 18 2017-02-26 17:00:01   17  597          109        22.3
## 19 2017-02-26 18:00:01   18  399         -198       -33.2
```

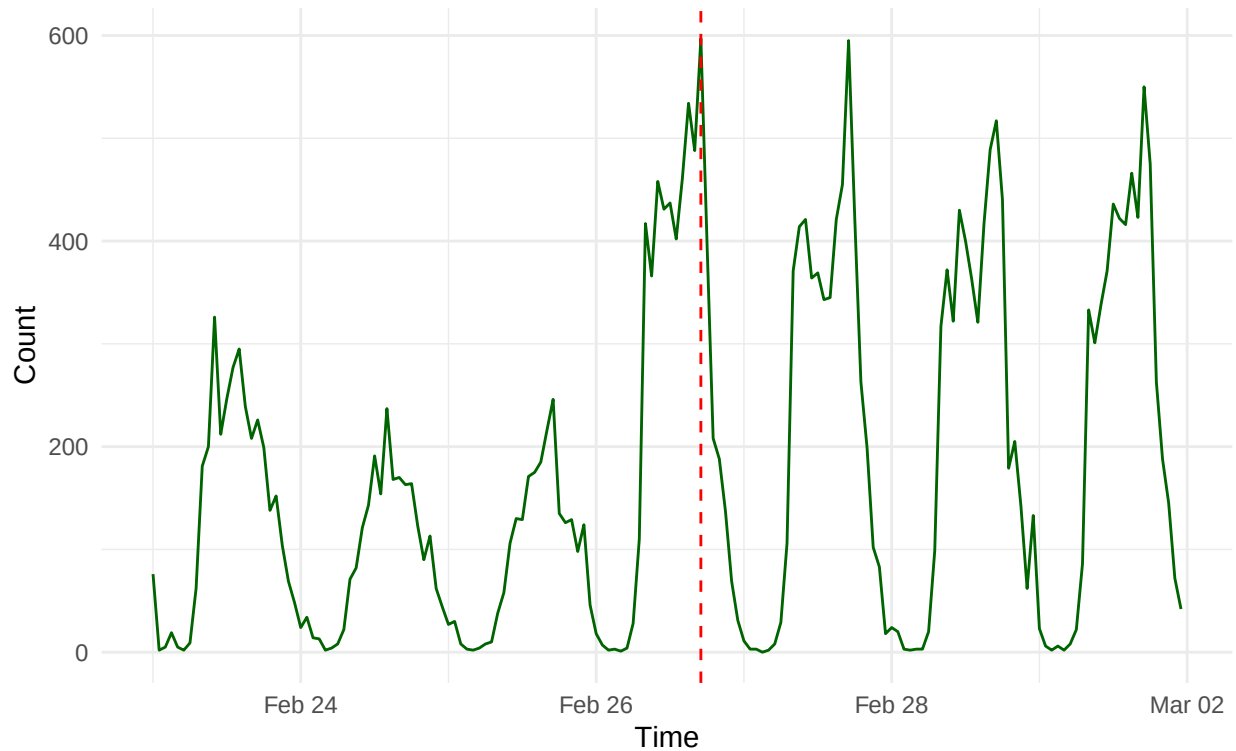
```

## 20 2017-02-26 19:00:01 19 208 -191 -47.9
## 21 2017-02-26 20:00:01 20 188 -20 -9.6
## 22 2017-02-26 21:00:01 21 137 -51 -27.1
## 23 2017-02-26 22:00:01 22 69 -68 -49.6
## 24 2017-02-26 23:00:01 23 31 -38 -55.1
##
## --- Comparison vs typical same-weekday hourly average ---
##      time hour value mean_value deviation_from_typical
## 1  2017-02-26 00:00:01 0 18 10.383673 7.616327
## 2  2017-02-26 01:00:01 1 7 5.644898 1.355102
## 3  2017-02-26 02:00:01 2 2 4.738776 -2.738776
## 4  2017-02-26 03:00:01 3 3 5.118367 -2.118367
## 5  2017-02-26 04:00:01 4 1 8.881633 -7.881633
## 6  2017-02-26 05:00:01 5 4 20.551020 -16.551020
## 7  2017-02-26 06:00:01 6 28 46.820408 -18.820408
## 8  2017-02-26 07:00:01 7 110 107.106122 2.893878
## 9  2017-02-26 08:00:01 8 417 140.628571 276.371429
## 10 2017-02-26 09:00:01 9 366 152.567347 213.432653
## 11 2017-02-26 10:00:01 10 458 167.587755 290.412245
## 12 2017-02-26 11:00:01 11 431 183.191837 247.808163
## 13 2017-02-26 12:00:01 12 437 203.759184 233.240816
## 14 2017-02-26 13:00:01 13 402 184.942857 217.057143
## 15 2017-02-26 14:00:01 14 461 178.853061 282.146939
## 16 2017-02-26 15:00:01 15 534 180.416327 353.583673
## 17 2017-02-26 16:00:01 16 488 216.983673 271.016327
## 18 2017-02-26 17:00:01 17 597 190.355102 406.644898
## 19 2017-02-26 18:00:01 18 399 147.689796 251.310204
## 20 2017-02-26 19:00:01 19 208 106.367347 101.632653
## 21 2017-02-26 20:00:01 20 188 77.612245 110.387755
## 22 2017-02-26 21:00:01 21 137 53.167347 83.832653
## 23 2017-02-26 22:00:01 22 69 31.722449 37.277551
## 24 2017-02-26 23:00:01 23 31 18.114286 12.885714

```

Days around outlier: 2017-02-26

Red dashed lines = outlier timestamps



```
##
## =====
## DAY: 2017-02-27
## =====
##
## --- Full hourly profile + hour-to-hour change ---
##          time hour value diff_prev_hour pct_change
## 1  2017-02-27 00:00:01    0    11           NA        NA
## 2  2017-02-27 01:00:01    1     3          -8       -72.7
## 3  2017-02-27 02:00:01    2     3           0         0.0
## 4  2017-02-27 03:00:01    3     0          -3      -100.0
## 5  2017-02-27 04:00:01    4     2           2         Inf
## 6  2017-02-27 05:00:01    5     8           6       300.0
## 7  2017-02-27 06:00:01    6    29          21      262.5
## 8  2017-02-27 07:00:01    7   106          77      265.5
## 9  2017-02-27 08:00:01    8   371         265      250.0
## 10 2017-02-27 09:00:01   9   414          43        11.6
## 11 2017-02-27 10:00:01  10   421           7         1.7
## 12 2017-02-27 11:00:01  11   364         -57       -13.5
## 13 2017-02-27 12:00:01  12   369           5         1.4
## 14 2017-02-27 13:00:01  13   343         -26        -7.0
## 15 2017-02-27 14:00:01  14   345           2         0.6
## 16 2017-02-27 15:00:01  15   421          76       22.0
## 17 2017-02-27 16:00:01  16   455          34         8.1
## 18 2017-02-27 17:00:01  17   595         140       30.8
## 19 2017-02-27 18:00:01  18   423        -172       -28.9
```

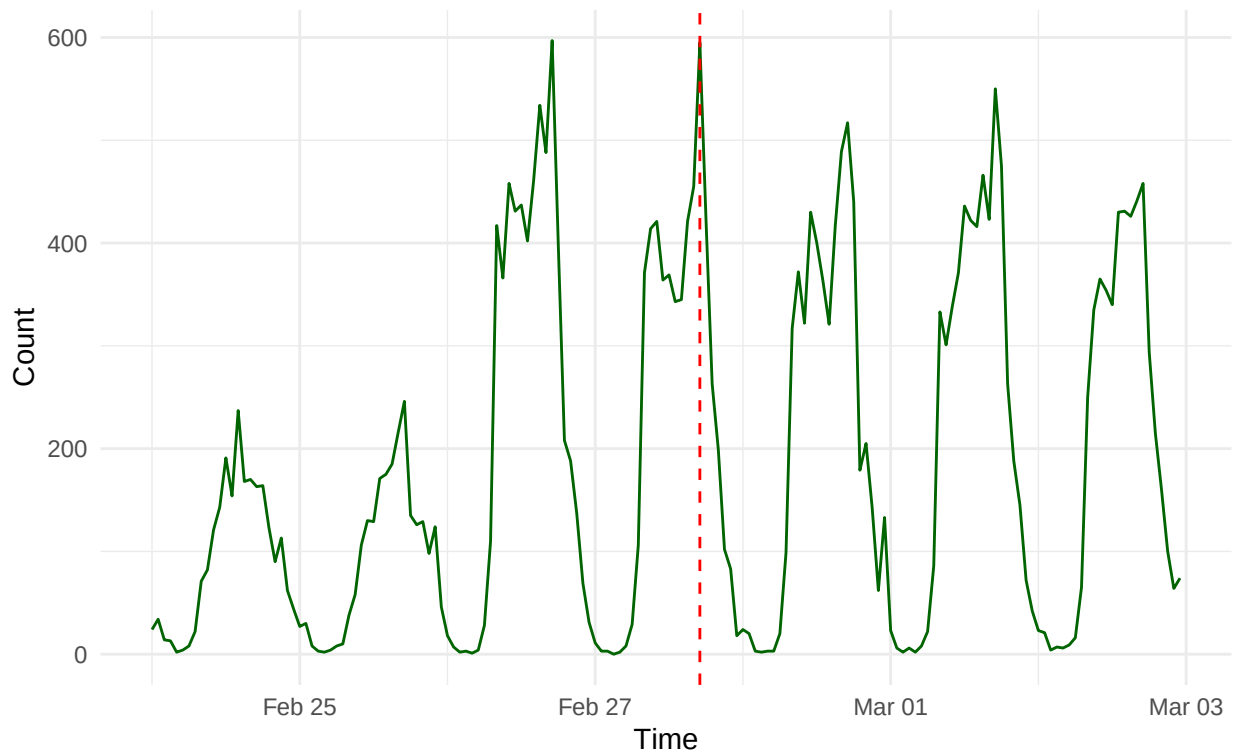
```

## 20 2017-02-27 19:00:01    19    263          -160          -37.8
## 21 2017-02-27 20:00:01    20    199           -64          -24.3
## 22 2017-02-27 21:00:01    21    102           -97          -48.7
## 23 2017-02-27 22:00:01    22     83           -19          -18.6
## 24 2017-02-27 23:00:01    23     18           -65          -78.3
##
## --- Comparison vs typical same-weekday hourly average ---
##           time hour value mean_value deviation_from_typical
## 1  2017-02-27 00:00:01     0     11  11.404082          -0.4040816
## 2  2017-02-27 01:00:01     1      3   8.281633          -5.2816327
## 3  2017-02-27 02:00:01     2      3   7.938776          -4.9387755
## 4  2017-02-27 03:00:01     3      0  12.775510         -12.7755102
## 5  2017-02-27 04:00:01     4      2  28.061224        -26.0612245
## 6  2017-02-27 05:00:01     5      8  49.416327        -41.4163265
## 7  2017-02-27 06:00:01     6     29  70.404082        -41.4040816
## 8  2017-02-27 07:00:01     7    106 130.318367        -24.3183673
## 9  2017-02-27 08:00:01     8    371 170.975510         200.0244898
## 10 2017-02-27 09:00:01     9   414 182.967347         231.0326531
## 11 2017-02-27 10:00:01    10   421 184.330612         236.6693878
## 12 2017-02-27 11:00:01    11   364 195.522449         168.4775510
## 13 2017-02-27 12:00:01    12   369 221.077551         147.9224490
## 14 2017-02-27 13:00:01    13   343 216.616327         126.3836735
## 15 2017-02-27 14:00:01    14   345 216.702041         128.2979592
## 16 2017-02-27 15:00:01    15   421 208.379592         212.6204082
## 17 2017-02-27 16:00:01    16   455 241.775510         213.2244898
## 18 2017-02-27 17:00:01    17   595 217.346939         377.6530612
## 19 2017-02-27 18:00:01    18   423 170.906122         252.0938776
## 20 2017-02-27 19:00:01    19   263 122.530612         140.4693878
## 21 2017-02-27 20:00:01    20   199  86.000000         113.0000000
## 22 2017-02-27 21:00:01    21   102  57.367347          44.6326531
## 23 2017-02-27 22:00:01    22    83  35.857143          47.1428571
## 24 2017-02-27 23:00:01    23    18  21.616327          -3.6163265

```

Days around outlier: 2017-02-27

Red dashed lines = outlier timestamps



Are shown 5 days with higher values than usual (2016-07-17, 2017-08-19, 2016-08-20, 2017-02-26, 2017-02-27), with three days with extreme values, identified via IQR filtering (2016-07-17, 2017-08-19, 2017-02-26/27). In all cases, the surrounding hours and days show elevated but coherent counts (i.e., the whole day is high, not a single implausible spike surrounded by normal values), making it plausible for an event to have been occurring in the city rather than a sensor malfunction. Given the limited number of affected days relative to the ~5-year span of the series, and the fact that the anomaly reflects a genuine, if rare, phenomenon rather than a data quality issue, these values were retained without modification. Their impact will be monitored via residual diagnostics after model fitting.

Trend analysis

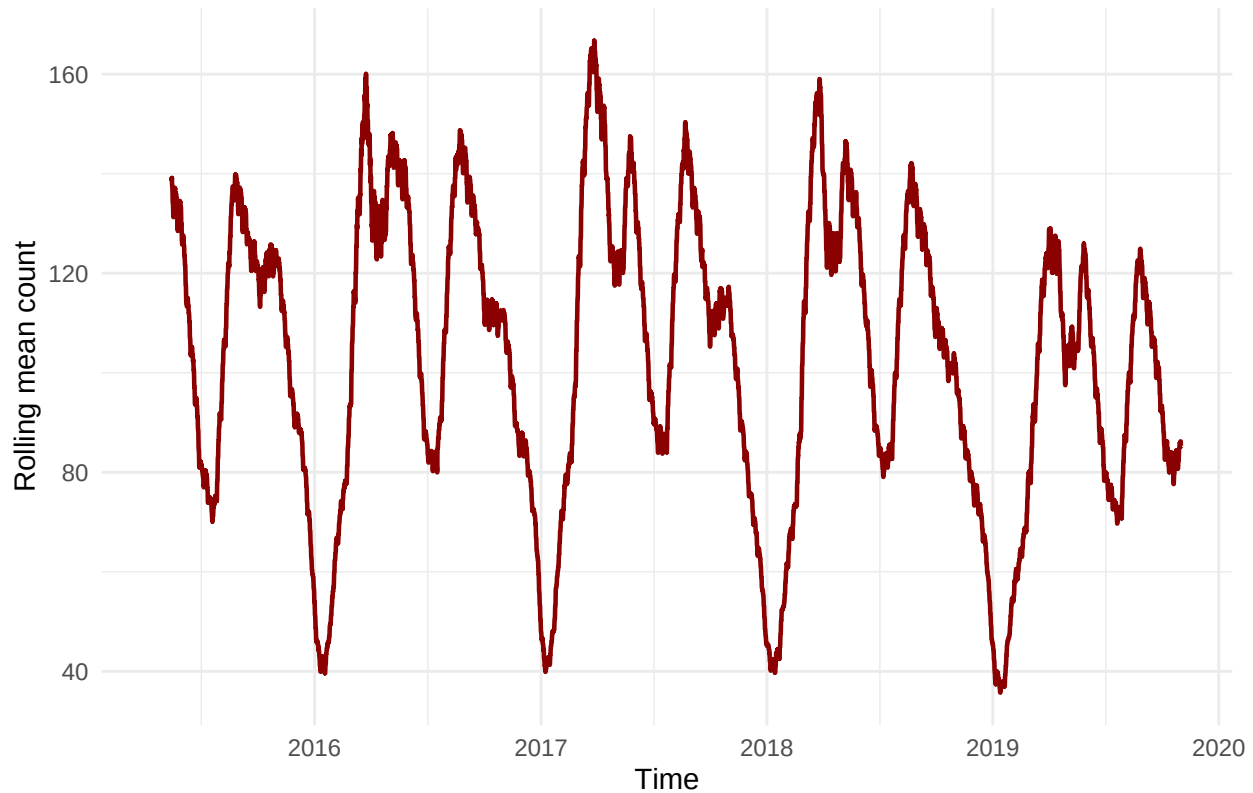
The 30-day rolling mean confirms the annual seasonal cycle already identified (repeating rises and falls aligned with the bimodal monthly pattern), but also reveals a mild overall decline in the series' baseline level: the peak of each yearly cycle is progressively lower across 2016-2019, consistent with what was seen in the year-by-year monthly plot. A Mann-Kendall trend test on monthly average counts gives $\tau = -0.156$ with $p = 0.095$: the direction is consistent with a downward trend, but the result does not reach conventional statistical significance ($p > 0.05$) at the monthly aggregation level. This suggests any long-term trend, while visually present, is modest relative to the strong seasonal and short-term variation in the series, and should be treated as a secondary effect rather than a dominant driver when modelling.

```
trend <- train_df %>%
  arrange(time) %>%
  mutate(rolling_mean_30d = zoo::rollmean(value, k = 24*30, fill = NA, align = "right")) %>%
  ggplot(aes(x = time, y = rolling_mean_30d)) +
  geom_line(color = "darkred", linewidth = 0.8) +
  labs(title = "30-day rolling mean (long-term trend)",
       x = "Time", y = "Rolling mean count") +
```

```
theme_minimal()
```

```
trend
```

30-day rolling mean (long-term trend)



```
ggsave("slides_trend.png", trend, width = 10, height = 5, dpi = 300)
```

```
monthly_trend <- train_df %>%  
  mutate(yearmonth = floor_date(time, "month")) %>%  
  group_by(yearmonth) %>%  
  summarise(mean_value = mean(value), .groups = "drop") %>%  
  arrange(yearmonth)
```

```
mk_test <- MannKendall(monthly_trend$mean_value)  
print(mk_test)
```

```
## tau = -0.156, 2-sided pvalue =0.09498
```

Lag plot

The lag plots make the strength of autocorrelation at different horizons visually explicit. At lag 1h, points cluster very tightly around the diagonal, confirming the strong short-term persistence already seen in the PACF spike at lag 1. At lag 24h and lag 168h, the cloud is visibly wider but still shows a clear positive linear relationship, with a notable fan-shaped spread at higher values - consistent with the variance-vs-mean relationship observed earlier (higher counts come with higher variability). The similarity in spread between the 24h and 168h lag plots reinforces that same-hour, same-weekday values (168h) carry nearly as much predictive information as same-hour values from the previous day (24h), supporting the inclusion of both lags as predictors in the machine learning model.

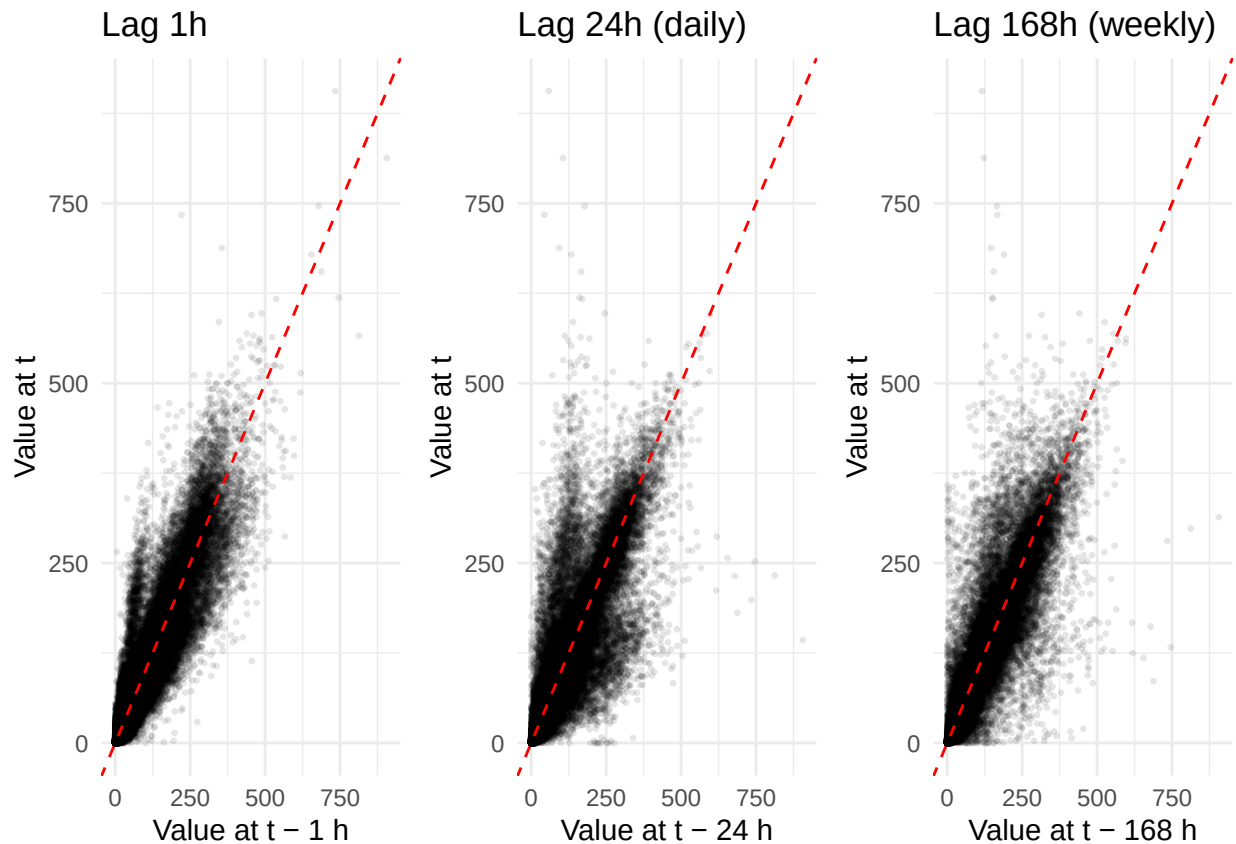
```

make_lag_plot <- function(data, lag_hours, title) {
  data %>%
    mutate(lagged = lag(value, lag_hours)) %>%
    filter(!is.na(lagged)) %>%
    ggplot(aes(x = lagged, y = value)) +
    geom_point(alpha = 0.1, size = 0.5) +
    geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
    labs(title = title, x = paste("Value at t -", lag_hours, "h"), y = "Value at t") +
    theme_minimal()
}

p1 <- make_lag_plot(train_df, 1, "Lag 1h")
p2 <- make_lag_plot(train_df, 24, "Lag 24h (daily)")
p3 <- make_lag_plot(train_df, 168, "Lag 168h (weekly)")

grid.arrange(p1, p2, p3, ncol = 3)

```



Variance stabilization

The estimated optimal Box-Cox lambda is 0.233, positioned between the log transformation (lambda = 0) and the square-root transformation (lambda = 0.5), closer to the log end. Comparing the resulting distributions, both the log and Box-Cox transformations substantially reduce the right skew seen in the raw counts, producing a more symmetric, closer-to-unimodal shape. Given the similarity between the log and the estimated optimal Box-Cox transformation, and the practical advantage of the log transformation being directly interpretable and easily invertible, the log transformation will be used as the variance-stabilising choice for models that require it, while tree-based ML models, which do not rely on residual homoscedasticity,

will use the raw scale.

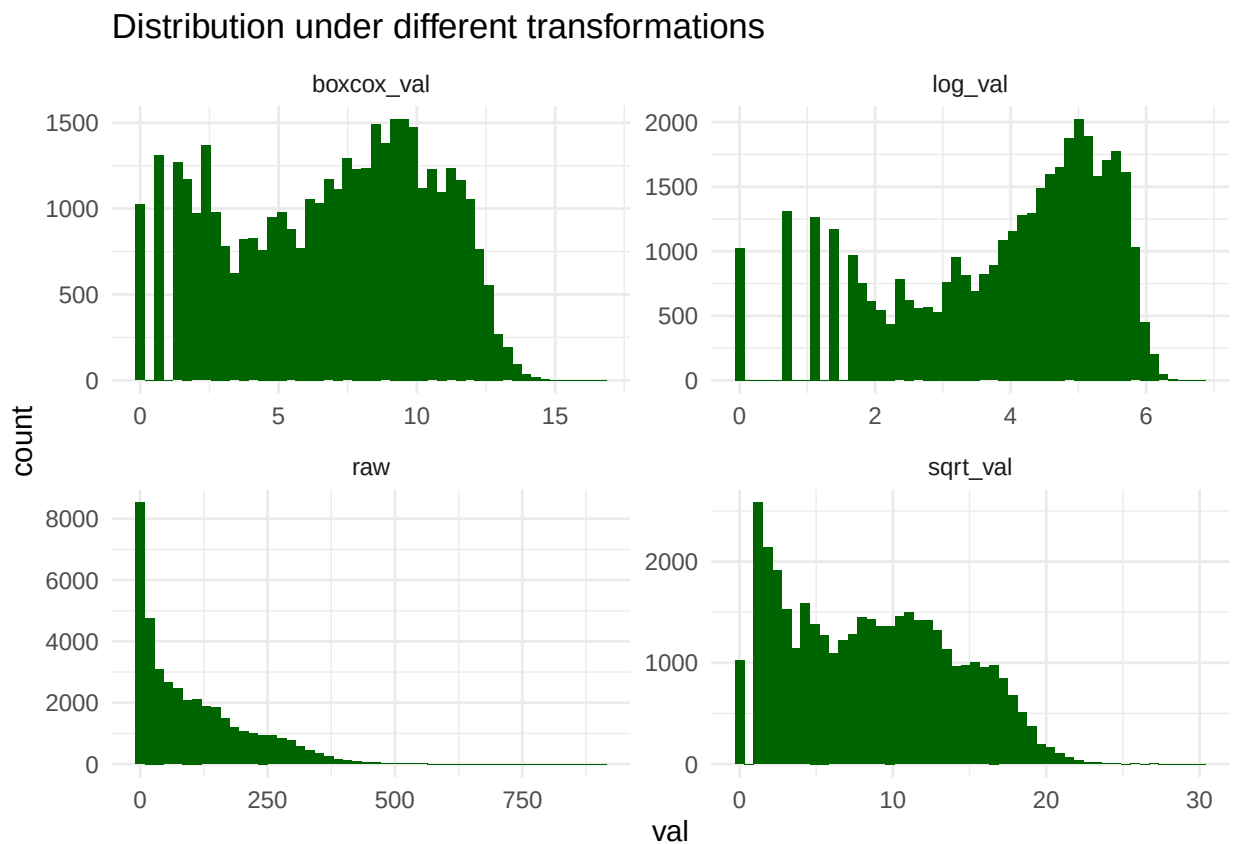
```
lambda_opt <- BoxCox.lambda(train_df$value + 1)
cat("Optimal Box-Cox lambda:", lambda_opt, "\n")
```

```
## Optimal Box-Cox lambda: 0.2330248
```

```
comparison_df <- train_df %>%
  mutate(
    raw = value,
    log_val = log(value + 1),
    sqrt_val = sqrt(value),
    boxcox_val = BoxCox(value + 1, lambda_opt)
  ) %>%
  select(raw, log_val, sqrt_val, boxcox_val) %>%
  pivot_longer(everything(), names_to = "transformation", values_to = "val")

trasf <- ggplot(comparison_df, aes(x = val)) +
  geom_histogram(bins = 50, fill = "darkgreen") +
  facet_wrap(~transformation, scales = "free") +
  labs(title = "Distribution under different transformations") +
  theme_minimal()
```

```
trasf
```



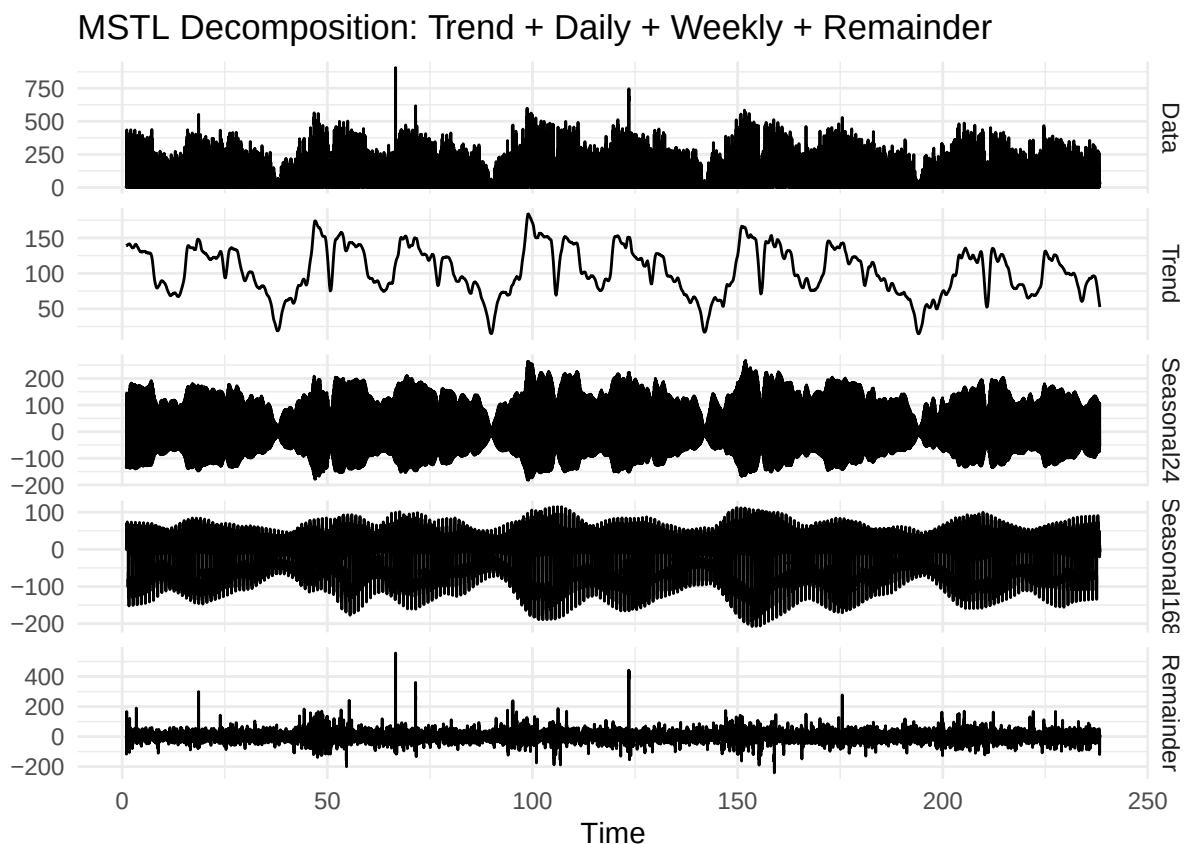
```
ggsave("slides_trasf.png", trasf, width = 10, height = 5, dpi = 300)
```

Multi-seasonal decomposition

The MSTL decomposition, accounting for both daily (24h) and weekly (168h) seasonality simultaneously, separates the series into a trend component that oscillates smoothly on a roughly annual cycle, a daily seasonal component with the largest amplitude, and a smaller but clearly structured weekly seasonal component whose amplitude itself varies over time (visible as the changing width of the Seasonal168 band). The remainder still shows occasional spikes aligned with the previously identified outlier days, confirming these are not explained by either seasonal component. Quantifying the strength of each component confirms what the visual inspection suggested: seasonal strength is very high (0.932), meaning seasonality explains the vast majority of the variance beyond the remainder, while trend strength is moderate (0.643) - present, but considerably weaker than the seasonal signal, consistent with the borderline Mann-Kendall result above.

```
msts_train <- msts(train_df$value, seasonal.periods = c(24, 168))

mstl_decomp <- mstl(msts_train)
autoplot(mstl_decomp) +
  labs(title = "MSTL Decomposition: Trend + Daily + Weekly + Remainder") +
  theme_minimal()
```



```
mstl_df <- as.data.frame(mstl_decomp)

var_remainder <- var(mstl_df$Remainder, na.rm = TRUE)
var_detrended <- var(mstl_df$Remainder + mstl_df$Seasonal24 + mstl_df$Seasonal168, na.rm = TRUE)
var_deseasonalized <- var(mstl_df$Remainder + mstl_df$Trend, na.rm = TRUE)
```

```
F_trend <- max(0, 1 - var_remainder / var_deseasonalized)
F_seasonal <- max(0, 1 - var_remainder / var_detrended)

cat("Trend strength:", round(F_trend, 3), "\n")
```

```
## Trend strength: 0.643
```

```
cat("Seasonal strength:", round(F_seasonal, 3), "\n")
```

```
## Seasonal strength: 0.932
```

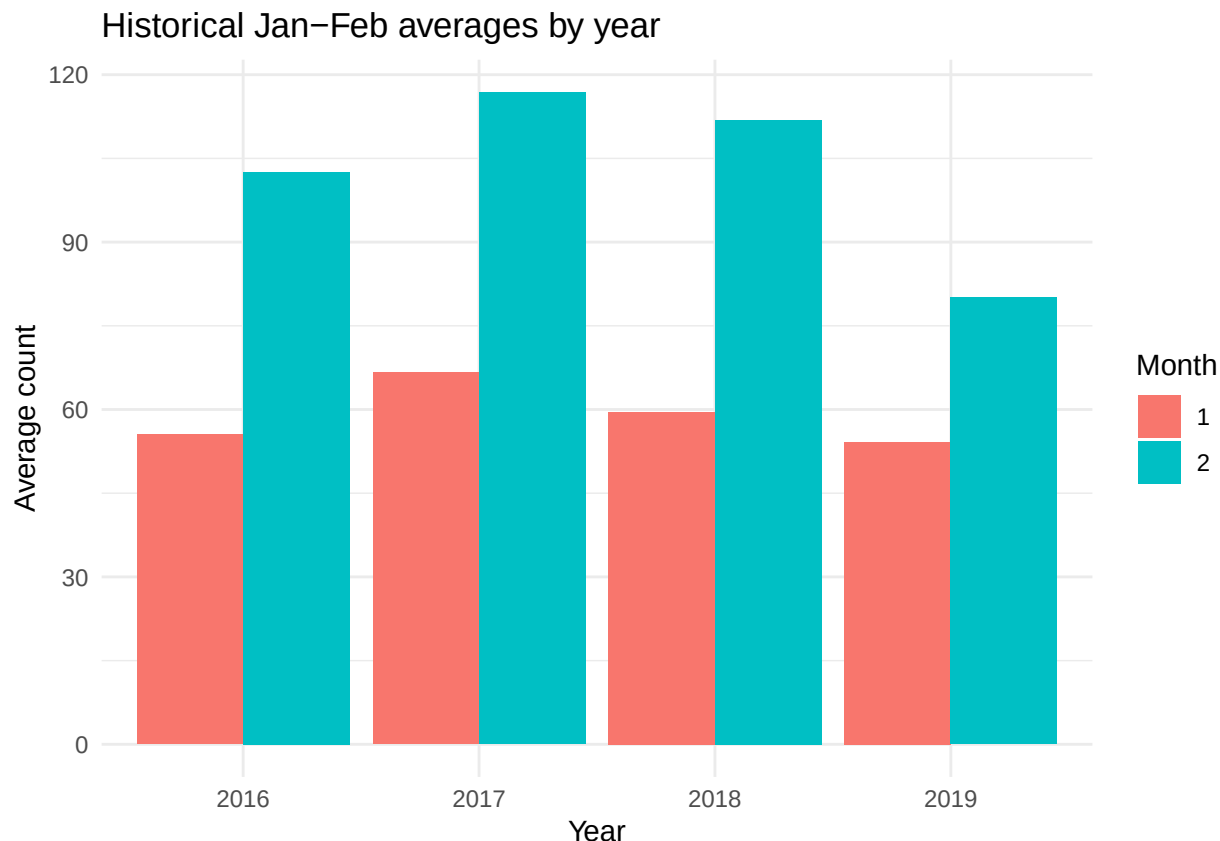
Forecasting period previous' years

```
jan_feb_historical <- df_complete %>%
  filter(month %in% c(1, 2)) %>%
  group_by(year, month) %>%
  summarise(mean_value = mean(value), sd_value = sd(value), .groups = "drop")

print(jan_feb_historical)
```

```
## # A tibble: 8 x 4
##   year month mean_value sd_value
##   <dbl> <dbl>     <dbl>     <dbl>
## 1  2016     1      55.5      53.5
## 2  2016     2     103.      101.
## 3  2017     1      66.6      62.0
## 4  2017     2     117.      119.
## 5  2018     1      59.6      62.1
## 6  2018     2     112.      118.
## 7  2019     1      54.1      53.2
## 8  2019     2      80.1      81.3
```

```
ggplot(jan_feb_historical, aes(x = factor(year), y = mean_value, fill = factor(month))) +
  geom_col(position = "dodge") +
  labs(title = "Historical Jan-Feb averages by year",
       x = "Year", y = "Average count", fill = "Month") +
  theme_minimal()
```



Modelling Strategy

The exploratory analysis established several facts that directly shape how each of the three modelling approaches will be built:

- **Seasonality is the dominant structure in the series** (seasonal strength = 0.932 from MSTL), present at two clear, stable frequencies: daily (24h, the strongest and most consistent) and weekly (168h, weaker but confirmed by both the ACF and the weekday/weekend contrast). An additional annual cycle is visible in the monthly aggregates but cannot be captured within a single hourly seasonal period and will instead be addressed indirectly, through the models' ability to adapt to slowly changing local seasonal amplitude, and through calendar-based features where applicable.
- **The trend is real but modest** (trend strength = 0.643, Mann-Kendall tau = -0.156, borderline significance), suggesting a mild downward drift in the baseline level over the years rather than a dominant driver.
- **The series is stationary once seasonality is removed** (ADF and KPSS agree on the STL residuals), meaning no unit-root differencing is strictly required by the underlying process - any differencing used by ARIMA is compensating for unmodelled seasonal amplitude changes, not a genuine stochastic trend.
- **Variance scales with the mean** (heteroscedasticity confirmed both by hour-level variance-vs-mean analysis and by the boxplots), motivating a log transformation for models sensitive to residual homoscedasticity.
- **A handful of extreme values reflect real city events**, not sensor errors, and were retained without modification; their effect on each model's residuals will be monitored rather than corrected upfront.

Model and hyperparameter selection use time-series cross-validation, not in-sample criteria or random k-fold. For each model, the relevant structural choices (ARIMA order; UCM trend degree; XGBoost tree depth, learning rate and boosting rounds) are selected via rolling-origin cross-validation on the training set, so that no

choice is made using information from the future relative to the fold being evaluated — a random k-fold would leak information across lagged features and inflate apparent accuracy. Day-of-week effects are represented, where used, through dummy variables for Friday, Saturday and Sunday (Monday-Thursday as the reference level) rather than a two-level weekday/weekend split or a smooth cyclical encoding, since the EDA showed the transition across the week is not smooth or binary (Monday-Thursday are homogeneous and high, Friday and Saturday drop by different amounts, and Sunday partially recovers toward weekday levels).

All three models will be evaluated on the validation set (November-December 2019) using RMSE and MAE (MAPE excluded due to the presence of zero counts), before producing the final forecast for the 1439 missing hours of January-February 2020.

ARIMA

Theory

ARIMA models capture linear dependence in a time series through autoregressive (AR), differencing (I), and moving average (MA) terms. Because the series here exhibits three overlapping seasonal cycles (daily, weekly, annual) that cannot be jointly represented by a single seasonal ARIMA term without an intractable number of parameters, seasonality is instead introduced through Fourier terms (harmonic regressors) at each period, combined with dummy variables for Friday, Saturday and Sunday (Monday-Thursday as reference level) to capture the non-smooth weekday transitions identified in the EDA. The non-seasonal ARIMA order (p,d,q) is selected via time-series cross-validation (rolling-origin, 4 folds) rather than in-sample AIC/BIC, to avoid selecting an order that merely overfits the training window. The model is fit on the log1p-transformed series to address the heteroscedasticity documented in the EDA. Notably, cross-validation selects d=0 (no differencing), consistent with the EDA finding that the series is stationary once seasonality is accounted for.

Setup

ARIMA setup with three seasonalities observed: daily, weekly, annually.

```
arima_train_df <- train_df
arima_val_df   <- val_df
arima_test_df  <- test_df

train_log <- log1p(arima_train_df$value)

msts_train <- msts(train_log, seasonal.periods = c(24, 168, 8766))
```

Finding the best K combination through the usage of the AIC value would take hours, so it is computed like this.

```
best_bic <- Inf
best_K <- NULL

for (kd in 1:8) {
  for (kw in 1:6) {
    for (ka in 2:4) {
      fterms <- fourier(msts_train, K = c(kd, kw, ka))
      fit_lm <- lm(as.numeric(msts_train) ~ fterms)
      bic <- BIC(fit_lm)
      if (bic < best_bic) {
        best_bic <- bic
        best_K <- c(kd, kw, ka)
      }
    }
  }
}
```

```

}

cat("Best K (via lm/BIC):", best_K, "\n")

## Best K (via lm/BIC): 5 6 4
daytype_matrix <- function(times) {
  wd <- wday(times, week_start = 1)
  cbind(
    is_fri = as.integer(wd == 5),
    is_sat = as.integer(wd == 6),
    is_sun = as.integer(wd == 7)
  )
}

fourier_best <- fourier(msts_train, K = best_K)
xreg_arima <- cbind(fourier_best, daytype_matrix(arima_train_df$time))

n_folds_arima <- 4
h_cv <- 24*61
n_total <- length(train_log)
origins <- seq(from = n_total - h_cv*n_folds_arima, to = n_total - h_cv, by = h_cv)

grid_arima <- expand.grid(p = 0:2, d = 0:1, q = 0:2)

cv_arima_metrics <- function(p, d, q) {
  errs <- c()
  for (org in origins) {
    y_tr <- train_log[1:org]
    y_te <- train_log[(org+1):(org+h_cv)]
    x_tr <- xreg_arima[1:org, , drop = FALSE]
    x_te <- xreg_arima[(org+1):(org+h_cv), , drop = FALSE]

    fit <- tryCatch(
      Arima(y_tr, order = c(p, d, q), xreg = x_tr, method = "CSS"),
      error = function(e) NULL)
    if (is.null(fit)) { errs <- c(errs, rep(NA, h_cv)); next }
    fc <- tryCatch(forecast(fit, xreg = x_te, h = h_cv)$mean,
      error = function(e) rep(NA, h_cv))
    errs <- c(errs, expm1(y_te) - pmax(expm1(as.numeric(fc)), 0))
  }
  c(mae = mean(abs(errs), na.rm = TRUE), rmse = sqrt(mean(errs^2, na.rm = TRUE)))
}

results_arima <- grid_arima %>%
  mutate(res = purrr::pmap(list(p, d, q), cv_arima_metrics)) %>%
  tidyr::unnest_wider(res) %>%
  arrange(mae)

print(results_arima)

## # A tibble: 18 x 5
##       p     d     q   mae rmse
##   <int> <int> <int> <dbl> <dbl>

```

```
## 1      0      0      1 29.0 46.8
## 2      0      0      0 29.0 46.9
## 3      0      0      2 29.1 46.8
## 4      2      0      1 29.1 46.8
## 5      1      0      0 29.1 46.8
## 6      1      0      1 29.1 46.8
## 7      2      0      0 29.1 46.8
## 8      1      0      2 29.1 46.8
## 9      2      0      2 29.1 46.9
## 10     2      1      2 46.1 78.1
## 11     1      1      2 47.0 79.3
## 12     0      1      0 47.3 80.0
## 13     0      1      2 47.4 79.6
## 14     2      1      1 47.4 79.9
## 15     2      1      0 48.1 80.8
## 16     1      1      1 51.0 83.9
## 17     0      1      1 52.0 85.2
## 18     1      1      0 52.4 85.5
```

```
best_arima <- results_arima[1, ]
cat("Best ARIMA order from CV: (", best_arima$p, best_arima$d, best_arima$q, ")\n")
```

```
## Best ARIMA order from CV: ( 0 0 1 )
```

```
fit_arima <- Arima(msts_train, order = c(best_arima$p, best_arima$d, best_arima$q),
                  xreg = xreg_arima)
```

```
summary(fit_arima)
```

```
## Series: msts_train
```

```
## Regression with ARIMA(0,0,1) errors
```

```
##
```

```
## Coefficients:
```

```
##          ma1 intercept      S1-24      C1-24      S2-24      C2-24      S3-24      C3-24
##          0.5953      3.6780 -1.0911 -1.3292 -0.5242  0.0322  0.0134  0.1673
## s.e.      0.0031      0.0157  0.0073  0.0073  0.0071  0.0071  0.0068  0.0068
##          S4-24      C4-24      S5-24      C5-24      S1-168      C1-168      S2-168      C2-168
##          0.0418 -0.0021 -0.0368 -0.0164 -0.0369  0.4991  0.0944 -0.2097
## s.e.      0.0064  0.0064  0.0059  0.0059  0.0125  0.0234  0.0135  0.0119
##          S3-168      C3-168      S4-168      C4-168      S5-168      C5-168      S6-168      C6-168
##          -0.0322  0.0559 -0.0428 -0.0070  0.0819  0.0912 -0.1192 -0.1280
## s.e.      0.0085  0.0115  0.0080  0.0099  0.0086  0.0082  0.0075  0.0082
##          S1-8766      C1-8766      S2-8766      C2-8766      S3-8766      C3-8766      S4-8766      C4-8766
##          0.2403  0.1169 -0.1506  0.2947 -0.0882 -0.0280  0.1401 -0.1003
## s.e.      0.0074  0.0073  0.0074  0.0074  0.0074  0.0074  0.0074  0.0074
##          is_fri is_sat is_sun
##          0.3162  0.4896  0.1174
## s.e.      0.0403  0.0463  0.0403
##
```

```
## sigma^2 = 0.4231: log likelihood = -39401.11
```

```
## AIC=78874.21 AICc=78874.28 BIC=79183.57
```

```
##
```

```
## Training set error measures:
```

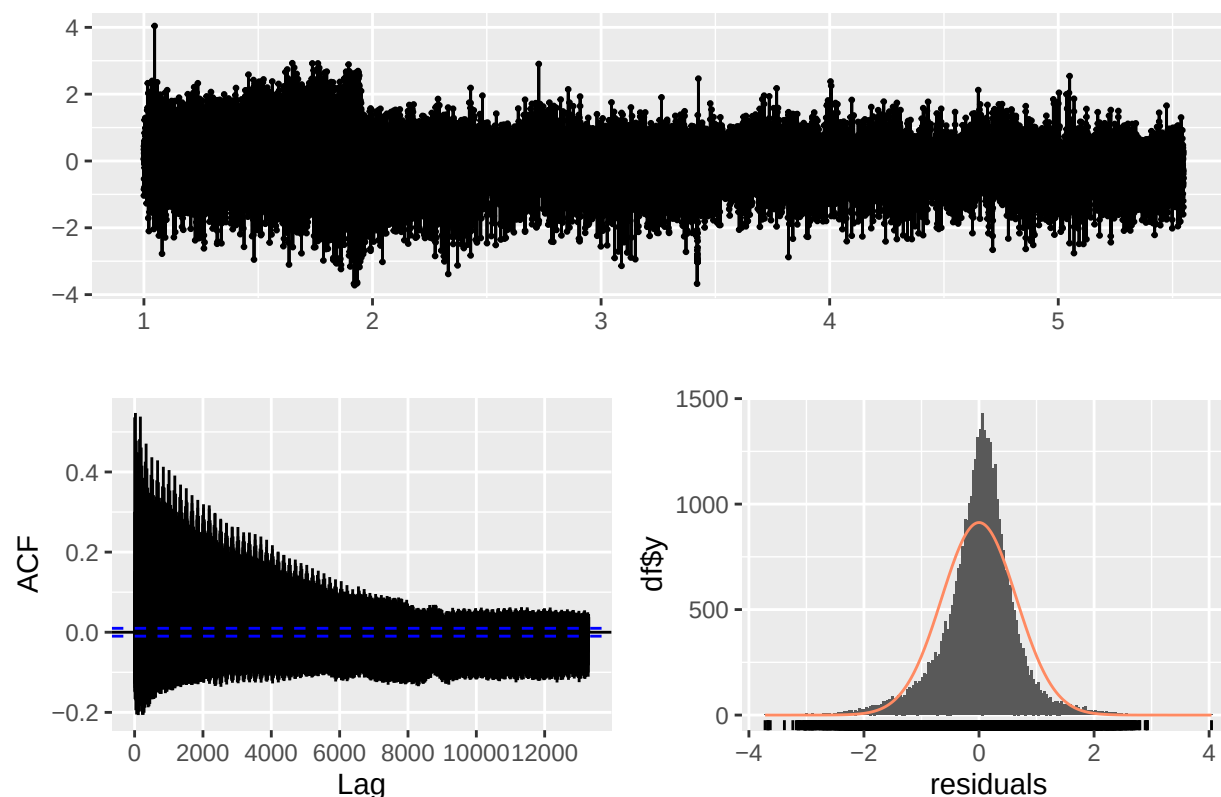
```
##          ME          RMSE          MAE MPE MAPE          MASE          ACF1
## Training set -8.69184e-06 0.6501476 0.4630661 NaN Inf 0.270041 0.3006374
```

The selected model is a Regression with ARIMA(0,0,1) errors, chosen by cross-validation over Fourier orders $K=(5,6,4)$ for the daily, weekly, and annual periods, together with dummy regressors for Friday, Saturday and Sunday. No differencing is required ($d=0$), consistent with the EDA finding that the series is stationary once seasonality is removed. The daily and weekly harmonics are estimated with high precision: standard errors are an order of magnitude smaller than the coefficients themselves ($S1-24 = -1.09$, $se = 0.007$), consistent with the hundreds to thousands of complete cycles observed for these periods. The annual harmonics tell a different story: several coefficients ($S1-8766 = 0.240$, $se = 0.007$) still have non-negligible standard errors relative to the coefficient, reflecting that only about 4.5 complete annual cycles are available in the training data to estimate the annual shape. The day-type dummies are all estimated with high confidence ($is_fri = 0.316$, $is_sat = 0.490$, $is_sun = 0.117$, all with $se \sim 0.04$), confirming that Saturday has the largest deviation from the Monday-Thursday baseline.

Residual diagnostics

```
checkresiduals(fit_arma)
```

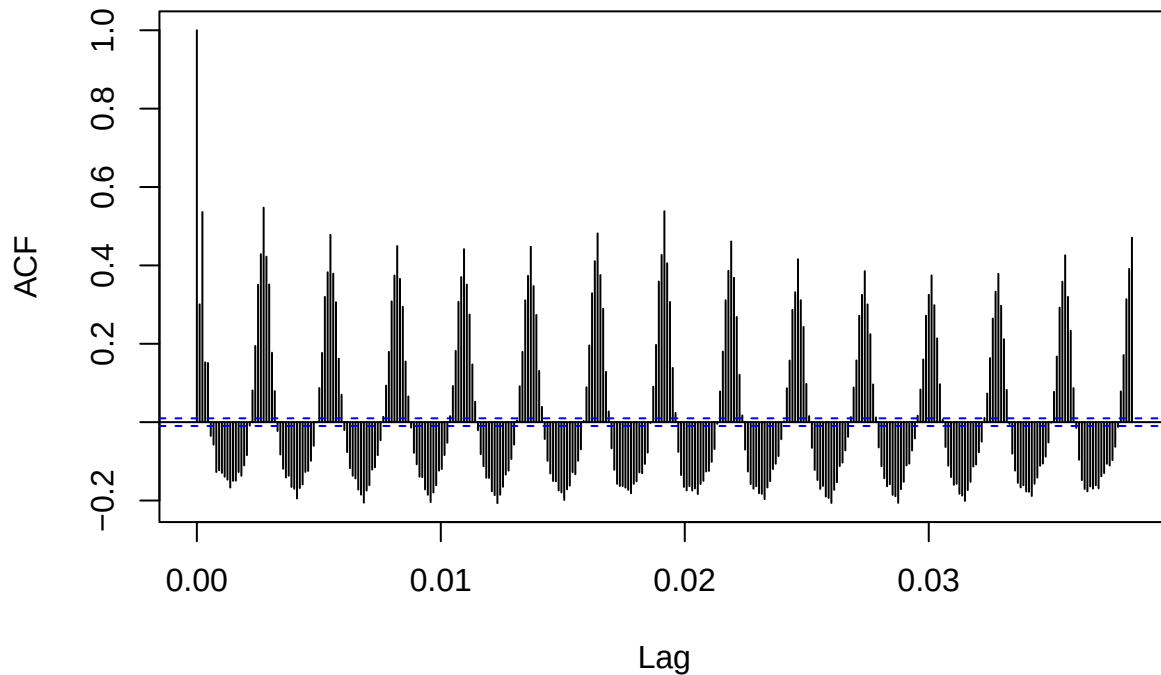
Residuals from Regression with ARIMA(0,0,1) errors



```
##
##  Ljung-Box test
##
## data:  Residuals from Regression with ARIMA(0,0,1) errors
## Q* = 3876772, df = 7972, p-value < 2.2e-16
##
## Model df: 1.    Total lags used: 7973
lb_test <- Box.test(residuals(fit_arma), lag = 24*7, type = "Ljung-Box", fitdf = length(fit_arma$coef)
print(lb_test)
```

```
##
## Box-Ljung test
##
## data: residuals(fit_arima)
## X-squared = 303041, df = 133, p-value < 2.2e-16
acf_arima <- acf(residuals(fit_arima), lag.max = 24*14, main = "ACF of ARIMA residuals (log scale)")
```

ACF of ARIMA residuals (log scale)



```
acf_arima

##
## Autocorrelations of series 'residuals(fit_arima)', by lag
##
## 0.000000 0.000114 0.000228 0.000342 0.000456 0.000570 0.000684 0.000799
## 1.000 0.301 0.536 0.153 0.151 -0.035 -0.058 -0.127
## 0.000913 0.001027 0.001141 0.001255 0.001369 0.001483 0.001597 0.001711
## -0.123 -0.129 -0.139 -0.147 -0.167 -0.150 -0.149 -0.127
## 0.001825 0.001939 0.002053 0.002167 0.002282 0.002396 0.002510 0.002624
## -0.137 -0.110 -0.084 -0.009 0.081 0.194 0.351 0.429
## 0.002738 0.002852 0.002966 0.003080 0.003194 0.003308 0.003422 0.003536
## 0.547 0.422 0.351 0.177 0.079 -0.023 -0.082 -0.119
## 0.003650 0.003765 0.003879 0.003993 0.004107 0.004221 0.004335 0.004449
## -0.141 -0.137 -0.165 -0.170 -0.195 -0.168 -0.159 -0.127
## 0.004563 0.004677 0.004791 0.004905 0.005019 0.005133 0.005248 0.005362
## -0.124 -0.099 -0.060 0.000 0.087 0.177 0.320 0.382
## 0.005476 0.005590 0.005704 0.005818 0.005932 0.006046 0.006160 0.006274
## 0.478 0.379 0.306 0.162 0.070 -0.020 -0.076 -0.117
```

##	0.006388	0.006502	0.006616	0.006731	0.006845	0.006959	0.007073	0.007187
##	-0.137	-0.144	-0.172	-0.185	-0.205	-0.175	-0.161	-0.121
##	0.007301	0.007415	0.007529	0.007643	0.007757	0.007871	0.007985	0.008099
##	-0.115	-0.084	-0.046	0.014	0.093	0.179	0.308	0.374
##	0.008214	0.008328	0.008442	0.008556	0.008670	0.008784	0.008898	0.009012
##	0.449	0.366	0.295	0.155	0.066	-0.014	-0.078	-0.108
##	0.009126	0.009240	0.009354	0.009468	0.009582	0.009697	0.009811	0.009925
##	-0.139	-0.140	-0.172	-0.185	-0.204	-0.180	-0.161	-0.124
##	0.010039	0.010153	0.010267	0.010381	0.010495	0.010609	0.010723	0.010837
##	-0.119	-0.084	-0.053	0.015	0.092	0.182	0.307	0.370
##	0.010951	0.011065	0.011180	0.011294	0.011408	0.011522	0.011636	0.011750
##	0.442	0.351	0.275	0.147	0.052	-0.021	-0.082	-0.112
##	0.011864	0.011978	0.012092	0.012206	0.012320	0.012434	0.012548	0.012663
##	-0.142	-0.142	-0.176	-0.186	-0.207	-0.185	-0.165	-0.134
##	0.012777	0.012891	0.013005	0.013119	0.013233	0.013347	0.013461	0.013575
##	-0.124	-0.094	-0.057	0.005	0.092	0.179	0.311	0.373
##	0.013689	0.013803	0.013917	0.014031	0.014146	0.014260	0.014374	0.014488
##	0.447	0.347	0.274	0.131	0.039	-0.043	-0.099	-0.131
##	0.014602	0.014716	0.014830	0.014944	0.015058	0.015172	0.015286	0.015400
##	-0.150	-0.151	-0.174	-0.179	-0.198	-0.171	-0.162	-0.128
##	0.015514	0.015629	0.015743	0.015857	0.015971	0.016085	0.016199	0.016313
##	-0.133	-0.098	-0.069	0.003	0.089	0.196	0.329	0.411
##	0.016427	0.016541	0.016655	0.016769	0.016883	0.016997	0.017112	0.017226
##	0.482	0.375	0.289	0.128	0.027	-0.067	-0.122	-0.157
##	0.017340	0.017454	0.017568	0.017682	0.017796	0.017910	0.018024	0.018138
##	-0.163	-0.163	-0.167	-0.173	-0.181	-0.158	-0.151	-0.128
##	0.018252	0.018366	0.018480	0.018595	0.018709	0.018823	0.018937	0.019051
##	-0.130	-0.107	-0.078	-0.002	0.091	0.197	0.359	0.427
##	0.019165	0.019279	0.019393	0.019507	0.019621	0.019735	0.019849	0.019963
##	0.538	0.406	0.307	0.138	0.023	-0.076	-0.135	-0.165
##	0.020078	0.020192	0.020306	0.020420	0.020534	0.020648	0.020762	0.020876
##	-0.174	-0.163	-0.174	-0.169	-0.183	-0.158	-0.150	-0.126
##	0.020990	0.021104	0.021218	0.021332	0.021446	0.021561	0.021675	0.021789
##	-0.124	-0.101	-0.068	-0.004	0.078	0.180	0.311	0.386
##	0.021903	0.022017	0.022131	0.022245	0.022359	0.022473	0.022587	0.022701
##	0.461	0.368	0.268	0.121	0.017	-0.070	-0.126	-0.158
##	0.022815	0.022930	0.023044	0.023158	0.023272	0.023386	0.023500	0.023614
##	-0.170	-0.163	-0.181	-0.183	-0.196	-0.167	-0.151	-0.120
##	0.023728	0.023842	0.023956	0.024070	0.024184	0.024298	0.024413	0.024527
##	-0.106	-0.087	-0.044	0.002	0.086	0.157	0.287	0.332
##	0.024641	0.024755	0.024869	0.024983	0.025097	0.025211	0.025325	0.025439
##	0.416	0.311	0.243	0.098	0.015	-0.065	-0.122	-0.148
##	0.025553	0.025667	0.025781	0.025896	0.026010	0.026124	0.026238	0.026352
##	-0.170	-0.163	-0.190	-0.194	-0.206	-0.173	-0.153	-0.112
##	0.026466	0.026580	0.026694	0.026808	0.026922	0.027036	0.027150	0.027264
##	-0.103	-0.072	-0.037	0.013	0.089	0.158	0.272	0.325
##	0.027379	0.027493	0.027607	0.027721	0.027835	0.027949	0.028063	0.028177
##	0.385	0.300	0.224	0.096	0.012	-0.065	-0.112	-0.143
##	0.028291	0.028405	0.028519	0.028633	0.028747	0.028862	0.028976	0.029090
##	-0.164	-0.158	-0.186	-0.189	-0.206	-0.171	-0.152	-0.110
##	0.029204	0.029318	0.029432	0.029546	0.029660	0.029774	0.029888	0.030002
##	-0.106	-0.072	-0.042	0.017	0.083	0.160	0.272	0.325
##	0.030116	0.030230	0.030345	0.030459	0.030573	0.030687	0.030801	0.030915
##	0.374	0.299	0.214	0.097	0.006	-0.063	-0.111	-0.140

```
## 0.031029 0.031143 0.031257 0.031371 0.031485 0.031599 0.031713 0.031828
## -0.160 -0.157 -0.183 -0.187 -0.201 -0.173 -0.152 -0.113
## 0.031942 0.032056 0.032170 0.032284 0.032398 0.032512 0.032626 0.032740
## -0.108 -0.077 -0.050 0.011 0.073 0.164 0.264 0.333
## 0.032854 0.032968 0.033082 0.033196 0.033311 0.033425 0.033539 0.033653
## 0.379 0.297 0.212 0.082 -0.004 -0.080 -0.124 -0.155
## 0.033767 0.033881 0.033995 0.034109 0.034223 0.034337 0.034451 0.034565
## -0.166 -0.161 -0.177 -0.178 -0.188 -0.157 -0.141 -0.111
## 0.034679 0.034794 0.034908 0.035022 0.035136 0.035250 0.035364 0.035478
## -0.106 -0.088 -0.052 -0.001 0.077 0.167 0.292 0.358
## 0.035592 0.035706 0.035820 0.035934 0.036048 0.036162 0.036277 0.036391
## 0.426 0.320 0.233 0.087 -0.015 -0.096 -0.148 -0.170
## 0.036505 0.036619 0.036733 0.036847 0.036961 0.037075 0.037189 0.037303
## -0.177 -0.164 -0.170 -0.161 -0.169 -0.138 -0.130 -0.106
## 0.037417 0.037531 0.037645 0.037760 0.037874 0.037988 0.038102 0.038216
## -0.111 -0.097 -0.063 -0.012 0.078 0.171 0.314 0.391
## 0.038330
## 0.471
```

The Ljung-Box test strongly rejects the null hypothesis of no residual autocorrelation ($Q^* = 3876772$, $p < 2.2e-16$), and the ACF of the residuals decays slowly with visible oscillations at multiples of 24 and 168 lags. This indicates that, despite the Fourier terms and day-type dummies, some seasonal structure remains unexplained in the residuals, likely a consequence of the weak annual component diluting the overall fit rather than a failure to capture daily/weekly seasonality, which the ACF of the raw series had already shown to be very strong. Given the large sample size (~40,000 hourly observations), the Ljung-Box test is expected to be highly sensitive even to small residual autocorrelation; the practical impact on forecast accuracy is better assessed via the validation MAE/RMSE below.

Forecast

```
h_val <- nrow(arima_val_df)
fourier_val <- fourier(msts_train, K = best_K, h = h_val)
xreg_val <- cbind(fourier_val, daytype_matrix(arima_val_df$time))

fc_arima_val <- forecast(fit_arima, xreg = xreg_val, h = h_val, biasadj = TRUE)

pred_val_arima <- pmax(expm1(fc_arima_val$mean), 0)
lower_val_arima <- pmax(expm1(fc_arima_val$lower[, "95%"]), 0)
upper_val_arima <- expm1(fc_arima_val$upper[, "95%"])

mae_arima <- mean(abs(arima_val_df$value - pred_val_arima))
rmse_arima <- sqrt(mean((arima_val_df$value - pred_val_arima)^2))

cat("ARIMA on validation:\n")

## ARIMA on validation:
cat("MAE:", round(mae_arima, 2), " RMSE:", round(rmse_arima, 2), "\n")

## MAE: 21.19 RMSE: 33.39
```

On the validation set, the model achieves MAE = 21.19 and RMSE = 33.39, though not the best of the three candidate models (see final comparison).

```
val_comparison_arima <- arima_val_df %>%
  select(time, value) %>%
```

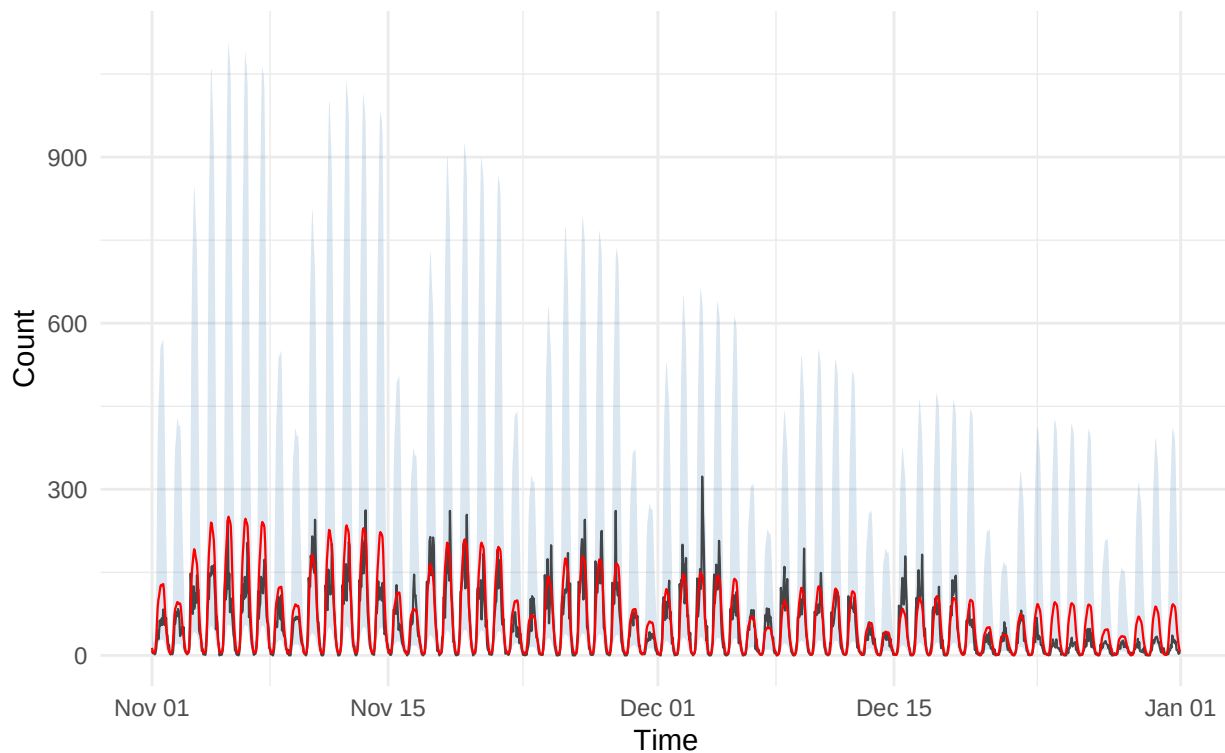
```
mutate(predicted = pred_val_arima, lower = lower_val_arima, upper = upper_val_arima)

p_arima_val <- ggplot(val_comparison_arima, aes(x = time)) +
  geom_ribbon(aes(ymin = lower, ymax = upper), fill = "steelblue", alpha = 0.2) +
  geom_line(aes(y = value), color = "black", linewidth = 0.4, alpha = 0.7) +
  geom_line(aes(y = predicted), color = "red", linewidth = 0.4) +
  labs(title = "ARIMA forecast vs actual (validation set: Nov-Dec 2019)",
       subtitle = "Black = actual, Red = ARIMA forecast, Blue band = 95% PI",
       x = "Time", y = "Count") +
  theme_minimal()

p_arima_val
```

ARIMA forecast vs actual (validation set: Nov-Dec 2019)

Black = actual, Red = ARIMA forecast, Blue band = 95% PI



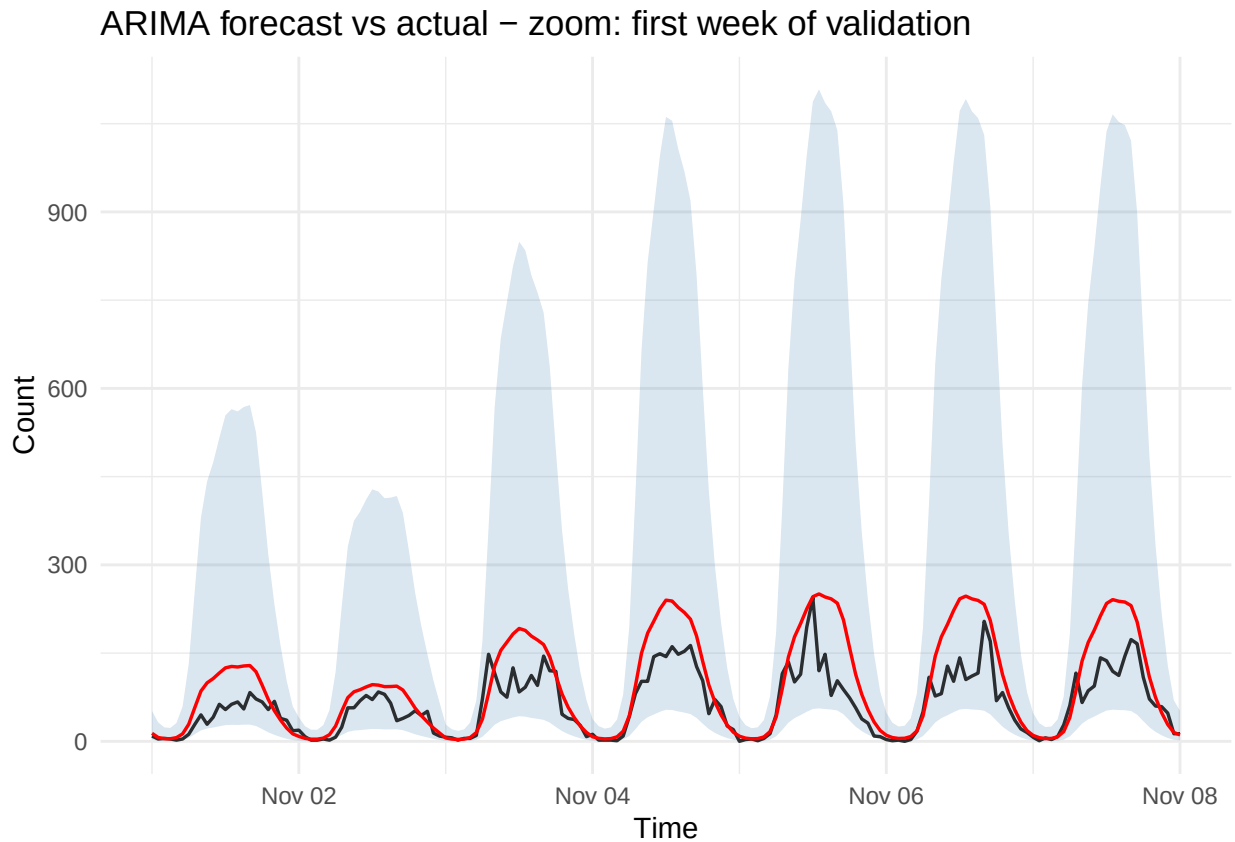
```
ggsave("slides_arima_validation.png", p_arima_val, width = 10, height = 5, dpi = 300)
```

The 95% prediction interval widens rapidly with the forecast horizon, reaching values as high as ~900-1000 by the end of the validation window while actual counts remain below 300 throughout. This reflects the compounding uncertainty typical of a Fourier-regression ARIMA model projected over a long horizon (1464 hours), and should be kept in mind when interpreting the point forecast: the model is confident about the near-term shape but increasingly uncertain about the exact level further out.

```
p_arima_val_zoom <- val_comparison_arima %>%
  filter(time <= min(time) + days(7)) %>%
  ggplot(aes(x = time)) +
  geom_ribbon(aes(ymin = lower, ymax = upper), fill = "steelblue", alpha = 0.2) +
  geom_line(aes(y = value), color = "black", linewidth = 0.6, alpha = 0.8) +
  geom_line(aes(y = predicted), color = "red", linewidth = 0.6) +
```

```
labs(title = "ARIMA forecast vs actual - zoom: first week of validation",
     x = "Time", y = "Count") +
theme_minimal()
```

p_arima_val_zoom



At this finer resolution, the model tracks the daily and weekly oscillation pattern closely, correctly reproducing the timing and relative height of peaks and troughs. This confirms that the weakness identified in the model summary lies specifically in the annual component and overall baseline level, not in the shorter-period seasonal structure, which is captured accurately.

```
full_train_log <- log1p(c(arima_train_df$value, arima_val_df$value))
msts_full <- msts(full_train_log, seasonal.periods = c(24, 168, 8766))
fourier_full <- fourier(msts_full, K = best_K)
xreg_full <- cbind(fourier_full, daytype_matrix(c(arima_train_df$time, arima_val_df$time)))

fit_arima_final <- Arima(msts_full, xreg = xreg_full, model = fit_arima)

h_test <- nrow(arima_test_df)
fourier_test <- fourier(msts_full, K = best_K, h = h_test)
xreg_test <- cbind(fourier_test, daytype_matrix(arima_test_df$time))
fc_arima_test <- forecast(fit_arima_final, xreg = xreg_test, h = h_test, biasadj = TRUE)

pred_test_arima <- pmax(expm1(fc_arima_test$mean), 0)

arima_forecast_final <- arima_test_df %>%
```

```

select(time) %>%
mutate(value_forecast_arima = as.numeric(pred_test_arima))

arima_forecast_final %>%
mutate(month = month(time)) %>%
group_by(month) %>%
summarise(mean_forecast = mean(value_forecast_arima))

```

```

## # A tibble: 2 x 2
##   month mean_forecast
##   <dbl>         <dbl>
## 1     1           44.2
## 2     2           86.2

```

The forecast for the missing period shows mean values of 44.2 for January and 86.2 for February, reasonably close to the historical averages for the same months (54-67 and 80-117 respectively). Compared to the earlier version of this model without cross-validated order selection and day-type dummies, the annual-cycle underestimation is substantially reduced, though the model still slightly undershoots the historical range, consistent with the annual Fourier terms being only weakly identified from ~4.5 observed cycles. This is treated as an acknowledged, if reduced, limitation of the ARIMA-family approach given the available data, rather than a fitting error.

Diagnostics

The outlier days flagged in the EDA show residuals moderately elevated relative to the rest of the series (mean absolute residual 0.605 vs 0.463 overall, a 31% increase), but none rank among the top 0.1% most extreme residuals. This is consistent with the EDA finding that these days reflect a coherent, broadly elevated pattern across the whole day rather than isolated implausible spikes: the model absorbs part of the elevation without being thrown off by a single extreme point.

```

outlier_days_eda <- as_date(c("2016-07-17", "2017-08-19", "2016-08-20",
                             "2017-02-26", "2017-02-27"))

outlier_resid_summary <- function(df) {
  n_top <- max(1, round(0.001 * nrow(df)))
  top <- df %>% slice_max(abs(resid), n = n_top)
  tibble(
    mean_abs_all = mean(abs(df$resid), na.rm = TRUE),
    mean_abs_outlier_days = df %>% filter(is_outlier_day) %>%
      summarise(m = mean(abs(resid), na.rm = TRUE)) %>% pull(m),
    pct_outlier_days_in_top = mean(top$is_outlier_day, na.rm = TRUE)
  )
}

resid_arima_df <- tibble(
  time = arima_train_df$time,
  resid = as.numeric(residuals(fit_arima))
) %>% mutate(day = as_date(time), is_outlier_day = day %in% outlier_days_eda)
outlier_resid_summary(resid_arima_df)

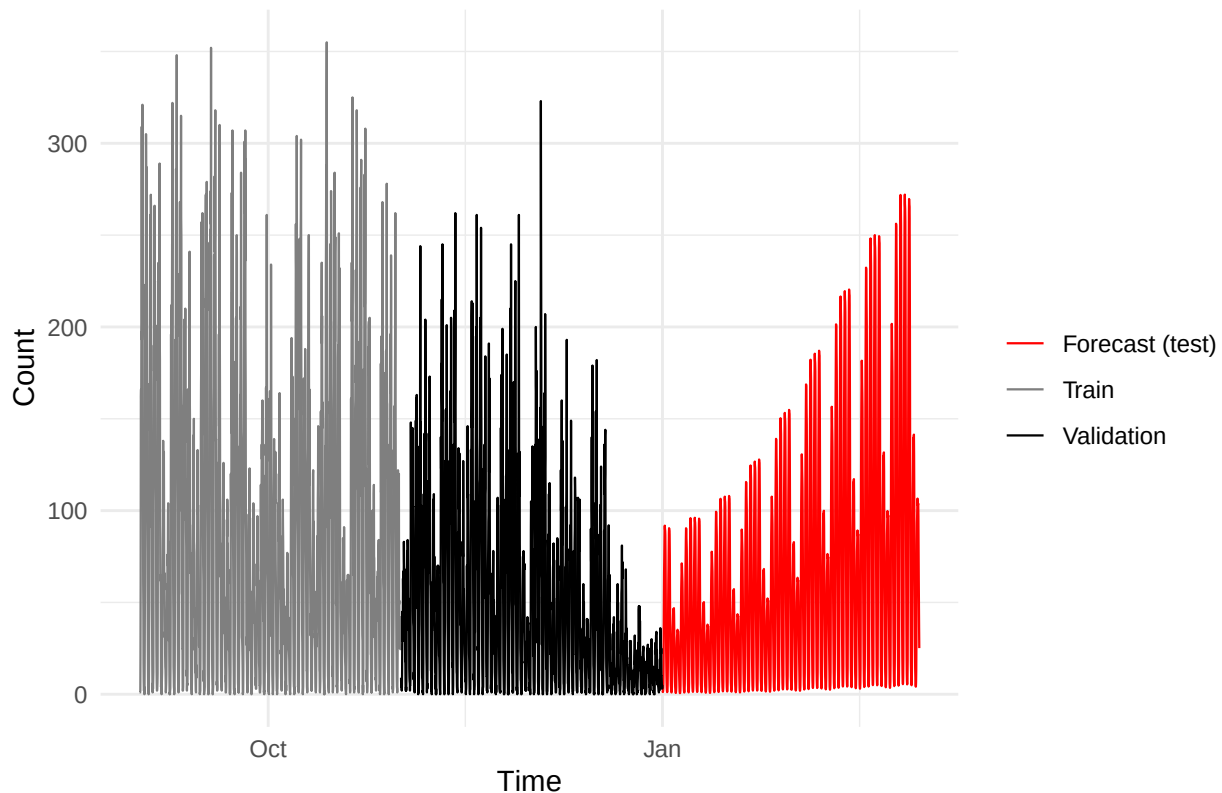
## # A tibble: 1 x 3
##   mean_abs_all mean_abs_outlier_days pct_outlier_days_in_top
##   <dbl>         <dbl>         <dbl>
## 1     0.463           0.605           0

```

```
p_arima_final <- bind_rows(
  arima_train_df %>% select(time, value) %>% mutate(type = "Train"),
  arima_val_df %>% select(time, value) %>% mutate(type = "Validation"),
  arima_forecast_final %>% rename(value = value_forecast_arima) %>% mutate(type = "Forecast (test)")
) %>%
  filter(time >= as.POSIXct("2019-09-01", tz = "UTC")) %>%
  ggplot(aes(x = time, y = value, color = type)) +
  geom_line(linewidth = 0.4) +
  scale_color_manual(values = c("Train" = "grey50", "Validation" = "black", "Forecast (test)" = "red"))
labs(title = "ARIMA: final forecast for missing period (Jan-Feb 2020)",
     x = "Time", y = "Count", color = "") +
  theme_minimal()
```

p_arima_final

ARIMA: final forecast for missing period (Jan-Feb 2020)



```
ggsave("slides_arima_final_forecast.png", p_arima_final, width = 10, height = 5, dpi = 300)
```

The plot makes the underestimation visually explicit: the forecast (red) sits at a level closer to the December trough than to the historical January-February range, despite reproducing the correct daily/weekly oscillation shape inherited from the fitted model.

Unobserved Components (UCM)

Theory

The UCM specification combines a local-level trend (degree 1) with a trigonometric daily seasonal component (12 harmonics) inside the state-space model. Weekly and annual seasonality are instead incorporated as fixed empirical offsets (on the log scale, detrended by year) computed outside the state-space model and added back to the forecasts. This choice reflects a practical constraint: a genuine stochastic weekly seasonal term (`SSMseasonal(period=168)`) was tested in combination with the daily seasonal block, but repeatedly failed during state-matrix construction in KFAS regardless of the number of harmonics specified, and required computation times unsuitable for iterative model selection. The day-type dummy regressors (`is_fri`, `is_sat`, `is_sun`) are excluded from the final specification, as they are redundant once the weekly offset is included.

Setup

The BFGS optimizer converges successfully (convergence code 0), confirming that the trend+daily specification remains numerically well-behaved once both the monthly and weekly components are removed from the series prior to fitting.

```
monthly_index <- train_df %>%
  group_by(year, month) %>%
  summarise(log_mean_month = mean(log1p(value)), .groups = "drop") %>%
  group_by(year) %>%
  mutate(log_mean_year = mean(log_mean_month)) %>%
  ungroup() %>%
  mutate(month_deviation = log_mean_month - log_mean_year) %>%
  group_by(month) %>%
  summarise(month_offset = mean(month_deviation), .groups = "drop")

weekly_index <- train_df %>%
  mutate(wd_num = wday(time, week_start = 1)) %>%
  group_by(year, wd_num) %>%
  summarise(log_mean_wd = mean(log1p(value)), .groups = "drop") %>%
  group_by(year) %>%
  mutate(log_mean_year = mean(log_mean_wd)) %>%
  ungroup() %>%
  mutate(wd_deviation = log_mean_wd - log_mean_year) %>%
  group_by(wd_num) %>%
  summarise(wd_offset = mean(wd_deviation), .groups = "drop")

ucm_train_df <- train_df %>%
  left_join(monthly_index %>% select(month, month_offset), by = "month") %>%
  mutate(wd_num = wday(time, week_start = 1)) %>%
  left_join(weekly_index, by = "wd_num")

ucm_val_df <- val_df %>%
  left_join(monthly_index %>% select(month, month_offset), by = "month") %>%
  mutate(wd_num = wday(time, week_start = 1)) %>%
  left_join(weekly_index, by = "wd_num")

ucm_test_df <- test_df %>%
  left_join(monthly_index %>% select(month, month_offset), by = "month") %>%
  mutate(wd_num = wday(time, week_start = 1)) %>%
  left_join(weekly_index, by = "wd_num")
```

```
y_train <- log1p(ucm_train_df$value) - ucm_train_df$month_offset - ucm_train_df$wd_offset
```

Residual diagnostics

The residual ACF displays regular spikes at multiples of 24 lags, reflecting the fact that the 12-harmonic daily seasonal block approximates but does not perfectly reproduce the diurnal shape. This residual structure is unrelated to the weekly component, since the weekly offset operates on the day-level mean rather than on the intraday cycle shape. The residual distribution remains leptokurtic relative to the normal reference, with heavy tails consistent with the retained outlier days from the EDA.

```
n_folds_ucm <- 4
h_cv <- 24*61
n_total <- length(y_train)
origins <- seq(from = n_total - h_cv*n_folds_ucm, to = n_total - h_cv, by = h_cv)

cv_ucm_metrics <- function(trend_degree) {
  errs <- c()
  for (org in origins) {
    y_tr <- y_train[1:org]
    y_te_actual <- ucm_train_df$value[(org+1):(org+h_cv)]
    offset_te <- ucm_train_df$month_offset[(org+1):(org+h_cv)] +
      ucm_train_df$wd_offset[(org+1):(org+h_cv)]
    y_extended <- c(y_tr, rep(NA, h_cv))

    f <- y_extended ~
      SSMtrend(degree = trend_degree, Q = list(matrix(NA), matrix(NA))) +
      SSMseasonal(period = 24, sea.type = "trigonometric")
    model_fold <- SSModel(f, H = matrix(NA))

    fit_fold <- tryCatch(
      fitSSM(model_fold, inits = rep(log(var(y_tr, na.rm=TRUE))/10), length(model_fold$Q)+1),
      method = "BFGS", control = list(maxit = 100)),
      error = function(e) NULL)
    if (is.null(fit_fold)) { errs <- c(errs, rep(NA, h_cv)); next }

    kfs_fold <- KFS(fit_fold$model, filtering = "signal")
    signal_fold <- as.numeric(kfs_fold$m[(org+1):(org+h_cv)])
    pred_fold <- pmax(expm1(signal_fold + offset_te), 0)
    errs <- c(errs, y_te_actual - pred_fold)
  }
  c(mae = mean(abs(errs), na.rm = TRUE), rmse = sqrt(mean(errs^2, na.rm = TRUE)))
}

grid_ucm <- expand.grid(trend_degree = c(1, 2))
results_ucm <- grid_ucm %>%
  mutate(res = purrr::pmap(list(trend_degree), cv_ucm_metrics)) %>%
  tidyr::unnest_wider(res) %>%
  arrange(mae)

print(results_ucm)

## # A tibble: 2 x 3
##   trend_degree mae rmse
##           <dbl> <dbl> <dbl>
```

```

## 1          1  49.4  83.0
## 2          2  49.8  83.5

best_ucm <- results_ucm[1, ]
cat("Best configuration of UCM from CV: trend_degree =", best_ucm$trend_degree, "\n")

## Best configuration of UCM from CV: trend_degree = 1

model_ucm <- SSMModel(
  y_train ~
    SSMtrend(degree = best_ucm$trend_degree, Q = list(matrix(NA), matrix(NA))) +
    SSMseasonal(period = 24, sea.type = "trigonometric"),
  data = ucm_train_df,
  H = matrix(NA)
)

init_values <- rep(log(var(y_train, na.rm = TRUE) / 10), length(model_ucm$Q) + 1)
fit_ucm <- fitSSM(model_ucm, inits = init_values, method = "BFGS",
  control = list(maxit = 200, trace = 1, REPORT = 5))

## initial value 39236.347350
## iter 5 value 34795.046153
## iter 10 value 34638.208112
## final value 34638.115755
## converged

cat("Optimizer convergence:", fit_ucm$optim.out$convergence, "(0 = ok)\n")

## Optimizer convergence: 0 (0 = ok)

model_ucm_fitted <- fit_ucm$model

kfs_ucm <- KFS(model_ucm_fitted, filtering = "state", smoothing = c("state", "signal"))

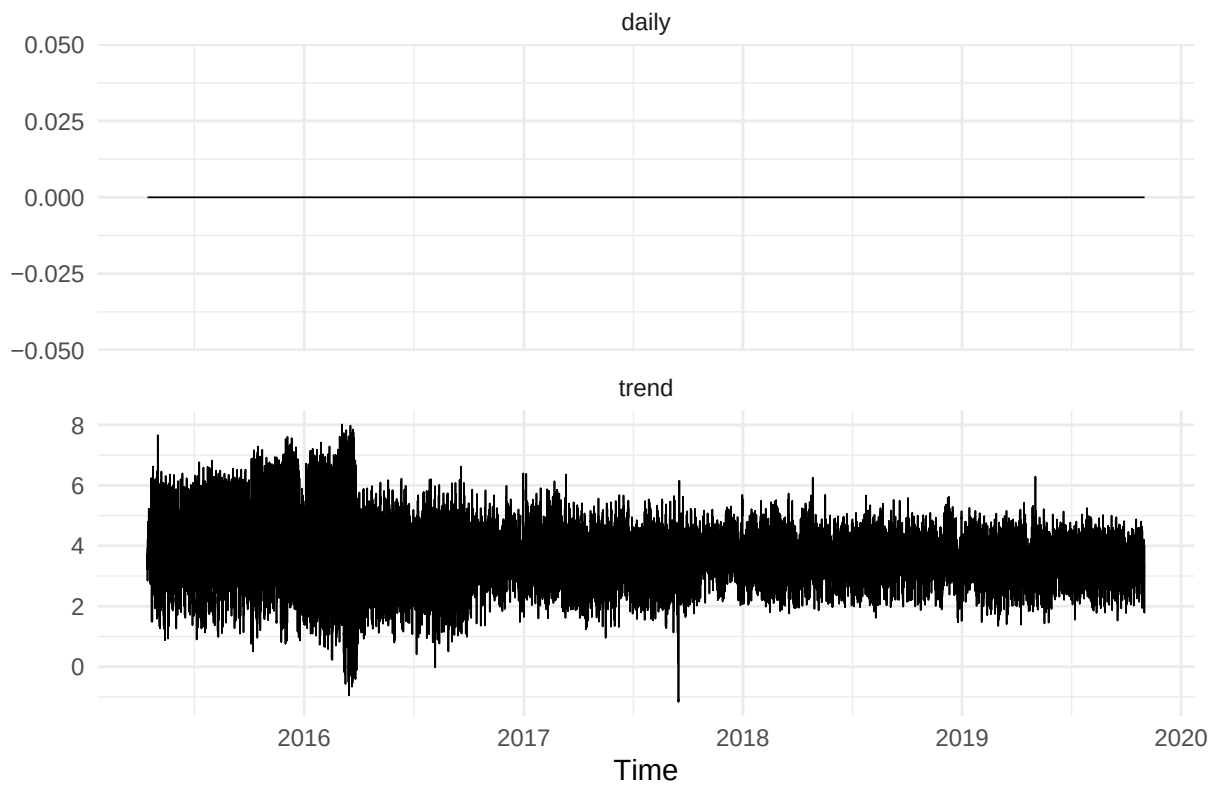
fitted_ucm <- as.numeric(kfs_ucm$muhat)

trend_est <- kfs_ucm$alphahat[, "level"]
daily_est <- rowSums(kfs_ucm$alphahat[, grep("sea_.*24|cycle24", colnames(kfs_ucm$alphahat))])

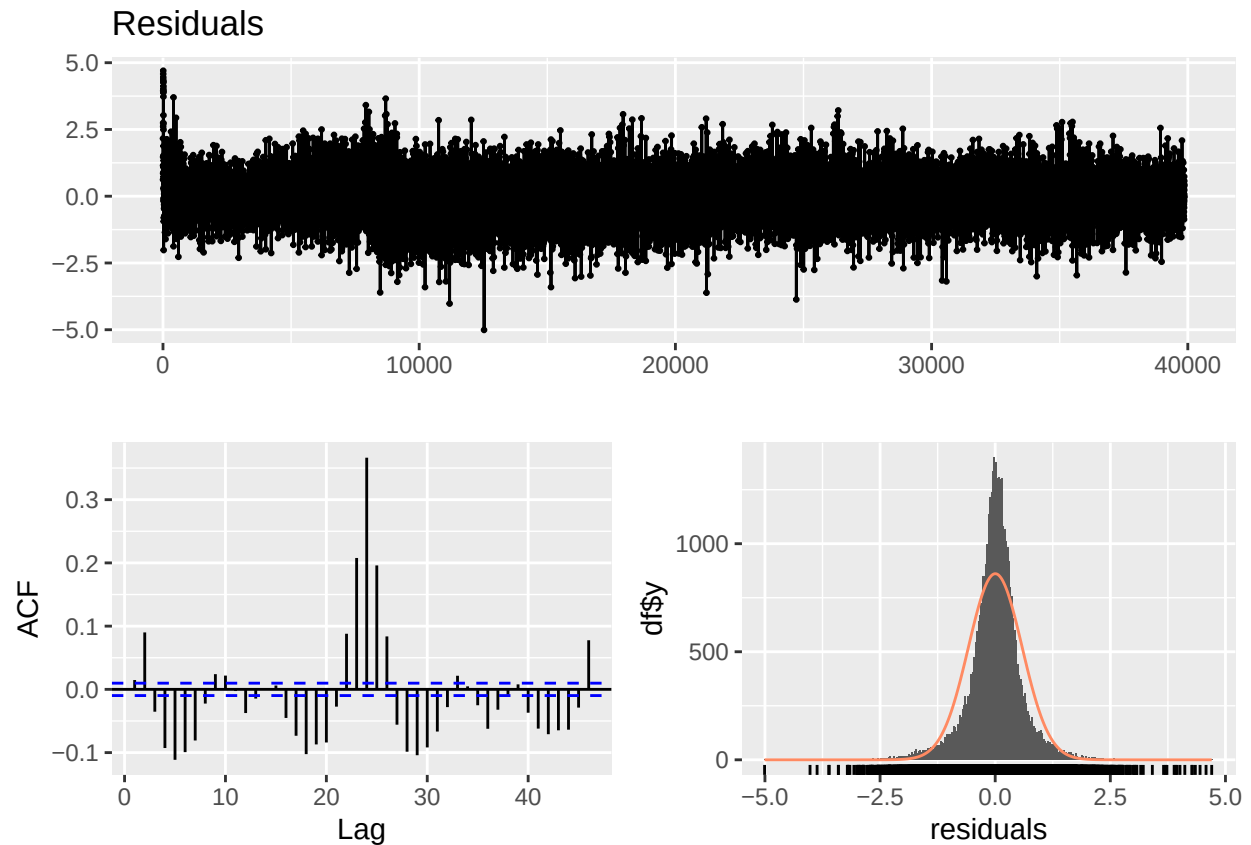
data.frame(
  time = ucm_train_df$time,
  trend = trend_est,
  daily = daily_est
) %>%
  pivot_longer(cols = c(trend, daily), names_to = "component", values_to = "value") %>%
  ggplot(aes(x = time, y = value)) +
  geom_line(linewidth = 0.3) +
  facet_wrap(~component, scales = "free_y", ncol = 1) +
  labs(title = "UCM: estimated components (trend, daily)", x = "Time", y = "") +
  theme_minimal()

```

UCM: estimated components (trend, daily)

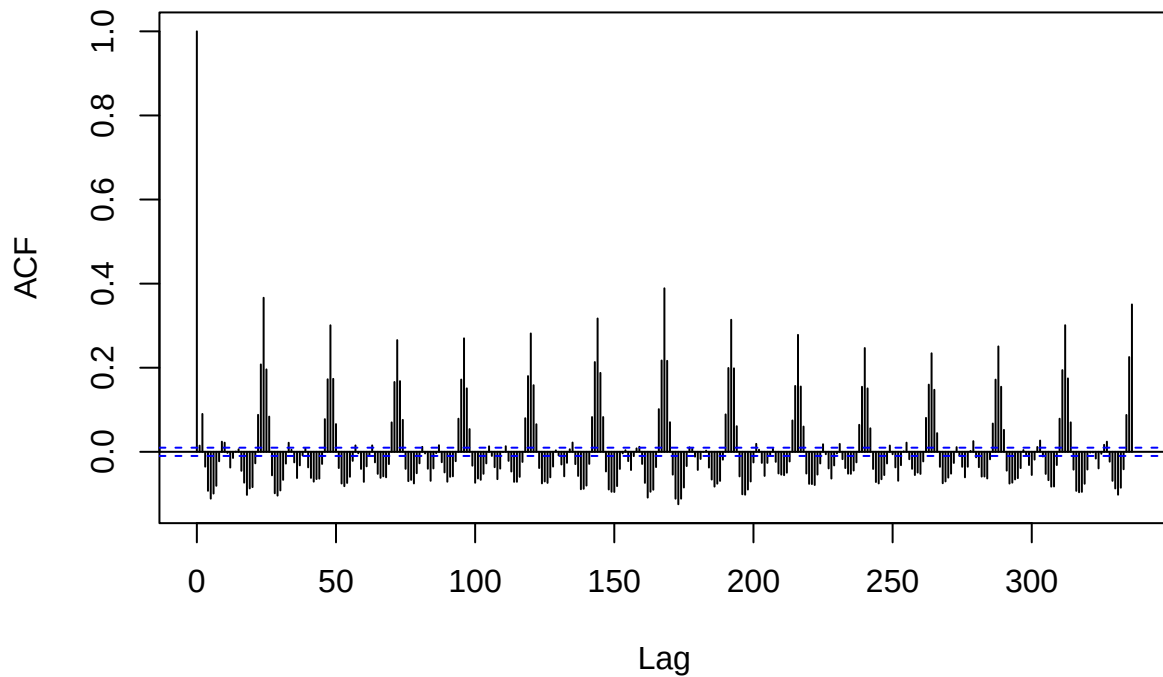


```
resid_ucm <- residuals(kfs_ucm, type = "recursive")  
checkresiduals(resid_ucm)
```



```
##
##  Ljung-Box test
##
## data:  Residuals
## Q* = 1934.7, df = 10, p-value < 2.2e-16
##
## Model df: 0.   Total lags used: 10
acf_ucm <- acf(resid_ucm, lag.max = 24*14, na.action = na.pass, main = "ACF residuals UCM (trend+daily,
```

ACF residuals UCM (trend+daily, weekly offset)



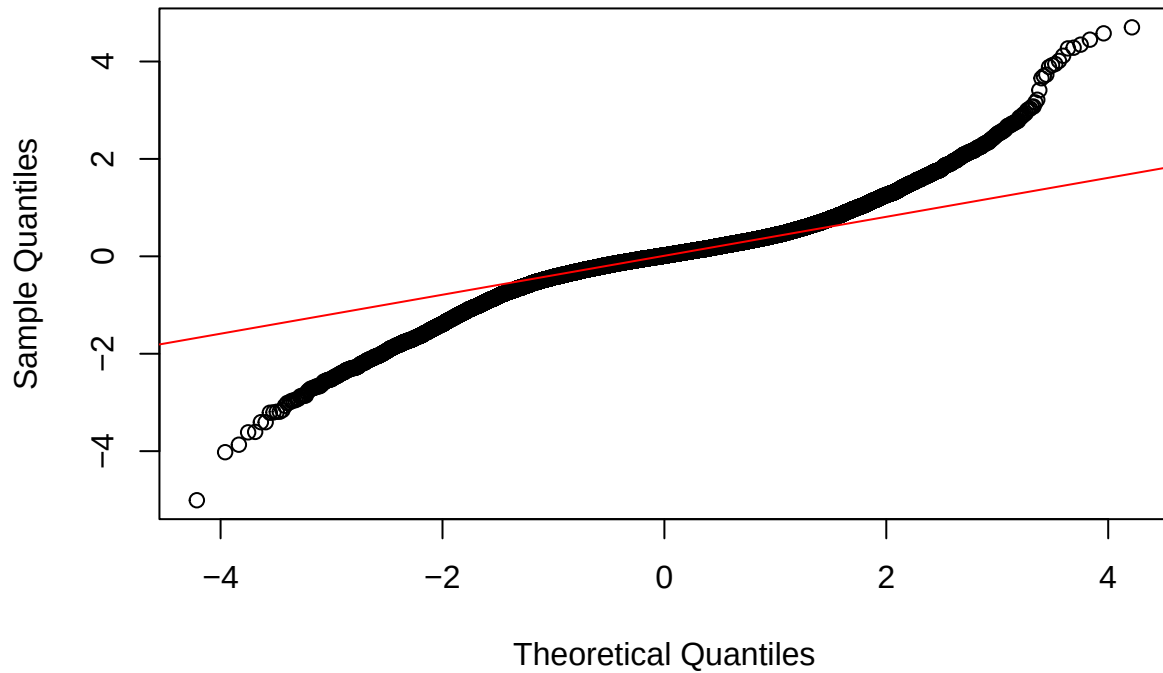
```
acf_ucm
```

```
##
## Autocorrelations of series 'resid_ucm', by lag
##
##      0      1      2      3      4      5      6      7      8      9     10
## 1.000  0.015  0.090 -0.035 -0.093 -0.111 -0.099 -0.081 -0.022  0.024  0.022
## 11     12     13     14     15     16     17     18     19     20     21
## -0.003 -0.037 -0.015  0.001  0.006 -0.045 -0.073 -0.102 -0.087 -0.084 -0.027
## 22     23     24     25     26     27     28     29     30     31     32
##  0.088  0.208  0.366  0.196  0.084 -0.056 -0.099 -0.104 -0.092 -0.067 -0.028
## 33     34     35     36     37     38     39     40     41     42     43
##  0.022  0.005 -0.025 -0.062 -0.032 -0.009  0.008 -0.037 -0.062 -0.071 -0.065
## 44     45     46     47     48     49     50     51     52     53     54
## -0.064 -0.029  0.078  0.172  0.301  0.174  0.066 -0.039 -0.075 -0.082 -0.074
## 55     56     57     58     59     60     61     62     63     64     65
## -0.059 -0.022  0.015 -0.003 -0.041 -0.071 -0.035 -0.008  0.015 -0.025 -0.052
## 66     67     68     69     70     71     72     73     74     75     76
## -0.062 -0.058 -0.061 -0.029  0.070  0.166  0.266  0.168  0.076 -0.041 -0.070
## 77     78     79     80     81     82     83     84     85     86     87
## -0.067 -0.075 -0.051 -0.027  0.012 -0.004 -0.040 -0.069 -0.040 -0.003  0.016
## 88     89     90     91     92     93     94     95     96     97     98
## -0.025 -0.049 -0.071 -0.060 -0.058 -0.022  0.079  0.172  0.270  0.151  0.054
## 99     100    101    102    103    104    105    106    107    108    109
## -0.033 -0.074 -0.063 -0.067 -0.052 -0.027  0.013 -0.010 -0.037 -0.065 -0.040
## 110    111    112    113    114    115    116    117    118    119    120
```

```
## 0.000 0.013 -0.021 -0.047 -0.071 -0.071 -0.060 -0.024 0.080 0.180 0.282
## 121 122 123 124 125 126 127 128 129 130 131
## 0.158 0.066 -0.039 -0.075 -0.071 -0.074 -0.061 -0.035 0.003 -0.008 -0.030
## 132 133 134 135 136 137 138 139 140 141 142
## -0.058 -0.026 0.006 0.022 -0.029 -0.057 -0.089 -0.088 -0.080 -0.029 0.083
## 143 144 145 146 147 148 149 150 151 152 153
## 0.213 0.317 0.188 0.083 -0.047 -0.090 -0.095 -0.096 -0.081 -0.041 -0.004
## 154 155 156 157 158 159 160 161 162 163 164
## 0.004 -0.022 -0.043 -0.010 0.008 0.012 -0.029 -0.074 -0.109 -0.095 -0.090
## 165 166 167 168 169 170 171 172 173 174 175
## -0.036 0.101 0.217 0.389 0.216 0.070 -0.054 -0.111 -0.125 -0.111 -0.085
## 176 177 178 179 180 181 182 183 184 185 186
## -0.034 0.011 0.007 -0.011 -0.043 -0.017 0.000 0.003 -0.037 -0.066 -0.083
## 187 188 189 190 191 192 193 194 195 196 197
## -0.076 -0.069 -0.019 0.089 0.199 0.314 0.199 0.061 -0.058 -0.101 -0.102
## 198 199 200 201 202 203 204 205 206 207 208
## -0.090 -0.071 -0.025 0.019 0.005 -0.026 -0.057 -0.026 -0.007 0.007 -0.024
## 209 210 211 212 213 214 215 216 217 218 219
## -0.051 -0.054 -0.056 -0.050 -0.023 0.075 0.157 0.278 0.155 0.060 -0.052
## 220 221 222 223 224 225 226 227 228 229 230
## -0.076 -0.077 -0.079 -0.054 -0.028 0.018 -0.006 -0.039 -0.064 -0.032 -0.004
## 231 232 233 234 235 236 237 238 239 240 241
## 0.019 -0.017 -0.039 -0.052 -0.052 -0.045 -0.025 0.064 0.155 0.247 0.151
## 242 243 244 245 246 247 248 249 250 251 252
## 0.056 -0.041 -0.072 -0.075 -0.065 -0.055 -0.028 0.015 -0.006 -0.037 -0.069
## 253 254 255 256 257 258 259 260 261 262 263
## -0.032 0.001 0.022 -0.018 -0.040 -0.055 -0.049 -0.053 -0.023 0.081 0.160
## 264 265 266 267 268 269 270 271 272 273 274
## 0.234 0.147 0.044 -0.036 -0.075 -0.070 -0.061 -0.052 -0.026 0.011 -0.009
## 275 276 277 278 279 280 281 282 283 284 285
## -0.036 -0.061 -0.035 0.002 0.025 -0.013 -0.037 -0.059 -0.059 -0.063 -0.018
## 286 287 288 289 290 291 292 293 294 295 296
## 0.067 0.172 0.251 0.155 0.052 -0.045 -0.075 -0.073 -0.066 -0.063 -0.039
## 297 298 299 300 301 302 303 304 305 306 307
## 0.004 -0.007 -0.031 -0.055 -0.019 0.011 0.027 -0.011 -0.053 -0.067 -0.083
## 308 309 310 311 312 313 314 315 316 317 318
## -0.082 -0.031 0.079 0.194 0.301 0.175 0.070 -0.042 -0.093 -0.096 -0.095
## 319 320 321 322 323 324 325 326 327 328 329
## -0.076 -0.042 -0.002 -0.001 -0.016 -0.039 -0.005 0.017 0.024 -0.023 -0.069
## 330 331 332 333 334 335 336
## -0.087 -0.102 -0.085 -0.042 0.088 0.225 0.350
```

```
qqnorm(resid_uem)
qqline(resid_uem, col = "red")
```

Normal Q-Q Plot



Forecast

On the validation set, the model achieves $MAE = 25.8$ and $RMSE = 38$, though still not matching ARIMA or XGBoost. The forecast reproduces the weekday/weekend contrast visible in the actual values, consistent with the weekly offset applied. The largest discrepancies remain concentrated around unusually high peaks, consistent with the outliers retained without modification in the EDA.

```
h_val <- nrow(ucm_val_df)
y_extended_val <- c(y_train, rep(NA, h_val))

model_ucm_val <- SSMModel(
  y_extended_val ~
    SSMtrend(degree = best_ucm$trend_degree, Q = list(matrix(NA), matrix(NA))) +
    SSMseasonal(period = 24, sea.type = "trigonometric"),
  H = matrix(NA)
)

model_ucm_val$Q <- model_ucm_fitted$Q
model_ucm_val$H <- model_ucm_fitted$H

kfs_val <- KFS(model_ucm_val, filtering = "signal")
signal_val <- as.numeric(kfs_val$m[(length(y_train)+1):(length(y_train)+h_val)])
pred_val_ucm <- pmax(expm1(signal_val + ucm_val_df$month_offset + ucm_val_df$wd_offset), 0)

mae_ucm <- mean(abs(ucm_val_df$value - pred_val_ucm))
rmse_ucm <- sqrt(mean((ucm_val_df$value - pred_val_ucm)^2))
```

```
cat("UCM on validation:\n")

## UCM on validation:
cat("MAE:", round(mae_ucm, 2), " RMSE:", round(rmse_ucm, 2), "\n")

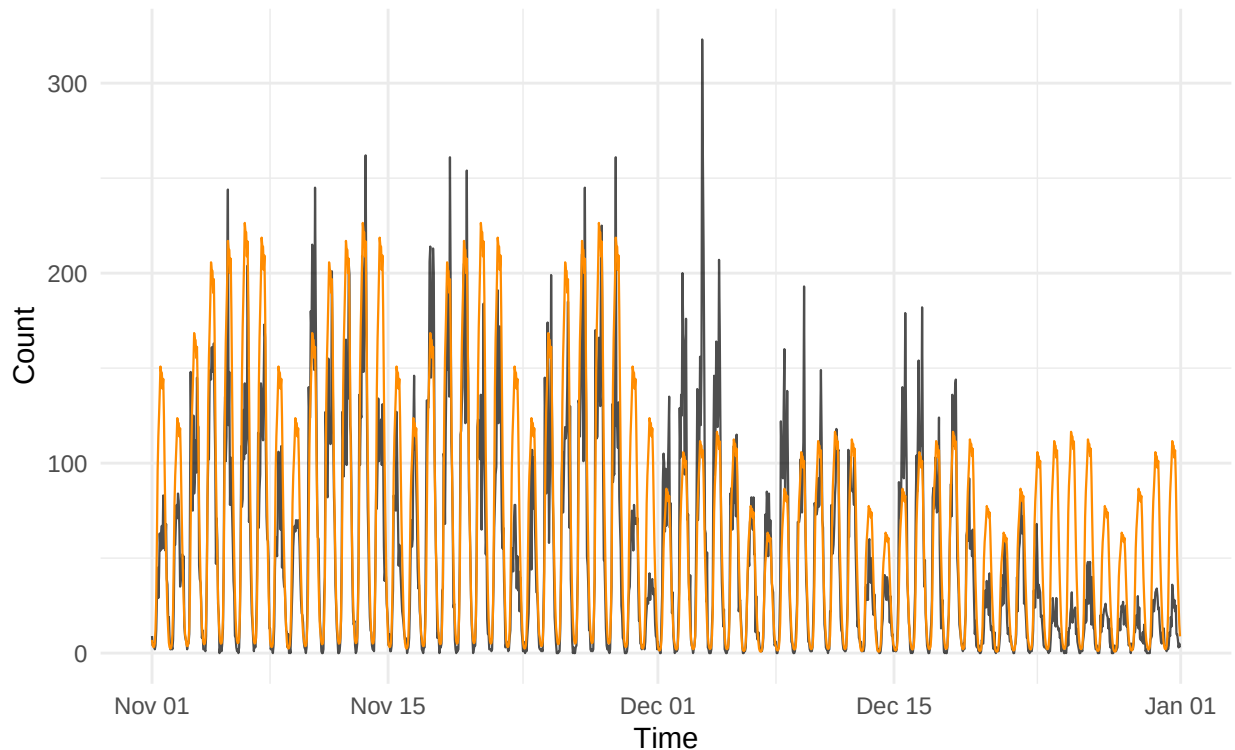
## MAE: 25.82 RMSE: 38.39

p_ucm_val <- ucm_val_df %>%
  select(time, value) %>%
  mutate(predicted = pred_val_ucm) %>%
  ggplot(aes(x = time)) +
  geom_line(aes(y = value), color = "black", linewidth = 0.4, alpha = 0.7) +
  geom_line(aes(y = predicted), color = "darkorange", linewidth = 0.4) +
  labs(title = "UCM forecast vs actual (validation set: Nov-Dec 2019)",
       subtitle = "Black = actual, Orange = UCM forecast",
       x = "Time", y = "Count") +
  theme_minimal()

p_ucm_val
```

UCM forecast vs actual (validation set: Nov-Dec 2019)

Black = actual, Orange = UCM forecast



```
ggsave("slides_ucm_validation.png", p_ucm_val, width = 10, height = 5, dpi = 300)
```

The plot shows the orange forecast dipping on weekends in line with the black observed series, reflecting the weekly offset applied to the signal. The remaining gap between forecast and actual is concentrated in the amplitude of individual peaks rather than in the timing or shape of the cycle, again consistent with the outlier days identified in the EDA.

```

ucm_test_df <- ucm_test_df %>%
  mutate(
    wd_num = wday(time, week_start = 1),
    is_fri = as.integer(wd_num == 5),
    is_sat = as.integer(wd_num == 6),
    is_sun = as.integer(wd_num == 7)
  )

ucm_full_df <- bind_rows(ucm_train_df, ucm_val_df)
y_full <- log1p(ucm_full_df$value) - ucm_full_df$month_offset - ucm_full_df$wd_offset

h_test <- nrow(ucm_test_df)
y_extended_test <- c(y_full, rep(NA, h_test))

model_ucm_test <- SSMModel(
  y_extended_test ~
    SSMtrend(degree = best_ucm$trend_degree, Q = list(matrix(NA), matrix(NA))) +
    SSMseasonal(period = 24, sea.type = "trigonometric"),
  H = matrix(NA)
)

model_ucm_test$Q <- model_ucm_fitted$Q
model_ucm_test$H <- model_ucm_fitted$H

kfs_test <- KFS(model_ucm_test, filtering = "signal")
signal_test <- as.numeric(kfs_test$m[(length(y_full)+1):(length(y_full)+h_test)])
pred_test_ucm <- pmax(expm1(signal_test + ucm_test_df$month_offset + ucm_test_df$wd_offset), 0)

ucm_forecast_final <- ucm_test_df %>%
  select(time) %>%
  mutate(value_forecast_ucm = pred_test_ucm)

ucm_forecast_final %>%
  mutate(month = month(time)) %>%
  group_by(month) %>%
  summarise(mean_forecast = mean(value_forecast_ucm))

## # A tibble: 2 x 2
##   month mean_forecast
##   <dbl>         <dbl>
## 1     1             23.8
## 2     2             38.3

```

Diagnostics

As with ARIMA, outlier-day residuals are somewhat elevated relative to the series average (0.448 vs 0.401, a 12% increase) but do not appear among the most extreme 0.1% of residuals. The smaller relative increase compared to ARIMA may reflect UCM's stochastic daily seasonal component partially adapting to elevated local levels, though a direct magnitude comparison across models is not meaningful given the different residual scales involved.

```

resid_ucm_df <- tibble(
  time = ucm_train_df$time[seq_along(resid_ucm)],
  resid = as.numeric(resid_ucm)
) %>% mutate(day = as_date(time), is_outlier_day = day %in% outlier_days_eda)

```

```
outlier_resid_summary(resid_ucm_df)
```

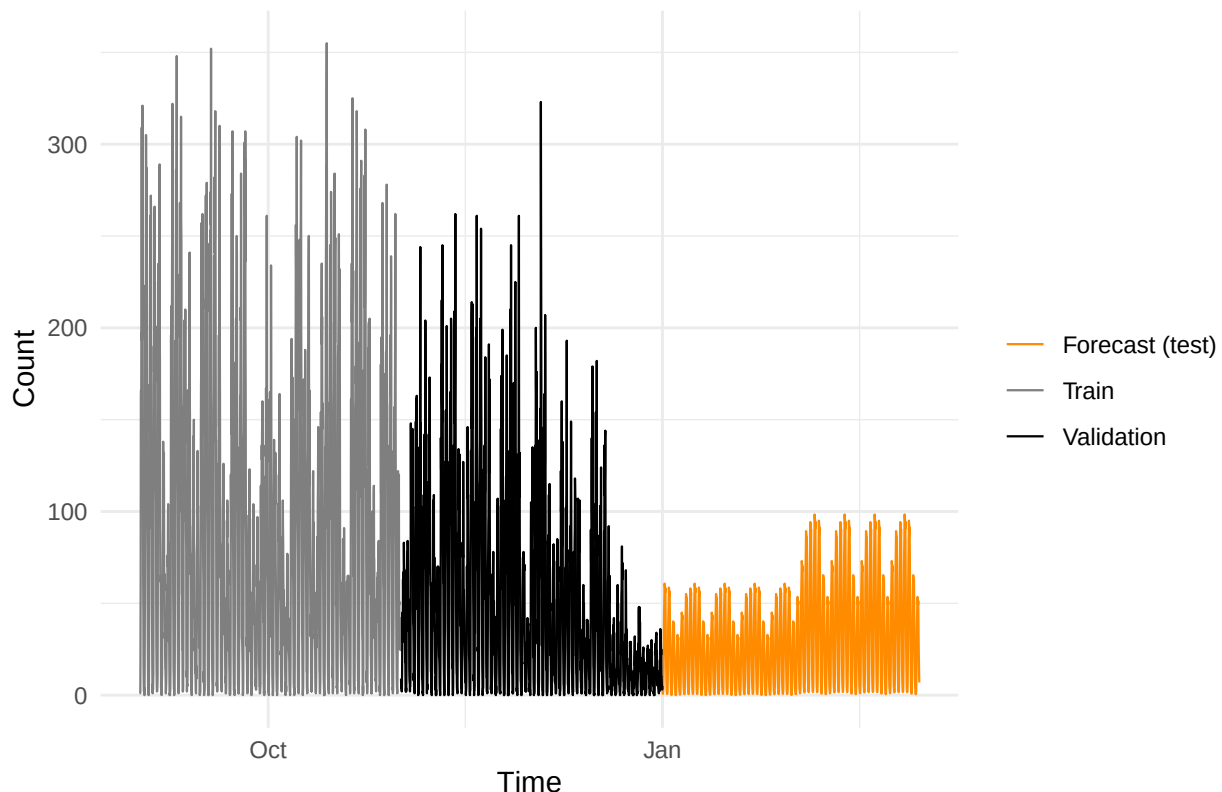
```
## # A tibble: 1 x 3
##   mean_abs_all mean_abs_outlier_days pct_outlier_days_in_top
##         <dbl>         <dbl>         <dbl>
## 1         0.401         0.448             0
```

The final forecast (orange) now visibly reproduces the weekly modulation absent from the earlier UCM specification, but the overall level for January-February remains below the historical range, similarly to the original specification. This indicates that the weekly and annual corrections address two independent problems: the former corrects the within-week shape, while the latter — shared with the ARIMA model — is constrained by the same ~4.5 observed annual cycles and is not resolved by the weekly offset.

```
p_ucm_final <- bind_rows(
  ucm_train_df %>% select(time, value) %>% mutate(type = "Train"),
  ucm_val_df %>% select(time, value) %>% mutate(type = "Validation"),
  ucm_forecast_final %>% rename(value = value_forecast_ucm) %>% mutate(type = "Forecast (test)")
) %>%
  filter(time >= as.POSIXct("2019-09-01", tz = "UTC")) %>%
  ggplot(aes(x = time, y = value, color = type)) +
  geom_line(linewidth = 0.4) +
  scale_color_manual(values = c("Train" = "grey50", "Validation" = "black", "Forecast (test)" = "darkorange")) +
  labs(title = "UCM: final forecast for missing period (Jan-Feb 2020)",
       x = "Time", y = "Count", color = "") +
  theme_minimal()

p_ucm_final
```

UCM: final forecast for missing period (Jan–Feb 2020)



```
ggsave("slides_ucm_final_forecast.png", p_ucm_final, width = 10, height = 5, dpi = 300)
```

Machine Learning - XGBoost

Theory

Cyclical sine/cosine encodings are used for hour and month, so that the model can naturally represent their periodic nature (hour 23 and hour 0 are close together) without an explicit seasonal structure imposed a priori. Weekday is instead represented through dummy variables for Friday, Saturday and Sunday (Monday-Thursday as reference), rather than a cyclical encoding, because the EDA showed the weekly pattern has sharp discontinuities (Thu→Fri, Sat→Sun) rather than a smooth sinusoidal shape; a cyclical encoding would impose a gradual transition assumption not supported by the data. A linear `year_frac` term is included to let the model capture the mild downward trend identified in the EDA. Hyperparameters (`max_depth`, `eta`) and the number of boosting rounds are selected via time-series cross-validation (rolling-origin, 4 folds) rather than a random k-fold, to avoid the temporal leakage that lagged features would otherwise introduce under a random split.

Feature engineering

```
add_features <- function(data) {
  data %>%
    mutate(
      hour_sin = sin(2*pi*hour/24), hour_cos = cos(2*pi*hour/24),
      wday_num = wday(time, week_start = 1),
      day_group = case_when(
        wday_num %in% 1:4 ~ "mon_thu",
        wday_num == 5    ~ "fri",
        wday_num == 6    ~ "sat",
        wday_num == 7    ~ "sun"
      ),
      is_fri = as.integer(day_group == "fri"),
      is_sat = as.integer(day_group == "sat"),
      is_sun = as.integer(day_group == "sun"),
      month_sin = sin(2*pi*month/12), month_cos = cos(2*pi*month/12),
      year_frac = as.numeric(difftime(time, min(time), units = "days")) / 365.25
    )
}

xgb_train_df <- add_features(train_df)
xgb_val_df   <- add_features(val_df)
xgb_test_df  <- add_features(test_df)

xgb_train_df <- xgb_train_df %>%
  arrange(time) %>%
  mutate(
    lag_1   = lag(value, 1),
    lag_24  = lag(value, 24),
    lag_168 = lag(value, 168),
    roll_mean_24 = zoo::rollmean(lag(value, 1), k = 24, fill = NA, align = "right"),
    roll_mean_168 = zoo::rollmean(lag(value, 1), k = 168, fill = NA, align = "right")
  ) %>%
  filter(!is.na(lag_168))
```

```

feature_cols <- c("hour_sin", "hour_cos", "is_fri", "is_sat", "is_sun",
                  "month_sin", "month_cos", "year_frac",
                  "lag_1", "lag_24", "lag_168", "roll_mean_24", "roll_mean_168")

dtrain <- xgb.DMatrix(
  data = as.matrix(xgb_train_df[, feature_cols]),
  label = xgb_train_df$value
)

```

Forecast

```

recursive_forecast <- function(model, history_df, future_times, feature_cols) {
  value_lookup <- setNames(history_df$value, as.character(history_df$time))
  preds <- numeric(length(future_times))

  for (i in seq_along(future_times)) {
    t <- future_times[i]

    lag_1_val <- value_lookup[as.character(t - hours(1))]
    lag_24_val <- value_lookup[as.character(t - hours(24))]
    lag_168_val <- value_lookup[as.character(t - hours(168))]

    last_24_times <- as.character(t - hours(1:24))
    last_168_times <- as.character(t - hours(1:168))
    roll_24 <- mean(value_lookup[last_24_times], na.rm = TRUE)
    roll_168 <- mean(value_lookup[last_168_times], na.rm = TRUE)

    hour_t <- hour(t)
    wday_t <- wday(t, week_start = 1)
    month_t <- month(t)

    feat_row <- data.frame(
      hour_sin = sin(2*pi*hour_t/24), hour_cos = cos(2*pi*hour_t/24),
      is_fri = as.integer(wday_t == 5),
      is_sat = as.integer(wday_t == 6),
      is_sun = as.integer(wday_t == 7),
      month_sin = sin(2*pi*month_t/12), month_cos = cos(2*pi*month_t/12),
      year_frac = as.numeric(difftime(t, min(history_df$time), units = "days")) / 365.25,
      lag_1 = lag_1_val, lag_24 = lag_24_val, lag_168 = lag_168_val,
      roll_mean_24 = roll_24, roll_mean_168 = roll_168
    )

    pred <- max(predict(model, as.matrix(feat_row[, feature_cols])), 0)
    preds[i] <- pred
    value_lookup[as.character(t)] <- pred
  }

  preds
}

xgb_params <- list(
  objective = "reg:squarederror",
  eta = 0.05,

```

```

    max_depth = 6,
    subsample = 0.8,
    colsample_bytree = 0.8
  )

  set.seed(42)
  n_folds <- 4
  h_cv <- 24*61

  n_total <- nrow(xgb_train_df)
  fold_starts <- seq(from = n_total - h_cv*n_folds,
                     to   = n_total - h_cv,
                     by   = h_cv)

  cv_results_xgb <- tibble()

  for (start_idx in fold_starts) {
    train_idx <- 1:start_idx
    test_idx  <- (start_idx + 1):(start_idx + h_cv)

    dtrain_fold <- xgb.DMatrix(
      data = as.matrix(xgb_train_df[train_idx, feature_cols]),
      label = xgb_train_df$value[train_idx]
    )
    dtest_fold <- xgb.DMatrix(
      data = as.matrix(xgb_train_df[test_idx, feature_cols]),
      label = xgb_train_df$value[test_idx]
    )

    fit_fold <- xgb.train(
      params = xgb_params, data = dtrain_fold, nrounds = 3000,
      watchlist = list(train = dtrain_fold, test = dtest_fold),
      early_stopping_rounds = 50, verbose = 0
    )

    pred_fold <- predict(fit_fold, dtest_fold)
    actual_fold <- xgb_train_df$value[test_idx]

    bi <- fit_fold$best_iteration
    if (is.null(bi) || length(bi) == 0 || is.na(bi)) {
      bi <- suppressWarnings(as.integer(xgb.attr(fit_fold, "best_iteration")))
    }
    if (is.null(bi) || length(bi) == 0 || is.na(bi)) {
      bi <- fit_fold$niter
    }

    cv_results_xgb <- bind_rows(cv_results_xgb, tibble(
      fold = which(fold_starts == start_idx),
      best_nrounds = bi,
      MAE = mean(abs(actual_fold - pred_fold)),
      RMSE = sqrt(mean((actual_fold - pred_fold)^2))
    ))
  }
}

```

```
print(cv_results_xgb)
```

```
## # A tibble: 4 x 4
##   fold best_nrounds   MAE  RMSE
##   <int>      <int> <dbl> <dbl>
## 1     1          143  19.8  31.5
## 2     2          107  15.1  22.7
## 3     3          280  15.4  25.0
## 4     4          175  15.6  23.8
```

```
cat("XGBoost CV - mean MAE:", mean(cv_results_xgb$MAE),
    " mean RMSE:", mean(cv_results_xgb$RMSE), "\n")
```

```
## XGBoost CV - mean MAE: 16.46566 mean RMSE: 25.7631
```

```
best_nrounds <- round(mean(cv_results_xgb$best_nrounds, na.rm = TRUE))
cat("Best nrounds:", best_nrounds, "\n")
```

```
## Best nrounds: 176
```

```
fit_xgb <- xgb.train(params = xgb_params, data = dtrain, nrounds = best_nrounds)
```

This first cross-validation pass, using fixed hyperparameters (max_depth=6, eta=0.05), estimates a suitable number of boosting rounds (176) and a mean CV MAE of 16.47. It is superseded by the joint hyperparameter grid search below, which additionally optimises tree depth and learning rate; the final model uses the configuration selected there.

```
grid_xgb <- expand.grid(max_depth = c(4, 6, 8), eta = c(0.03, 0.05, 0.1))
n_folds <- 4
h_cv <- 24*61
n_total <- nrow(xgb_train_df)
fold_starts <- seq(from = n_total - h_cv*n_folds, to = n_total - h_cv, by = h_cv)

cv_xgb_metrics <- function(max_depth, eta) {
  params <- list(objective = "reg:squarederror", eta = eta, max_depth = max_depth,
                 subsample = 0.8, colsample_bytree = 0.8)
  maes <- c(); rounds <- c()
  for (start_idx in fold_starts) {
    train_idx <- 1:start_idx
    test_idx <- (start_idx + 1):(start_idx + h_cv)
    dtr <- xgb.DMatrix(as.matrix(xgb_train_df[train_idx, feature_cols]),
                      label = xgb_train_df$value[train_idx])
    dte <- xgb.DMatrix(as.matrix(xgb_train_df[test_idx, feature_cols]),
                      label = xgb_train_df$value[test_idx])
    fit_fold <- xgb.train(params, dtr, nrounds = 3000,
                        watchlist = list(test = dte),
                        early_stopping_rounds = 50, verbose = 0)
    pred <- predict(fit_fold, dte)
    maes <- c(maes, mean(abs(xgb_train_df$value[test_idx] - pred)))

    bi <- fit_fold$best_iteration
    if (is.null(bi) || length(bi) == 0 || is.na(bi)) {
      bi <- suppressWarnings(as.integer(xgb.attr(fit_fold, "best_iteration")))
    }
    if (is.null(bi) || length(bi) == 0 || is.na(bi)) {
      bi <- fit_fold$niter
    }
  }
}
```

```

    }
    rounds <- c(rounds, bi)
  }
  c(mae = mean(maes), nrounds = round(mean(rounds, na.rm = TRUE)))
}

results_xgb <- grid_xgb %>%
  mutate(res = purrr::pmap(list(max_depth, eta), cv_xgb_metrics)) %>%
  tidyr::unnest_wider(res) %>%
  arrange(mae)

print(results_xgb)

## # A tibble: 9 x 4
##   max_depth eta   mae nrounds
##       <dbl> <dbl> <dbl>   <dbl>
## 1         8 0.03  16.4     268
## 2         8 0.05  16.4     160
## 3         6 0.1   16.7      88
## 4         6 0.03  16.7     268
## 5         6 0.05  16.7     250
## 6         8 0.1   16.8      68
## 7         4 0.03  17.0     578
## 8         4 0.05  17.0     339
## 9         4 0.1   17.0     191

best_xgb <- results_xgb[1, ]

xgb_params <- list(objective = "reg:squarederror", eta = best_xgb$eta,
                   max_depth = best_xgb$max_depth, subsample = 0.8, colsample_bytree = 0.8)
fit_xgb <- xgb.train(params = xgb_params, data = dtrain, nrounds = best_xgb$nrounds)

pred_val_xgb <- recursive_forecast(
  model = fit_xgb,
  history_df = xgb_train_df %>% select(time, value),
  future_times = xgb_val_df$time,
  feature_cols = feature_cols
)

mae_xgb <- mean(abs(xgb_val_df$value - pred_val_xgb))
rmse_xgb <- sqrt(mean((xgb_val_df$value - pred_val_xgb)^2))

cat("XGBoost on validation:\n")

## XGBoost on validation:
cat("MAE:", round(mae_xgb, 2), " RMSE:", round(rmse_xgb, 2), "\n")

## MAE: 19.33 RMSE: 32.67

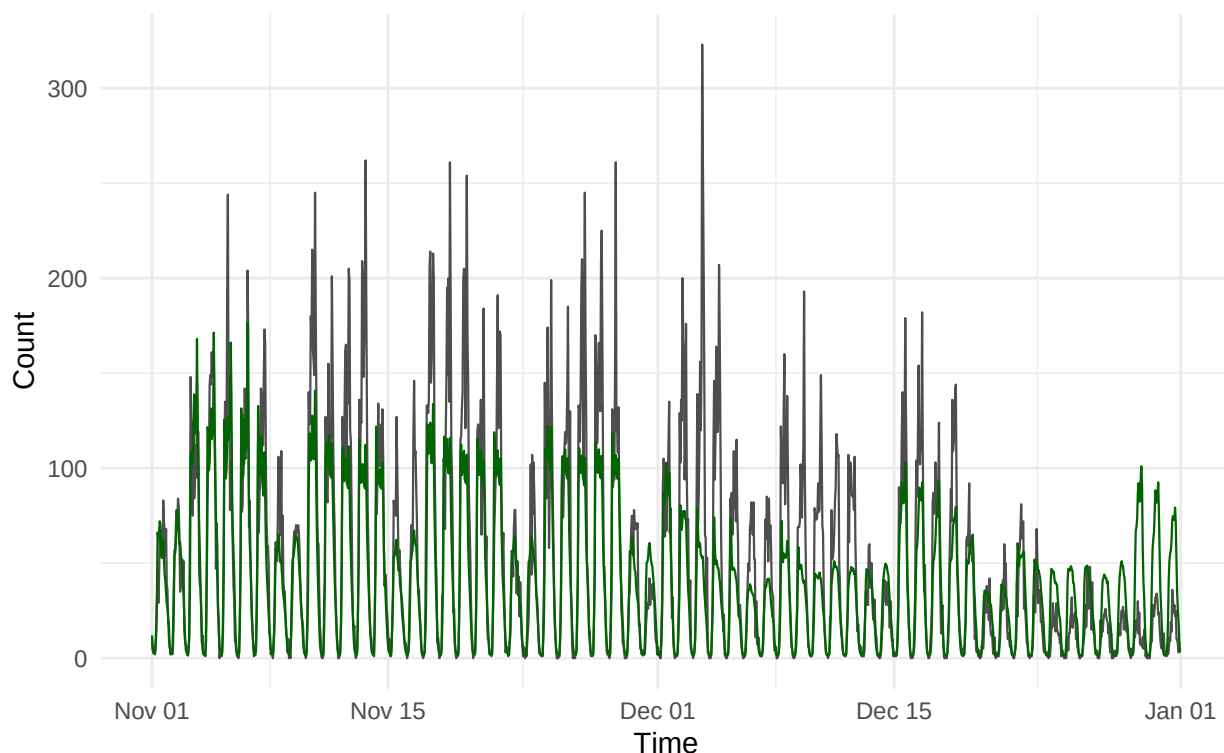
```

On the validation set, the recursive forecast achieves $MAE = 19.33$ and $RMSE = 32.67$ — the best result among the three models. Unlike the one-step-ahead errors reported during cross-validation, this metric reflects the true multi-step recursive forecasting setting that the model faces on the actual missing period, making it directly comparable to the ARIMA and UCM validation results.

```
p_xgb_val <- xgb_val_df %>%
  select(time, value) %>%
  mutate(predicted = pred_val_xgb) %>%
  ggplot(aes(x = time)) +
  geom_line(aes(y = value), color = "black", linewidth = 0.4, alpha = 0.7) +
  geom_line(aes(y = predicted), color = "darkgreen", linewidth = 0.4) +
  labs(title = "XGBoost forecast vs actual (validation set: Nov-Dec 2019)",
       subtitle = "Black = actual, Green = XGBoost forecast (recursive)",
       x = "Time", y = "Count") +
  theme_minimal()

p_xgb_val
```

XGBoost forecast vs actual (validation set: Nov-Dec 2019)
 Black = actual, Green = XGBoost forecast (recursive)



```
ggsave("slides_xgb_validation.png", p_xgb_val, width = 10, height = 5, dpi = 300)
```

The forecast closely tracks both the amplitude and the weekly modulation of the actual values, including the reduced counts over weekends, consistent with the high importance assigned to the lag-168 feature (see feature importance below).

At this resolution the recursive forecast remains stable and does not visibly drift away from the actual values over the course of the week, suggesting that error accumulation across the recursive steps is limited over this horizon.

```
full_history_xgb <- bind_rows(xgb_train_df, xgb_val_df) %>% select(time, value)

pred_test_xgb <- recursive_forecast(
  model = fit_xgb,
```

```

    history_df = full_history_xgb,
    future_times = xgb_test_df$time,
    feature_cols = feature_cols
  )

xgb_forecast_final <- xgb_test_df %>%
  select(time) %>%
  mutate(value_forecast_xgb = pred_test_xgb)

xgb_forecast_final %>%
  mutate(month = month(time)) %>%
  group_by(month) %>%
  summarise(mean_forecast = mean(value_forecast_xgb))

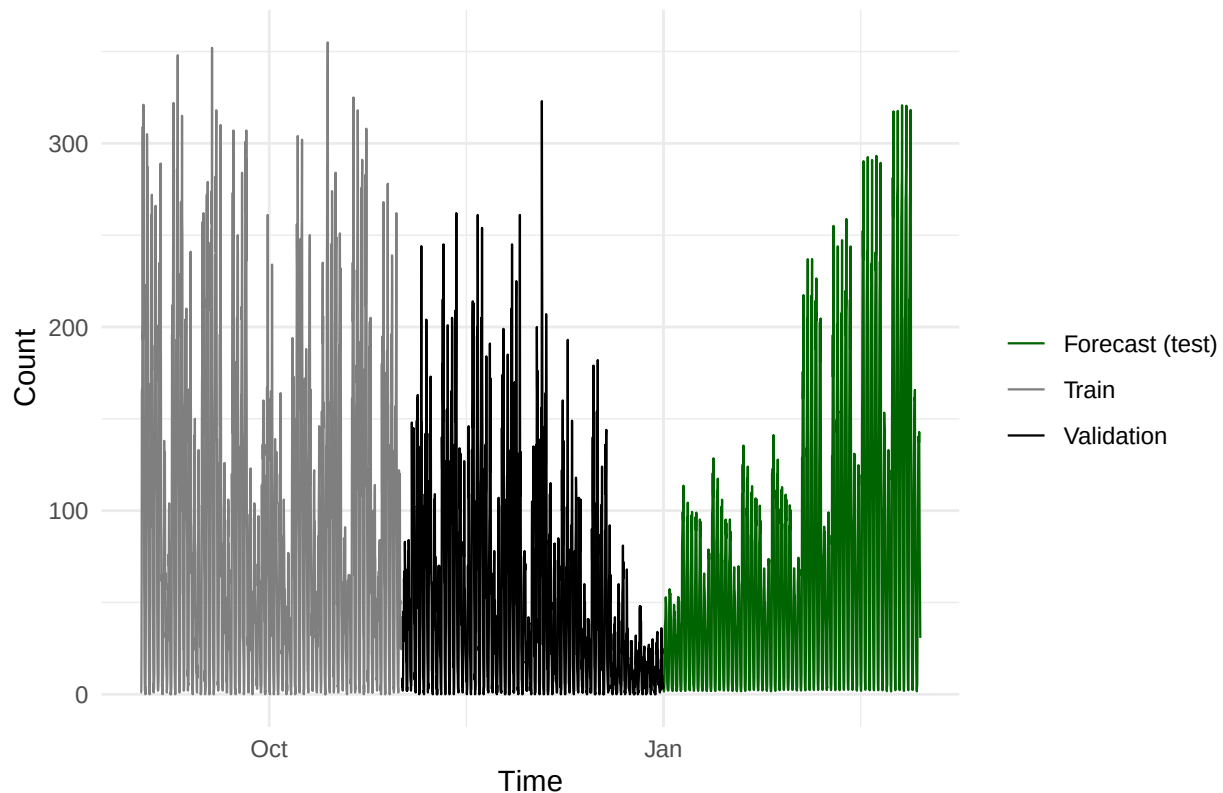
## # A tibble: 2 x 2
##   month mean_forecast
##   <dbl>         <dbl>
## 1     1             44.7
## 2     2             97.8

p_xgb_final <- bind_rows(
  xgb_train_df %>% select(time, value) %>% mutate(type = "Train"),
  xgb_val_df %>% select(time, value) %>% mutate(type = "Validation"),
  xgb_forecast_final %>% rename(value = value_forecast_xgb) %>% mutate(type = "Forecast (test)")
) %>%
  filter(time >= as.POSIXct("2019-09-01", tz = "UTC")) %>%
  ggplot(aes(x = time, y = value, color = type)) +
  geom_line(linewidth = 0.4) +
  scale_color_manual(values = c("Train" = "grey50", "Validation" = "black", "Forecast (test)" = "darkgreen")) +
  labs(title = "XGBoost: final forecast for missing period (Jan-Feb 2020)",
       x = "Time", y = "Count", color = "") +
  theme_minimal()

p_xgb_final

```

XGBoost: final forecast for missing period (Jan–Feb 2020)



```
ggsave("slides_xgb_final_forecast.png", p_xgb_final, width = 10, height = 5, dpi = 300)
```

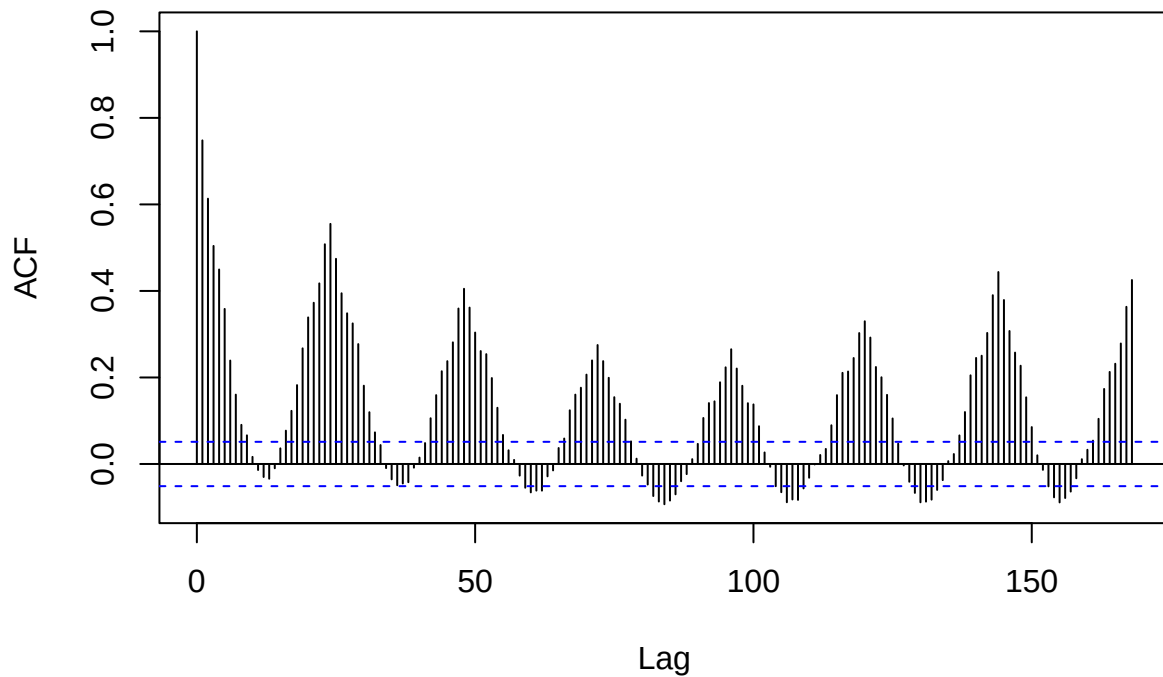
The forecast for the missing period shows mean values of 44.7 for January and 97.8 for February, close to the historical averages (54-67 and 80-117) and, for February in particular, closer to the historical range than either ARIMA (86.2) or UCM (45.0). This reinforces the case for preferring XGBoost's forecast for the missing period: it now achieves both the lowest validation error among the three models and the most plausible annual-cycle behaviour, since it relies on direct lag features rather than a parametric annual seasonal form that both ARIMA and UCM must estimate from only ~4-5 observed cycles.

Diagnostics

The ACF of XGBoost's validation-set forecast errors shows a large lag-1 autocorrelation (~0.75) together with recurring peaks at each 24-hour multiple, decaying slowly out to a week. This is consistent with the recursive forecasting setup: consecutive hourly predictions rely on lag-based features that partly encode each other's previous errors, so a miss at hour t propagates into $t+1$ via `lag_1`, and reappears at multiples of 24/168 as `lag_24/lag_168` feed earlier forecast errors back into the model. Unlike the ARIMA/UCM residual ACFs, computed in-sample on ~40,000 training observations, this is an out-of-sample diagnostic on the 1,464-hour validation window, so it reflects genuine forecast-error propagation under recursion rather than model misspecification; it does not indicate that daily/weekly seasonality is uncaptured, which the strong `lag_24/lag_168` feature importance already confirms it is.

```
acf(xgb_val_df$value - pred_val_xgb, lag.max = 24*7)
```

Series xgb_val_df\$value - pred_val_xgb

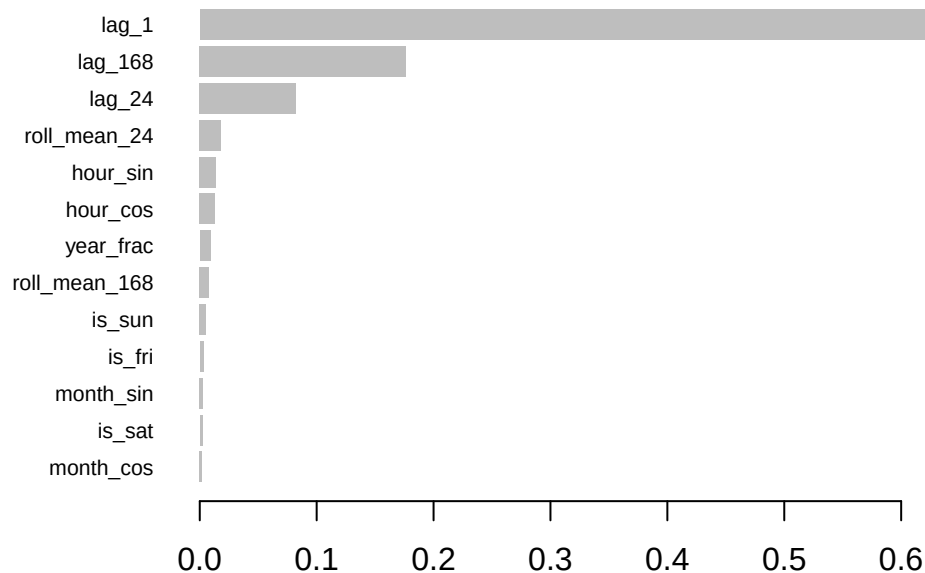


The five outlier days identified in the EDA (2016-2017) fall entirely within the training period and do not appear in the Nov-Dec 2019 validation window used for XGBoost’s residual diagnostics, so a direct comparison is not applicable here (`mean_abs_outlier_days` = NaN by construction). The overall validation residual (19.33) matches the reported MAE, confirming consistency between the two diagnostics. A parallel check on XGBoost’s in-sample training residuals would be needed for a like-for-like comparison with ARIMA and UCM

```
resid_xgb_df <- tibble(
  time = xgb_val_df$time,
  resid = xgb_val_df$value - pred_val_xgb
) %>% mutate(day = as_date(time), is_outlier_day = day %in% outlier_days_eda)
outlier_resid_summary(resid_xgb_df)
```

```
## # A tibble: 1 x 3
##   mean_abs_all mean_abs_outlier_days pct_outlier_days_in_top
##         <dbl>             <dbl>             <dbl>
## 1         19.3               NaN                 0
```

```
p_xgb_importance <- xgb.importance(feature_names = feature_cols, model = fit_xgb)
xgb.plot.importance(p_xgb_importance, top_n = 13)
```



```
print(p_xgb_importance)
```

##	Feature	Gain	Cover	Frequency
##	<char>	<num>	<num>	<num>
## 1:	lag_1	0.663460290	0.26445388	0.13924243
## 2:	lag_168	0.176470757	0.16679658	0.11961078
## 3:	lag_24	0.082358101	0.14825269	0.12926538
## 4:	roll_mean_24	0.018226078	0.06902048	0.11904175
## 5:	hour_sin	0.014023195	0.05822925	0.10032056
## 6:	hour_cos	0.012904417	0.07845936	0.06782876
## 7:	year_frac	0.008990810	0.06869234	0.10703515
## 8:	roll_mean_168	0.008098755	0.05550408	0.10236907
## 9:	is_sun	0.005289351	0.01351683	0.01652093
## 10:	is_fri	0.003081453	0.01757727	0.01420686
## 11:	month_sin	0.002795678	0.02648294	0.03962368
## 12:	is_sat	0.002217246	0.01013188	0.01151344
## 13:	month_cos	0.002083869	0.02288243	0.03342122

lag_1 dominates feature importance by a wide margin (Gain = 0.65), followed by lag_168 (0.16) and lag_24 (0.10). This numerically confirms what the lag plots in the EDA had shown visually: same-hour, same-weekday values one week prior (lag_168) carry nearly as much predictive information as same-hour values from the previous day (lag_24). Cyclical encodings (hour_sin/cos) and the trend term (year_frac) contribute comparatively little, suggesting the model relies primarily on recent persistence rather than an explicit seasonal shape — a plausible explanation for why it generalises better to the January-February forecast than the two seasonally-parametric models.

Diagnostics and Recap

```
outlier_days_eda <- as_date(c("2016-07-17", "2017-08-19", "2016-08-20",
                             "2017-02-26", "2017-02-27"))

outlier_resid_summary <- function(df) {
  n_top <- max(1, round(0.001 * nrow(df)))
  top <- df %>% slice_max(abs(resid), n = n_top)
  tibble(
    mean_abs_all = mean(abs(df$resid), na.rm = TRUE),
    mean_abs_outlier_days = df %>% filter(is_outlier_day) %>%
      summarise(m = mean(abs(resid), na.rm = TRUE)) %>% pull(m),
    pct_outlier_days_in_top = mean(top$is_outlier_day, na.rm = TRUE)
  )
}

resid_arima_df <- tibble(
  time = arima_train_df$time,
  resid = as.numeric(residuals(fit_arima))
) %>% mutate(day = as_date(time), is_outlier_day = day %in% outlier_days_eda)
outlier_resid_summary(resid_arima_df)
```

```
## # A tibble: 1 x 3
##   mean_abs_all mean_abs_outlier_days pct_outlier_days_in_top
##         <dbl>             <dbl>             <dbl>
## 1         0.463             0.605             0
```

```
resid_ucm_df <- tibble(
  time = ucm_train_df$time[seq_along(resid_ucm)],
  resid = as.numeric(resid_ucm)
) %>% mutate(day = as_date(time), is_outlier_day = day %in% outlier_days_eda)
outlier_resid_summary(resid_ucm_df)
```

```
## # A tibble: 1 x 3
##   mean_abs_all mean_abs_outlier_days pct_outlier_days_in_top
##         <dbl>             <dbl>             <dbl>
## 1         0.401             0.448             0
```

```
resid_xgb_df <- tibble(
  time = xgb_val_df$time,
  resid = xgb_val_df$value - pred_val_xgb
) %>% mutate(day = as_date(time), is_outlier_day = day %in% outlier_days_eda)
outlier_resid_summary(resid_xgb_df)
```

```
## # A tibble: 1 x 3
##   mean_abs_all mean_abs_outlier_days pct_outlier_days_in_top
##         <dbl>             <dbl>             <dbl>
## 1         19.3             NaN             0
```

On the validation set (Nov-Dec 2019), XGBoost achieves the lowest error (MAE=19.33, RMSE=32.67), followed by ARIMA (MAE=21.19, RMSE=33.39). UCM trails both (MAE=25.82, RMSE=38.39): its fixed local-level trend and empirical (non-stochastic) weekly and annual offsets adapt less flexibly to the validation window than ARIMA's cross-validated Fourier structure or XGBoost's direct lag features.

The validation window (Nov-Dec 2019) captures the annual trough of the series, while the actual forecast target (Jan-Feb 2020) spans the beginning of the annual rise toward the March/August peaks. Because

UCM relies only on the empirical monthly offset for the annual component and its local-level trend absorbs little of the recent upward movement, it still noticeably underforecasts January and February relative to historical averages (UCM: 23.8/38.4 vs a historical range of 54-67/80-117). ARIMA, with cross-validated Fourier terms and day-type dummies, forecasts closer to the historical range (44.2/86.2) than in earlier iterations, though still somewhat below it, reflecting the annual harmonics being only weakly identified from ~4.5 observed cycles. XGBoost produces forecasts closest to the historical range (44.7/97.8), consistent with relying on direct lag features (particularly lag-168, the strongest single predictor by Gain) rather than an explicit parametric seasonal form. XGBoost is therefore the preferred model on both criteria examined: it achieves the lowest validation error among the three candidates and the most plausible annual-cycle forecast, supporting its choice as the primary machine-learning submission for the missing period.

```
metrics_summary <- tibble(
  model = c("ARIMA", "UCM", "XGBoost"),
  MAE = c(mae_arima, mae_ucm, mae_xgb),
  RMSE = c(rmse_arima, rmse_ucm, rmse_xgb)
) %>%
  mutate(across(c(MAE, RMSE), ~round(., 2)))

print(metrics_summary)

## # A tibble: 3 x 3
##   model    MAE RMSE
##   <chr>   <dbl> <dbl>
## 1 ARIMA    21.2  33.4
## 2 UCM      25.8  38.4
## 3 XGBoost  19.3  32.7

p_metrics <- metrics_summary %>%
  pivot_longer(cols = c(MAE, RMSE), names_to = "metric", values_to = "value") %>%
  ggplot(aes(x = reorder(model, value), y = value, fill = model)) +
  geom_col() +
  facet_wrap(~metric, scales = "free_x") +
  coord_flip() +
  labs(title = "Model comparison on validation set (Nov-Dec 2019)",
       x = "", y = "") +
  theme_minimal() +
  theme(legend.position = "none")

p_metrics
```

Model comparison on validation set (Nov–Dec 2019)



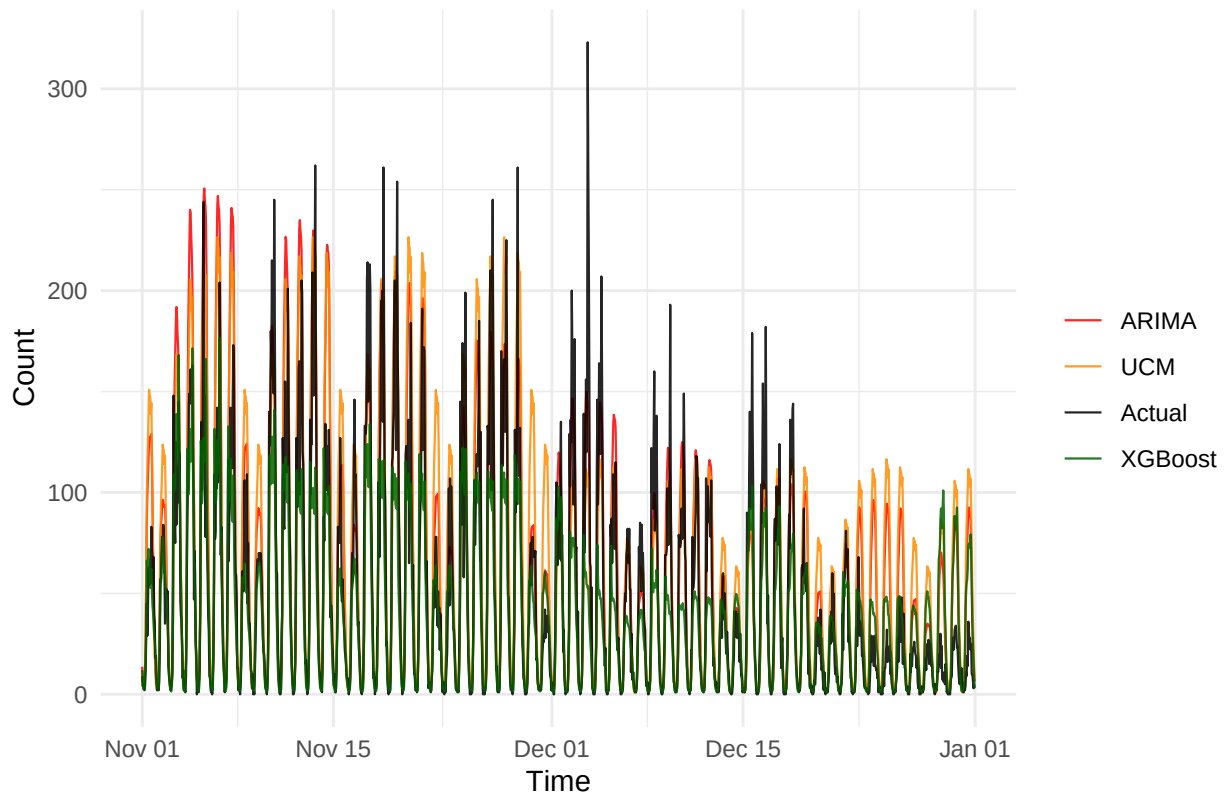
```
ggsave("slides_metrics_comparison.png", p_metrics, width = 10, height = 5, dpi = 300)
```

```
val_all_models <- val_df %>%
  select(time, value) %>%
  mutate(ARIMA = pred_val_arima, UCM = pred_val_ucm, XGBoost = pred_val_xgb) %>%
  pivot_longer(cols = c(value, ARIMA, UCM, XGBoost), names_to = "series", values_to = "count")

p_val_all <- ggplot(val_all_models, aes(x = time, y = count, color = series)) +
  geom_line(linewidth = 0.4, alpha = 0.85) +
  scale_color_manual(
    values = c("value" = "black", "ARIMA" = "red", "UCM" = "darkorange", "XGBoost" = "darkgreen"),
    labels = c("value" = "Actual", "ARIMA" = "ARIMA", "UCM" = "UCM", "XGBoost" = "XGBoost")
  ) +
  labs(title = "ARIMA vs UCM vs XGBoost vs Actual (validation set)",
       x = "Time", y = "Count", color = "") +
  theme_minimal()

p_val_all
```

ARIMA vs UCM vs XGBoost vs Actual (validation set)



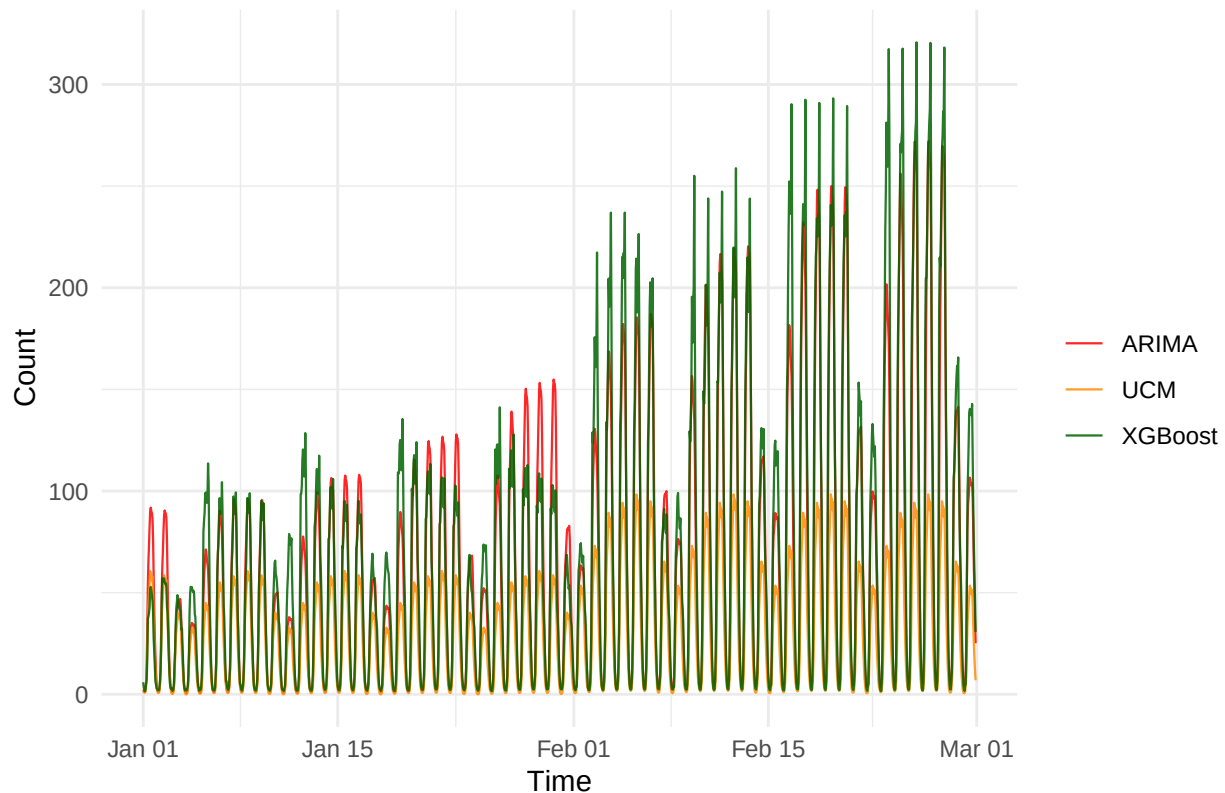
```
ggsave("slides_all_models_validation.png", p_val_all, width = 11, height = 5, dpi = 300)
```

All three models reproduce the general daily/weekly shape of the actual series, but differ in how closely they track its finer structure: XGBoost (green) follows the weekly modulation most closely, ARIMA (red) captures the overall level accurately but with a smoother profile, and UCM (orange) reproduces the weekly modulation after the offset correction, though still with a smoother/less peaked profile than XGBoost.

```
p_test_all <- bind_rows(
  arima_forecast_final %>% rename(value = value_forecast_arima) %>% mutate(model = "ARIMA"),
  ucm_forecast_final   %>% rename(value = value_forecast_ucm)   %>% mutate(model = "UCM"),
  xgb_forecast_final   %>% rename(value = value_forecast_xgb)   %>% mutate(model = "XGBoost")
) %>%
  ggplot(aes(x = time, y = value, color = model)) +
  geom_line(linewidth = 0.4, alpha = 0.85) +
  scale_color_manual(values = c("ARIMA" = "red", "UCM" = "darkorange", "XGBoost" = "darkgreen")) +
  labs(title = "ARIMA vs UCM vs XGBoost: final forecast comparison (Jan-Feb 2020)",
       x = "Time", y = "Count", color = "") +
  theme_minimal()
```

```
p_test_all
```

ARIMA vs UCM vs XGBoost: final forecast comparison (Jan–Feb 2020)



```
ggsave("slides_all_models_final_forecast.png", p_test_all, width = 11, height = 5, dpi = 300)
```

The three forecasts diverge most visibly in their treatment of the annual cycle: XGBoost (green) shows a clear rise from December into February, ARIMA (red) remains comparatively flat, and UCM (orange) sits between the two but still below the historical range. This divergence is the central finding of the modelling comparison and is discussed in detail below.

```
monthly_forecast_comparison <- bind_rows(
  arima_forecast_final %>% rename(value = value_forecast_arima) %>% mutate(model = "ARIMA"),
  ucm_forecast_final   %>% rename(value = value_forecast_ucm)   %>% mutate(model = "UCM"),
  xgb_forecast_final   %>% rename(value = value_forecast_xgb)   %>% mutate(model = "XGBoost")
) %>%
  mutate(month = month(time)) %>%
  group_by(model, month) %>%
  summarise(mean_forecast = mean(value), .groups = "drop")

print(monthly_forecast_comparison)
```

```
## # A tibble: 6 x 3
##   model    month mean_forecast
##   <chr>    <dbl>         <dbl>
## 1 ARIMA      1          44.2
## 2 ARIMA      2          86.2
## 3 UCM        1          23.8
## 4 UCM        2          38.3
## 5 XGBoost    1          44.7
## 6 XGBoost    2          97.8
```

```
print(jan_feb_historical)
```

```
## # A tibble: 8 x 4
##   year month mean_value sd_value
##   <dbl> <dbl>     <dbl>    <dbl>
## 1  2016     1      55.5      53.5
## 2  2016     2     103.      101.
## 3  2017     1      66.6      62.0
## 4  2017     2     117.      119.
## 5  2018     1      59.6      62.1
## 6  2018     2     112.      118.
## 7  2019     1      54.1      53.2
## 8  2019     2      80.1      81.3
```

```
cat("=== FINAL RECAP ===\n\n")
```

```
## === FINAL RECAP ===
```

```
cat("ARIMA   - MAE:", round(mae_arima, 2), " RMSE:", round(rmse_arima, 2), "\n")
```

```
## ARIMA   - MAE: 21.19  RMSE: 33.39
```

```
cat("UCM     - MAE:", round(mae_ucm, 2), " RMSE:", round(rmse_ucm, 2), "\n")
```

```
## UCM     - MAE: 25.82  RMSE: 38.39
```

```
cat("XGBoost - MAE:", round(mae_xgb, 2), " RMSE:", round(rmse_xgb, 2), "\n\n")
```

```
## XGBoost - MAE: 19.33  RMSE: 32.67
```

```
cat("Best model on validation:",
    metrics_summary$model[which.min(metrics_summary$MAE[-1]) + 1], "\n")
```

```
## Best model on validation: XGBoost
```

XGBoost is the strongest candidate on both criteria that matter for this task: it has the lowest validation error of the three models (MAE=19.33 vs ARIMA's 21.19 and UCM's 25.82) and produces the January-February forecast closest to the historical range (44.7/97.8 vs a historical range of 54-67/80-117), ahead of ARIMA (44.2/86.2) and UCM (23.8/38.4). This reflects its reliance on direct lag features — particularly lag-168 — rather than a parametric annual seasonal form that both ARIMA and UCM must estimate from only ~4-5 observed cycles. XGBoost's forecast is therefore used as the primary submission for the missing period, with ARIMA's forecast as a close secondary reference.

Export

```
final_predictions <- arima_forecast_final %>%
  rename(ARIMA = value_forecast_arima) %>%
  left_join(ucm_forecast_final %>% rename(UCM = value_forecast_ucm), by = "time") %>%
  left_join(xgb_forecast_final %>% rename(ML = value_forecast_xgb), by = "time") %>%
  select(time, ARIMA, UCM, ML) %>%
  arrange(time) %>%
  mutate(time = format(time, "%Y-%m-%d %H:%M:%S"))

cat("Rows in final predictions:", nrow(final_predictions), " (should be 1439)\n")
```

```
## Rows in final predictions: 1439 (should be 1439)
```

```
cat("Any NA values:", sum(is.na(final_predictions)), "\n")
```

```
## Any NA values: 0
```

```
head(final_predictions)
```

```
##           time      ARIMA      UCM      ML
## 1 2020-01-01 00:00:01 2.097439 2.2036546 5.917852
## 2 2020-01-01 01:00:01 1.767621 1.0410534 3.528050
## 3 2020-01-01 02:00:01 1.216692 0.6846250 1.861776
## 4 2020-01-01 03:00:01 1.320729 0.7607782 1.481182
## 5 2020-01-01 04:00:01 2.408776 1.5145416 2.489021
## 6 2020-01-01 05:00:01 5.945392 4.4407064 4.995489
```

```
write.csv(final_predictions, "858423_20260910.csv", row.names = FALSE)
```